ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



BÁO CÁO

ĐỒ ÁN TỐT NGHIỆP

Hệ thống quản lý các hoạt động Đoàn - Hội và hỏi đáp hoạt động ngoại khóa Trường Đại học Bách khoa

NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: .....

GVHD: ThS. Trần Ngọc Bảo Duy

GVPB: .....

—o0o—

SVTH1: Nguyễn Phương Duy - 2210526

SVTH2: Nguyễn Ngọc Quý - 2212870

SVTH3: Trần Quốc Toàn - 2213540

TP. HỒ CHÍ MINH, 05/2026

Lời cảm ơn

Trong quá trình thực hiện đồ án tốt nghiệp, chúng tôi nhận được sự hỗ trợ đặc biệt từ nhiều nguồn. Đầu tiên và quan trọng nhất, nhóm xin bày tỏ lòng biết ơn sâu sắc và chân thành đối với **ThS. Trần Ngọc Bảo Duy** – Giảng viên Bộ môn Khoa học Máy tính, người Thầy đã trực tiếp hướng dẫn, định hướng đề tài và tận tình chỉ bảo, giúp đỡ chúng tôi tháo gỡ những khó khăn trong suốt thời gian thực hiện và hoàn thành đồ án tốt nghiệp *"Hệ thống quản lý các hoạt động Đoàn - Hội và hỏi đáp hoạt động ngoại khóa Trường Đại học Bách khoa"*.

Chúng tôi xin chân thành cảm ơn **Trường Đại học Bách khoa (ĐHQG-HCM)** và Quý Thầy Cô thuộc **Khoa Khoa học và Kỹ thuật Máy tính** đã tận tình truyền đạt những kiến thức nền tảng quý báu, tạo điều kiện thuận lợi và môi trường học tập tốt nhất để chúng tôi có đủ hành trang thực hiện đồ án này.

Cuối cùng, chúng tôi xin cảm ơn gia đình, bạn bè và những người thân đã luôn động viên, ủng hộ, chia sẻ và là chỗ dựa tinh thần vững chắc giúp chúng tôi tập trung nghiên cứu và hoàn thành báo cáo đúng hạn.

Mặc dù đã nỗ lực hết mình, nhưng do kiến thức và thời gian có hạn, đồ án khó tránh khỏi những thiếu sót. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu của Thầy để đồ án ngày càng hoàn thiện hơn.

Chúng tôi xin chân thành cảm ơn!

Lời cam đoan

Chúng tôi xin cam đoan rằng hệ thống mà chúng tôi triển khai là độc lập và chỉ thuộc sở hữu của nhóm tôi, đồng thời được hướng dẫn bởi ThS. Trần Ngọc Bảo Duy. Các nội dung nghiên cứu và thiết kế đề tài này là trung thực, không sao chép hoặc sử dụng kết quả từ bất kỳ đề tài nào khác có tính tương tự.

Chúng tôi cam kết chịu trách nhiệm hoàn toàn về nội dung đồ án tốt nghiệp của mình và sẵn sàng chấp nhận mọi hình thức kỷ luật nếu phát hiện bất kỳ hành vi gian lận nào. Chúng tôi sẽ tuân thủ mọi quy tắc và quy định được đặt ra, và sẵn lòng chịu trách nhiệm trước Ban chủ nhiệm Khoa và Ban giám hiệu của nhà trường.

Mục lục

Danh sách hình vẽ

Danh sách bảng

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt	Diễn giải
CTXH	Công tác xã hội
API	Application Programming Interface (giao diện lập trình ứng dụng)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control (kiểm soát truy cập theo vai trò)
JWT	JSON Web Token
AI	Artificial Intelligence (trí tuệ nhân tạo)
UI	User Interface (giao diện người dùng)
DB	Database (cơ sở dữ liệu)
S3	Simple Storage Service (dịch vụ lưu trữ đối tượng)
HTTP/HTTPS	HyperText Transfer Protocol / HyperText Transfer Protocol Secure
SSE	Server-Sent Events
CRUD	Create, Read, Update, Delete

Bảng 1: Danh mục từ viết tắt sử dụng trong báo cáo

Thuật ngữ	Diễn giải ngắn
Ingestion	Quá trình nạp và đồng bộ dữ liệu nguồn vào hệ thống truy xuất ngữ nghĩa
Embedding	Biểu diễn vector của văn bản để phục vụ truy vấn tương đồng ngữ nghĩa
Vector Store	Kho dữ liệu lưu các vector embedding và metadata liên quan
Session	Phiên làm việc/hội thoại của người dùng trong một khoảng thời gian
Webhook	Cơ chế callback HTTP để hệ thống ngoài chủ động gửi trạng thái về backend
Feature Flag	Cờ cấu hình dùng bật/tắt tính năng theo môi trường hoặc nhóm người dùng

Bảng 2: Danh mục thuật ngữ kỹ thuật chính

Chương 1 Tổng quan đề tài

1.1 Giới thiệu

1.1.1 Đặt vấn đề

Trong bối cảnh giáo dục đại học tại Việt Nam, Trường Đại học Bách Khoa – Đại học Quốc gia TP.HCM là một trong những cơ sở đào tạo có quy mô lớn, với hơn 27.000 sinh viên đang theo học. Quy mô này phản ánh vai trò dẫn dắt của nhà trường trong đào tạo, đồng thời cũng đặt ra yêu cầu cao đối với công tác tổ chức, quản lý và hỗ trợ sinh viên, đặc biệt là trong các hoạt động của Đoàn Thanh niên và Hội Sinh viên.

Tuy nhiên, thực tế triển khai cho thấy công tác quản lý Đoàn – Hội hiện nay vẫn còn phụ thuộc nhiều vào phương thức thủ công và các kênh lưu trữ rời rạc. Dữ liệu liên quan đến người dùng, hoạt động và tài chính chưa được tập trung trên một hệ thống thống nhất mà thường phân tán trong hồ sơ giấy, bảng tính Excel hoặc các tệp Word lưu trữ cục bộ trên máy tính cá nhân của cán bộ. Cách tổ chức này làm cho việc tra cứu thông tin, tổng hợp báo cáo, theo dõi lịch sử tham gia và đối soát dữ liệu trở nên tốn nhiều thời gian, dễ sai sót và khó mở rộng.

Từ quá trình khảo sát thực tế tại Đoàn trường và Hội Sinh viên, có thể nhận thấy ba nhóm vấn đề cốt lõi cần được giải quyết:

1. **Quản lý người dùng và tổ chức**: sinh viên chưa có một cơ chế định danh số hóa xuyên suốt; việc xác minh tư cách đoàn viên hoặc hội viên còn phụ thuộc nhiều vào sổ đoàn, hồ sơ giấy và biên lai thủ công.

2. **Quản lý hoạt động công tác xã hội**: thông tin về các chương trình, phong trào và sự kiện hiện đang được đăng tải trên nhiều kênh khác nhau, gây phân tán dữ liệu, khó theo dõi và thiếu công cụ thống kê tự động.

3. **Quản lý tài chính**: quy trình thu – chi vẫn còn dựa nhiều vào tiền mặt và các khâu trung gian, tiềm ẩn rủi ro thất thoát, sai lệch số liệu và thiếu minh bạch trong đối soát.

Ngoài ba nhóm vấn đề trên, còn có một khoảng trống đáng chú ý về khả năng tra cứu thông tin và hỗ trợ người dùng. Trong thực tế, sinh viên thường phải tự tìm hoạt động phù hợp qua nhiều kênh truyền thông khác nhau, trong khi cán bộ lại mất nhiều thời gian để trả lời lặp lại các câu hỏi phổ biến về quy định, điều kiện tham gia hoặc cách thức thực hiện thủ tục. Điều này cho thấy nhu cầu không chỉ nằm ở việc lưu trữ dữ liệu, mà còn ở việc tổ chức lại quy trình tiếp cận thông tin theo hướng trực quan, tập trung và có khả năng hỗ trợ tự động.

Những hạn chế trên không chỉ làm giảm hiệu quả vận hành của tổ chức Đoàn – Hội mà còn ảnh hưởng trực tiếp đến trải nghiệm và quyền lợi của sinh viên. Vì vậy, việc xây dựng một giải pháp chuyển đổi số có tính hệ thống, tập trung và có khả năng mở rộng là yêu cầu cấp thiết.

1.1.2 Các hướng giải quyết liên quan

Xuất phát từ các vấn đề đã phân tích, có thể thấy rằng việc cải thiện công tác quản lý Đoàn – Hội không thể chỉ dựa trên các biện pháp cục bộ, mà cần một nền tảng số hóa thống nhất để xử lý đồng thời dữ liệu người dùng, hoạt động và tài chính. Hướng tiếp cận này giúp chuẩn hóa quy trình, giảm thao tác thủ công và tạo cơ sở cho việc khai thác dữ liệu hiệu quả hơn trong dài hạn.

Đối với bài toán quản lý người dùng và tổ chức, hướng giải quyết phù hợp là xây dựng hồ sơ điện tử cho từng đoàn viên, hội viên và sinh viên. Dữ liệu định danh được lưu trữ tập trung sẽ hỗ trợ xác minh nhanh tư cách tham gia, theo dõi lịch sử hoạt động và giảm phụ thuộc vào hồ sơ giấy truyền thống.

Đối với bài toán quản lý hoạt động công tác xã hội, cần tập trung hóa thông tin chương trình, phong trào và sự kiện trên một cổng thông tin duy nhất. Khi dữ liệu được chuẩn hóa và đồng bộ, sinh viên có thể dễ dàng tìm kiếm, đăng ký và theo dõi hoạt động; đồng thời cán bộ có thể thống kê, đánh giá và tổng hợp báo cáo theo từng giai đoạn một cách thuận tiện hơn.

Đối với bài toán quản lý tài chính, hướng giải quyết là số hóa quy trình thu – chi và từng bước giảm sự phụ thuộc vào tiền mặt. Việc ghi nhận, đối soát và truy xuất giao dịch trên hệ thống điện tử giúp tăng tính minh bạch, giảm rủi ro thất thoát, hạn chế sai lệch số liệu và giảm áp lực cho các cán bộ trung gian trong quá trình thu nộp.

Trên cơ sở đó, đề tài hướng đến việc xây dựng một cổng thông tin điện tử tập trung cho hoạt động Đoàn – Hội, đóng vai trò là nền tảng quản lý thống nhất cho dữ liệu người dùng, hoạt động và tài chính. Bên cạnh đó, việc tích hợp AI Chatbot hỗ trợ hỏi đáp tự động được kỳ vọng sẽ giúp sinh viên tiếp cận nhanh các thông tin thuộc phạm vi quản lý hoạt động công tác xã hội, đồng thời giảm tải công việc tra cứu và phản hồi cho cán bộ phụ trách.

Từ góc độ nghiên cứu và triển khai, đề tài không chỉ giải quyết bài toán quản lý dữ liệu đơn lẻ mà còn hướng đến việc kết nối các phân hệ nghiệp vụ vào cùng một hệ sinh thái số. Cách tiếp cận này cho phép dữ liệu phát sinh ở một phân hệ có thể được tái sử dụng ở các phân hệ khác, ví dụ như hồ sơ người dùng phục vụ phân quyền, dữ liệu hoạt động phục vụ chấm điểm CTXH, hoặc dữ liệu đã quản lý phục vụ chatbot tra cứu. Đây là điểm then chốt giúp hệ thống có giá trị sử dụng thực tế và có khả năng mở rộng về sau.

1.2 Mục tiêu

Mục tiêu của đề tài là xây dựng một hệ thống cổng thông tin Đoàn – Hội tập trung, hỗ trợ quản lý đồng bộ người dùng, hoạt động công tác xã hội và tài chính trên cùng một nền tảng. Thông qua việc số hóa và chuẩn hóa dữ liệu, hệ thống hướng đến việc giảm thao tác thủ công, tăng tính minh bạch trong quản lý và cải thiện hiệu quả vận hành cho cán bộ Đoàn – Hội.

Cụ thể, đề tài nhắm đến các mục tiêu sau:

- Xây dựng cơ chế quản lý tập trung thông tin đoàn viên, hội viên và sinh viên, phục vụ cho việc xác minh tư cách, tra cứu và theo dõi lịch sử tham gia hoạt động.
- Tập trung hóa và chuẩn hóa thông tin các hoạt động công tác xã hội trên một cổng thông tin duy nhất, giúp sinh viên dễ dàng tiếp cận, đăng ký và theo dõi hoạt động phù hợp.
- Hỗ trợ số hóa quy trình thu – chi và lưu trữ dữ liệu tài chính ở mức quản lý nghiệp vụ, nhằm tăng khả năng truy xuất, đối soát và kiểm soát số liệu.
- Hiện thực AI Chatbot cho công tác xã hội, cho phép tự động trả lời các câu hỏi về quy định CTXH, tra cứu thông tin hoạt động và gợi ý hoạt động phù hợp trong phạm vi dữ liệu đã được quản lý trên hệ thống.
- Tạo nền tảng để mở rộng các dịch vụ số khác trong tương lai, góp phần hiện đại hóa công tác quản lý Đoàn – Hội theo hướng đồng bộ và bền vững.

Bên cạnh các mục tiêu trên, đề tài còn hướng đến việc nâng cao trải nghiệm của sinh viên khi tiếp cận thông tin, đồng thời giảm khối lượng công việc tra cứu và phản hồi cho cán bộ phụ trách. Đây là cơ sở để hệ thống không chỉ đáp ứng nhu cầu quản lý trước mắt mà còn có thể phát triển thành một nền tảng số hỗ trợ hoạt động Đoàn – Hội lâu dài.

1.3 Phạm vi đề tài

Đề tài tập trung vào phạm vi xây dựng một hệ thống phần mềm phục vụ cho công tác quản lý Đoàn – Hội tại Trường Đại học Bách Khoa – Đại học Quốc gia TP.HCM. Phạm vi chức năng bao gồm quản lý thông tin người dùng, quản lý hoạt động công tác xã hội, quản lý các nội dung liên quan đến tài chính ở mức hỗ trợ nghiệp vụ, và tích hợp chatbot để hỗ trợ tra cứu thông tin trong phạm vi hoạt động công tác xã hội.

Về mặt triển khai, đề tài không đi sâu vào thay thế hoàn toàn các quy trình nghiệp vụ hiện có trong mọi đơn vị, mà tập trung vào việc xây dựng nền tảng trung tâm có thể áp dụng trước trong các tình huống quản lý phổ biến. Các nội dung như tích hợp sâu với hệ thống thanh toán bên thứ ba, chuẩn hóa toàn bộ dữ liệu lịch sử hoặc đồng bộ với tất cả quy trình nội bộ sẽ được xem là hướng phát triển mở rộng trong các giai đoạn tiếp theo.

1.4 Kế hoạch

Đề tài được triển khai theo các bước chính như sau:

Bước 1. Khảo sát và phân tích yêu cầu

Trong giai đoạn này, nhóm thực hiện khảo sát nghiệp vụ thực tế tại Đoàn trường và Hội Sinh viên, từ đó xác định các nhu cầu chính liên quan đến quản lý người dùng, hoạt động công tác xã hội và tài chính. Kết quả của bước này là bộ yêu cầu chức năng và phi chức năng làm cơ sở cho các giai đoạn tiếp theo.

Việc khảo sát không chỉ nhằm liệt kê chức năng cần có mà còn để nhận diện những điểm nghẽn trong quy trình hiện tại, mức độ lặp lại của các thao tác và thứ tự ưu tiên của từng nhu cầu nghiệp vụ. Nhờ đó, đề tài có thể tập trung vào đúng các vấn đề có ảnh hưởng lớn nhất đến hiệu quả quản lý thay vì dàn trải trên nhiều tình huống ít giá trị hơn.

Bước 2. Thiết kế kiến trúc và dữ liệu

Sau khi hoàn thành khảo sát, nhóm tiến hành xây dựng kiến trúc tổng thể của hệ thống, bao gồm phân rã các phân hệ chức năng, thiết kế cơ sở dữ liệu và xác định các luồng xử lý chính. Ở bước này, cấu trúc dữ liệu cho các hoạt động công tác xã hội và các thành phần liên quan đến chatbot cũng được xác định để bảo đảm khả năng tích hợp sau này.

Đây là bước chuyển từ nhu cầu nghiệp vụ sang mô hình triển khai cụ thể. Nếu kiến trúc và dữ liệu được thiết kế rõ ngay từ đầu, hệ thống sẽ hạn chế được việc chỉnh sửa lớn trong giai đoạn hiện thực, đồng thời tạo ra nền tảng đủ ổn định để mở rộng thêm các tính năng mới ở những giai đoạn tiếp theo.

Bước 3. Phát triển các chức năng cốt lõi

Trong giai đoạn này, nhóm hiện thực các chức năng nền tảng của hệ thống, bao gồm quản lý người dùng, quản lý hoạt động công tác xã hội và các chức năng hỗ trợ quản lý nghiệp vụ cơ bản. Mục tiêu của bước này là hoàn thiện phần lõi của cổng thông tin trước khi mở rộng sang các thành phần thông minh hơn.

Bước 4. Phát triển chatbot hỗ trợ CTXH

Sau khi các dữ liệu và chức năng quản lý hoạt động đã ổn định, nhóm tiếp tục phát triển AI Chatbot phục vụ tra cứu thông tin trong phạm vi hoạt động công tác xã hội. Chatbot được thiết kế để hỗ trợ trả lời các câu hỏi liên quan đến quy định, thông tin hoạt động và các nội dung tra cứu phổ biến dựa trên dữ liệu đã được quản lý trong hệ thống.

Bước 5. Kiểm tra và tối ưu

Sau khi hoàn thành các chức năng chính, nhóm tiến hành kiểm thử, rà soát lỗi và tối ưu hệ thống. Công việc ở bước này bao gồm kiểm tra tính đúng đắn của dữ liệu, đánh giá độ ổn định của các chức năng, tối ưu trải nghiệm sử dụng và tinh chỉnh chatbot để bảo đảm câu trả lời phù hợp với phạm vi CTXH.

Kết quả cuối cùng là một nền tảng cổng thông tin có thể hỗ trợ quản lý tập trung, đồng bộ và thuận tiện hơn cho công tác Đoàn – Hội, đồng thời có thêm thành phần chatbot hỗ trợ tra cứu thông tin trong phạm vi hoạt động công tác xã hội.

Chương 2 Kiến thức nền tảng

2.1 Khảo sát thực tế

Hiện nay, Trường Đại học Bách Khoa – Đại học Quốc gia TP.HCM đã có một cổng thông tin phục vụ cho hoạt động ngoại khóa và công tác xã hội, trong đó sinh viên có thể truy cập để xem thông báo, tra cứu hoạt động, xem thông tin cá nhân và đăng ký tham gia các hoạt động phù hợp. Phần khảo sát dưới đây được thực hiện dựa trên giao diện hiện tại của cổng này ở góc nhìn người dùng sinh viên, không phải từ mã nguồn của hệ thống đề xuất trong đề tài.

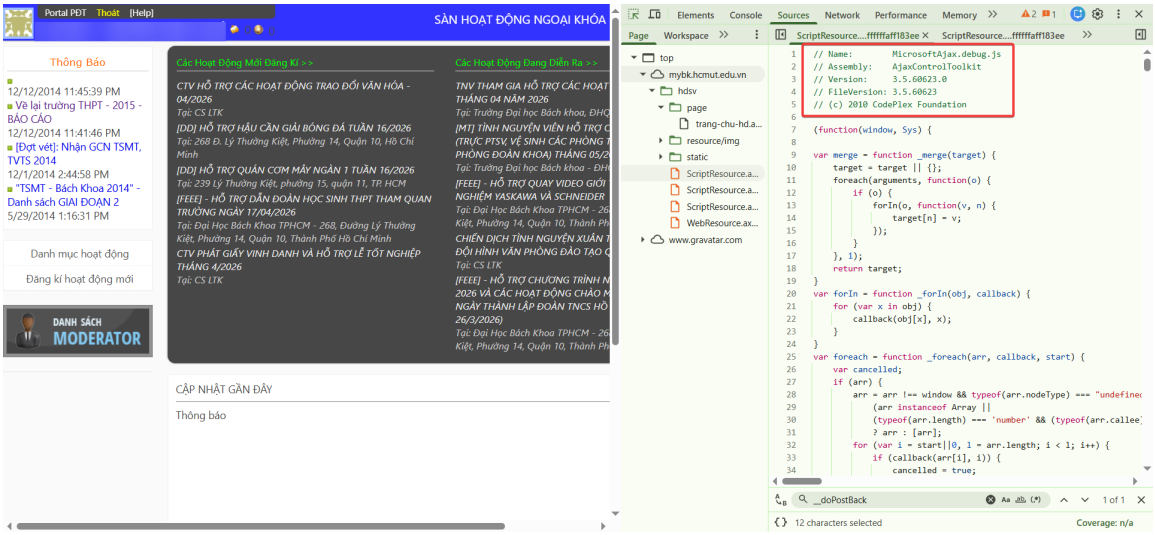
Từ quan sát giao diện hiện tại, có thể xác định hệ thống CTXH đang cần đáp ứng một số chức năng cơ bản sau:

- Công bố thông báo và nội dung cập nhật.
- Cung cấp danh sách hoạt động theo trạng thái.
- Hiển thị chi tiết hoạt động để sinh viên xem thông tin tổ chức, thời gian, địa điểm và nội dung.
- Cho phép sinh viên đăng ký hoặc theo dõi hoạt động.
- Hiển thị trang cá nhân để kiểm tra tổng số ngày CTXH tích lũy.
- Hỗ trợ khu vực dành cho đơn vị tổ chức đăng ký hoặc quản lý hoạt động mới.

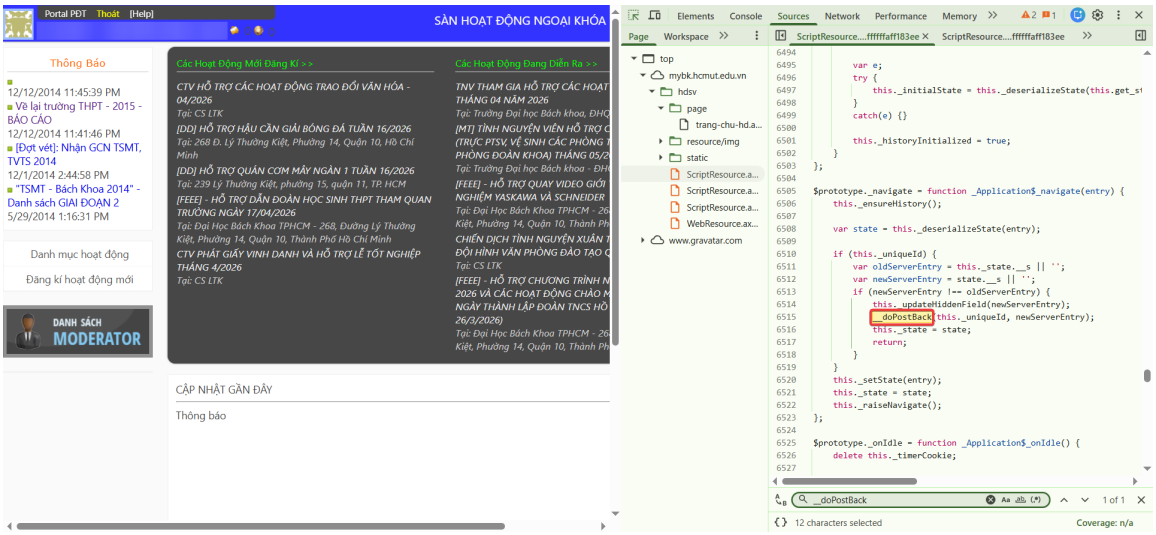
Đây là những chức năng cốt lõi mà một cổng thông tin CTXH cần có để phục vụ cả sinh viên lẫn cán bộ phụ trách.

2.1.1 Quan sát kỹ thuật từ phía người dùng

Thông qua quan sát bằng công cụ Developer Tools (F12) ở phía trình duyệt, có thể nhận thấy hệ thống hiện tại mang đặc trưng của nền tảng ASP.NET WebForms. Cụ thể, phía client sử dụng các script như `MicrosoftAjaxTimer.debug.js` thuộc `AjaxControlToolkit3.5.60623.0`, cùng cơ chế `__doPostBack` và các yêu cầu bất đồng bộ đến các trang `.aspx`.

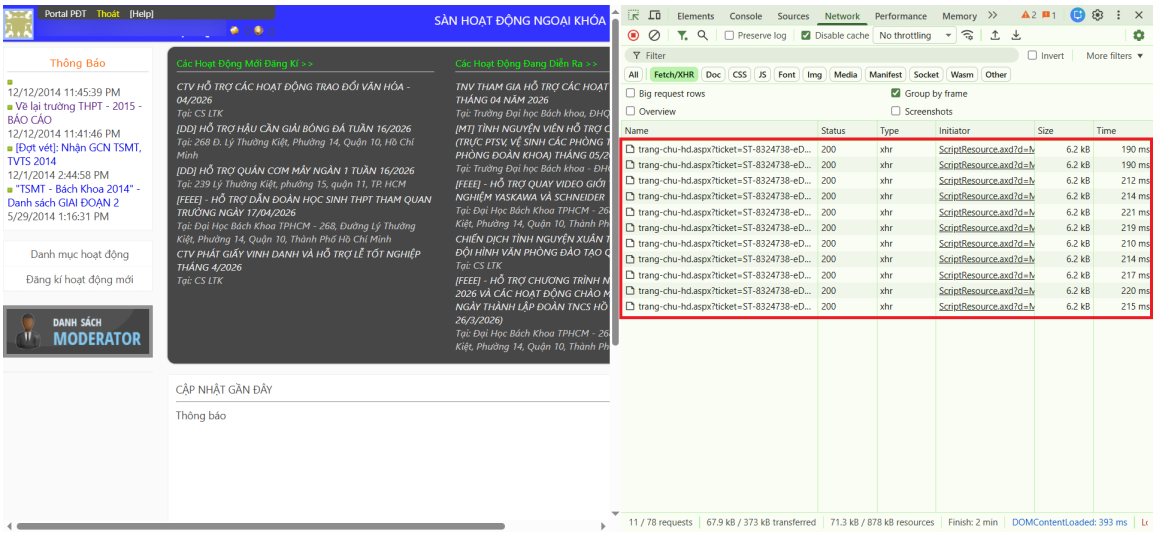


Hình 1: Quan sát script AjaxControlToolkit trong Developer Tools



Hình 2: Quan sát cơ chế `__doPostBack` trong hệ thống cũ

Ở lớp giao tiếp dữ liệu (Network), từ góc nhìn phía trình duyệt chưa ghi nhận API REST công khai theo kiểu tài nguyên (ví dụ `/api/...`); thay vào đó, các yêu cầu chủ yếu đi qua các endpoint trang `.aspx` và tài nguyên tĩnh. Đặc điểm này cho thấy hệ thống được thiết kế theo cách tiếp cận phù hợp với giai đoạn trước đây, đồng thời gợi ý nhu cầu chuẩn hóa giao tiếp dịch vụ nếu mở rộng đa kênh trong tương lai.



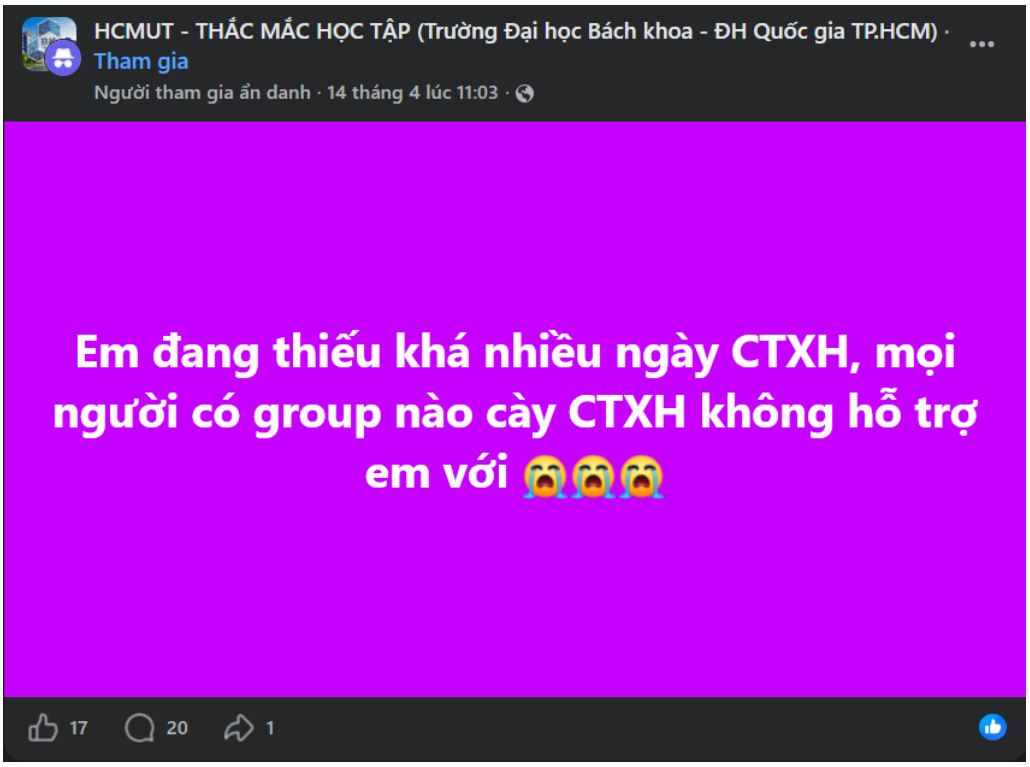
Hình 3: Quan sát lưu lượng network trong Developer Tools

2.1.2 Thực trạng mức độ sử dụng

Theo thực tế sử dụng hiện nay, cổng CTXH cũ vẫn còn giá trị nhất định ở chức năng tra cứu thông tin và theo dõi tổng số ngày CTXH cá nhân. Tuy nhiên, mức độ sử dụng cho các tác vụ tương tác thường xuyên không cao; nhiều hoạt động được truyền thông và tiếp nhận đăng ký chủ yếu qua các hội nhóm trên mạng xã hội, sau đó mới đối chiếu hoặc cập nhật lại trên hệ thống.

Thực trạng này phản ánh khoảng cách giữa chức năng hệ thống đang có và trải nghiệm người dùng mong đợi trong bối cảnh số, đặc biệt ở nhu cầu tiếp cận thông tin nhanh, đăng ký thuận tiện và theo dõi trạng thái tập trung.

Minh chứng thực tế cho thấy các hoạt động liên quan đến CTXH thường được lan truyền và tiếp nhận đăng ký thông qua mạng xã hội trước khi được đối chiếu lại trên hệ thống, thay vì người dùng truy cập trực tiếp cổng cũ để tìm kiếm thông tin.



Hình 4: Sinh viên tìm kiếm hoạt động CTXH trên Facebook



Hình 5: Thông tin tuyển tình nguyện viên CTXH được đăng trên Facebook

2.1.3 Giao diện thông tin và thông báo

Ở màn hình chính, hệ thống hiện tại cung cấp một khu vực thông báo ở cột bên trái, nơi sinh viên có thể theo dõi các nội dung cập nhật, bài viết liên quan đến hoạt động và một số thông báo đã lưu. Phần này đáp ứng tốt nhu cầu theo dõi thông tin tổng quan hằng ngày.



Hình 6: Khu vực thông báo trên giao diện cổng CTXH cũ

Từ góc độ khảo sát, phần thông báo đã thực hiện tốt vai trò công bố thông tin; điểm có thể cải thiện thêm là cơ chế lọc, tìm kiếm hoặc ưu tiên hiển thị nội dung mới để sinh viên theo dõi nhanh hơn các thông tin liên quan trực tiếp đến mình.

2.1.4 Danh mục và danh sách hoạt động

Khu vực danh mục hoạt động cho phép sinh viên xem các hoạt động theo trạng thái như đang diễn ra, đang mở đăng ký hoặc đã kết thúc. Khi chọn từng trạng thái, hệ thống hiển thị danh sách hoạt động tương ứng với tên hoạt động, đơn vị tổ chức, thời gian và địa điểm thực hiện. Nhìn chung, đây là chức năng quan trọng và đáp ứng đúng nhu cầu tra cứu cơ bản của sinh viên.

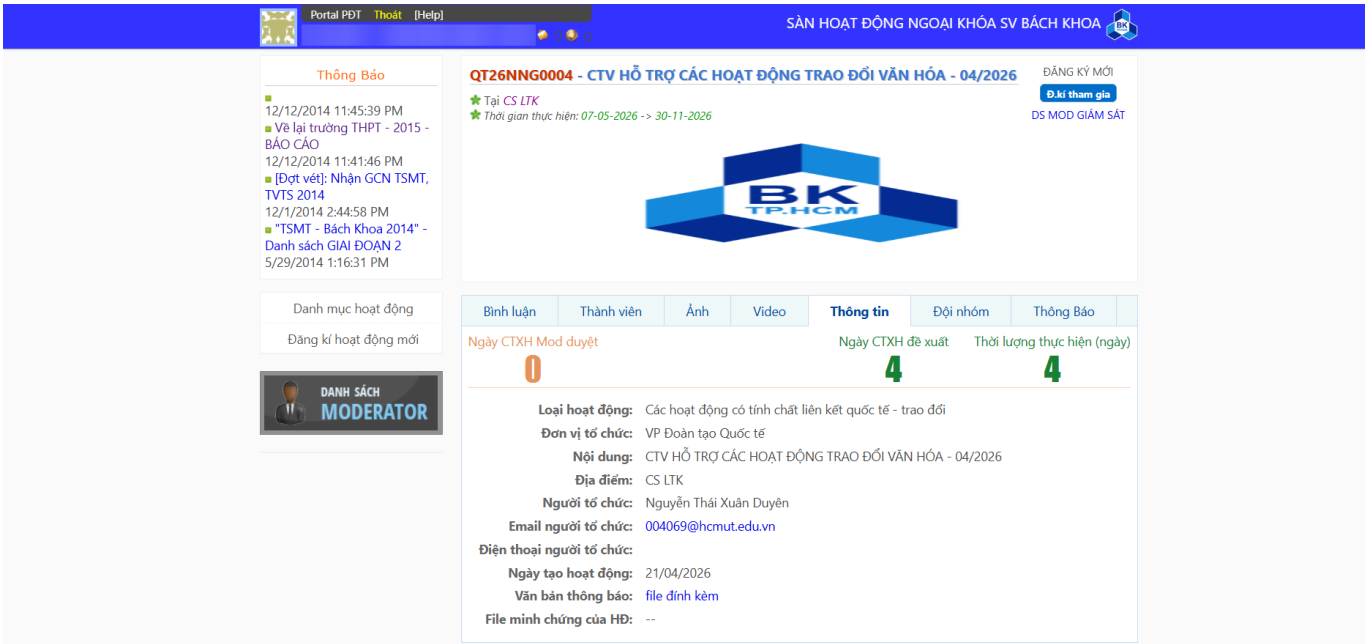


Hình 7: Danh mục và danh sách hoạt động trên cổng CTXH cũ

Khu vực này cho thấy hệ thống hiện tại đã đáp ứng tốt yêu cầu tra cứu cơ bản. Trong bối cảnh số lượng hoạt động ngày càng tăng, việc bổ sung thêm tìm kiếm, lọc nâng cao hoặc gợi ý theo nhu cầu cá nhân sẽ giúp trải nghiệm tra cứu thuận tiện hơn.

2.1.5 Chi tiết hoạt động và đăng ký tham gia

Khi truy cập vào từng hoạt động, sinh viên có thể xem trang chi tiết gồm tên hoạt động, banner, thời gian thực hiện, địa điểm, đơn vị tổ chức, nội dung mô tả và một số thông tin liên hệ. Một số màn hình còn thể hiện số ngày CTXH đề xuất, số ngày được duyệt và trạng thái của hoạt động.



Hình 8: Màn hình chi tiết hoạt động từ danh mục tra cứu

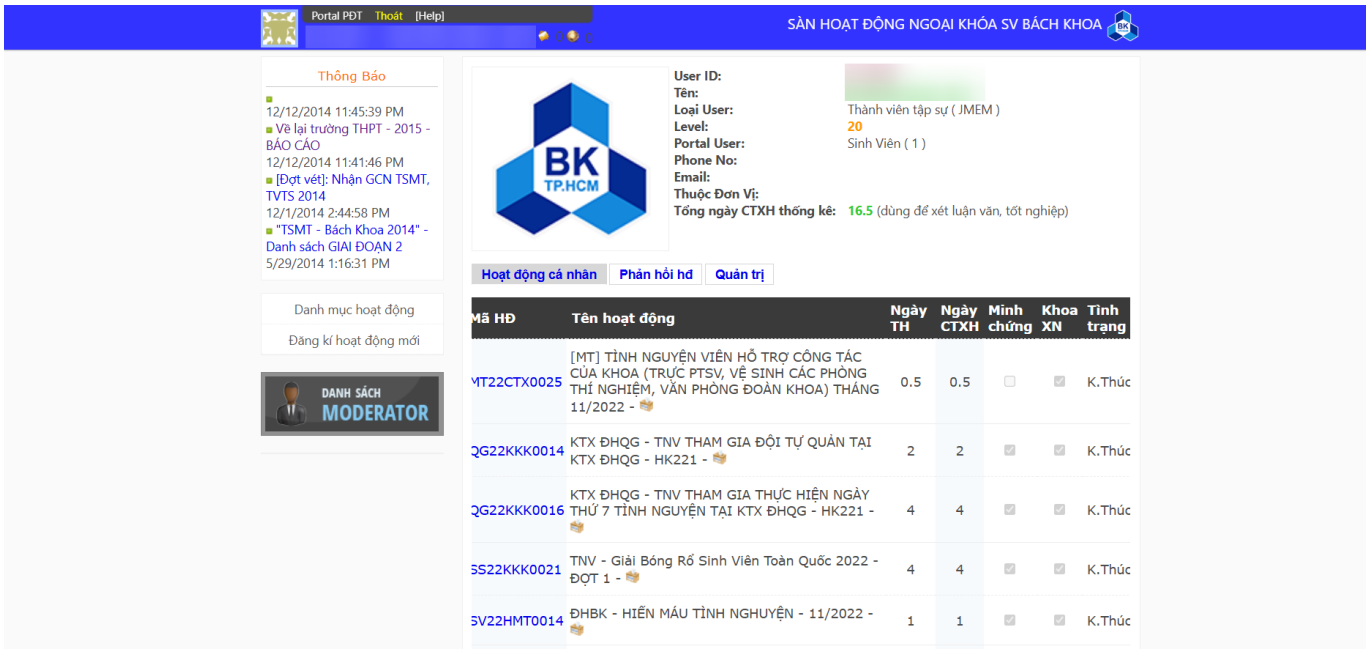
Từ trang chi tiết, sinh viên có thể đăng ký tham gia hoặc xem các mục liên quan như bình luận, thành viên, ảnh, video, thông báo và đội nhóm. Điều này cho thấy hệ thống hiện tại đã hỗ trợ được phần tương tác cơ bản giữa sinh

viên và hoạt động, nhưng thông tin vẫn được tách thành nhiều tab, nên việc tra cứu toàn bộ nội dung thường phải thực hiện qua nhiều bước.

Nhìn từ góc độ sử dụng thực tế, trang chi tiết đã cung cấp được các thông tin cần thiết để quyết định tham gia một hoạt động. Điểm có thể nâng cấp là cách tổ chức thông tin theo hướng cô đọng hơn, giúp sinh viên nắm nhanh các nội dung quan trọng như điều kiện tham gia, số ngày CTXH và thời gian thực hiện.

2.1.6 Trang cá nhân sinh viên

Ngoài phần tra cứu hoạt động, sinh viên còn có trang cá nhân để xem thông tin tài khoản và tổng số ngày CTXH đã tích lũy. Tại đây, hệ thống hiển thị danh sách các hoạt động cá nhân đã tham gia cùng các cột như ngày tính, ngày CTXH, minh chứng, khoa xác nhận và tình trạng. Đây là chức năng hữu ích cho việc theo dõi quá trình tích lũy ngày CTXH, nhưng giao diện vẫn còn thiên về bảng dữ liệu và chưa thật sự tối ưu cho việc tra cứu nhanh hoặc hỏi đáp theo ngữ cảnh.

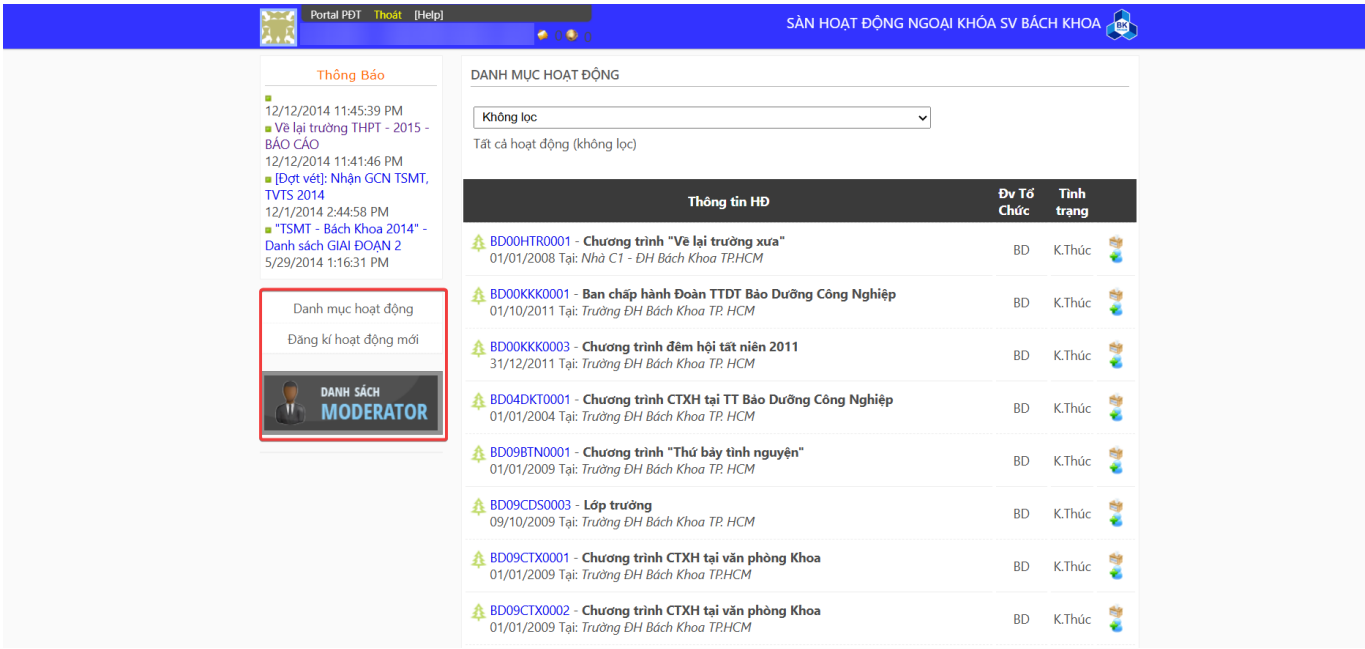


Hình 9: Trang cá nhân sinh viên và thông tin tích lũy ngày CTXH

Điểm cần lưu ý là dữ liệu được trình bày theo dạng bảng, khá phù hợp cho việc quản lý và đối soát. Nếu bổ sung thêm các điểm nhấn trực quan hoặc bộ lọc nhanh, trải nghiệm tra cứu của sinh viên sẽ thuận tiện hơn.

2.1.7 Đăng ký hoạt động mới và danh sách điều phối

Hệ thống còn có màn hình đăng ký hoạt động mới cho các đơn vị tổ chức, với các trường thông tin như loại hoạt động, đơn vị tổ chức, mã hoạt động, tên hoạt động, hình ảnh, mô tả và nội dung chi tiết. Bên cạnh đó, một số khu vực quản trị còn hiển thị danh sách moderator hoặc danh mục hoạt động theo khu vực.



Hình 10: Màn hình đăng ký hoạt động mới trên cổng CTXH cũ

Từ góc độ người dùng sinh viên, những màn hình này cho thấy cổng thông tin hiện tại đã có nền tảng nghiệp vụ tương đối đầy đủ cho việc công bố và theo dõi hoạt động CTXH. Về trải nghiệm, hệ thống có thể được cải thiện thêm ở mức độ đồng nhất giao diện và hỗ trợ tra cứu nhanh theo ngữ cảnh.

Ngoài ra, do hệ thống thiên về liệt kê và nhập liệu theo biểu mẫu, việc tìm hiểu thông tin mới vẫn dựa nhiều vào thao tác thủ công của người dùng. Đây là đặc điểm thường gặp ở các hệ thống được phát triển theo từng giai đoạn; vì vậy, định hướng nâng cấp theo hướng trực quan và thông minh hơn là phù hợp với nhu cầu hiện nay.

2.1.8 Một số điểm cần cải thiện của hệ thống hiện tại

Từ khảo sát ở cả góc nhìn nghiệp vụ và góc nhìn kỹ thuật phía người dùng, có thể tổng hợp một số điểm nên ưu tiên cải thiện như sau:

- Cần hiện đại hóa dần nền tảng giao diện: mô hình WebForms và Ajax toolkit phù hợp với giai đoạn trước nhưng có thể nâng cấp để tăng tốc độ và tính trực quan.
- Cần tối ưu tra cứu thông tin: người dùng hiện vẫn phải duyệt nhiều danh sách và nhiều tab để tổng hợp thông tin cho một hoạt động.
- Cần chuẩn hóa giao tiếp dịch vụ cho tích hợp mở rộng: phía người dùng hiện chủ yếu tương tác qua các trang `.aspx`.
- Cần tăng mức độ tập trung người dùng trên cổng chính: thực tế nhiều sinh viên theo dõi hoạt động mới qua mạng xã hội rồi mới quay lại đối chiếu trên hệ thống.
- Cần bổ sung lớp hỗ trợ thông minh: công cụ hỏi đáp theo ngữ cảnh sẽ giúp sinh viên tra cứu nhanh quy định, điều kiện và thông tin hoạt động.

Các điểm trên là cơ sở để đề tài định hướng xây dựng hệ thống mới theo hướng hiện đại hơn, thuận tiện hơn trong tra cứu và sử dụng, đồng thời có khả năng mở rộng tích hợp tốt hơn trong tương lai.

2.2 Cơ sở lý thuyết

2.2.1 Hệ thống thông tin quản lý và chuyển đổi số

Đề tài được định hướng theo tư duy của Hệ thống thông tin quản lý (Management Information Systems – MIS), trong đó dữ liệu, quy trình và báo cáo được tổ chức thành một thể thống nhất để hỗ trợ vận hành và ra quyết định. Với bối cảnh hoạt động Đoàn – Hội, cách tiếp cận này giúp liên kết các tác vụ vốn rời rạc như theo dõi hoạt động, xác nhận ngày CTXH, quản lý biểu mẫu và đối soát dữ liệu liên quan.

Song song đó, chuyển đổi số được xem là quá trình cải tiến liên tục thay vì chỉ thay đổi công cụ. Vì vậy, hệ thống không chỉ số hóa biểu mẫu và quy trình hiện có mà còn hướng đến việc chuẩn hóa luồng xử lý, giảm thao tác thủ công và tăng khả năng truy vết dữ liệu trong toàn bộ vòng đời nghiệp vụ.

2.2.2 Mức trưởng thành số trong môi trường giáo dục

Trong môi trường giáo dục, mức trưởng thành số (Digital Maturity) thể hiện qua khả năng vận hành quy trình trên nền tảng số, khả năng đo lường hiệu quả và khả năng phục vụ người dùng theo thời gian thực. Đối với phạm vi đề tài, điều này được thể hiện qua ba định hướng chính: số hóa quy trình nghiệp vụ CTXH, minh bạch hóa dữ liệu vận hành và nâng cao khả năng hỗ trợ sinh viên theo ngữ cảnh.

Khung trưởng thành số cũng là cơ sở để xác định lộ trình phát triển hệ thống theo từng giai đoạn. Ở giai đoạn đầu, hệ thống tập trung chuẩn hóa dữ liệu và quy trình; ở giai đoạn tiếp theo, hệ thống tăng cường khả năng phân tích, giám sát và hỗ trợ tự động để nâng cao chất lượng trải nghiệm người dùng.

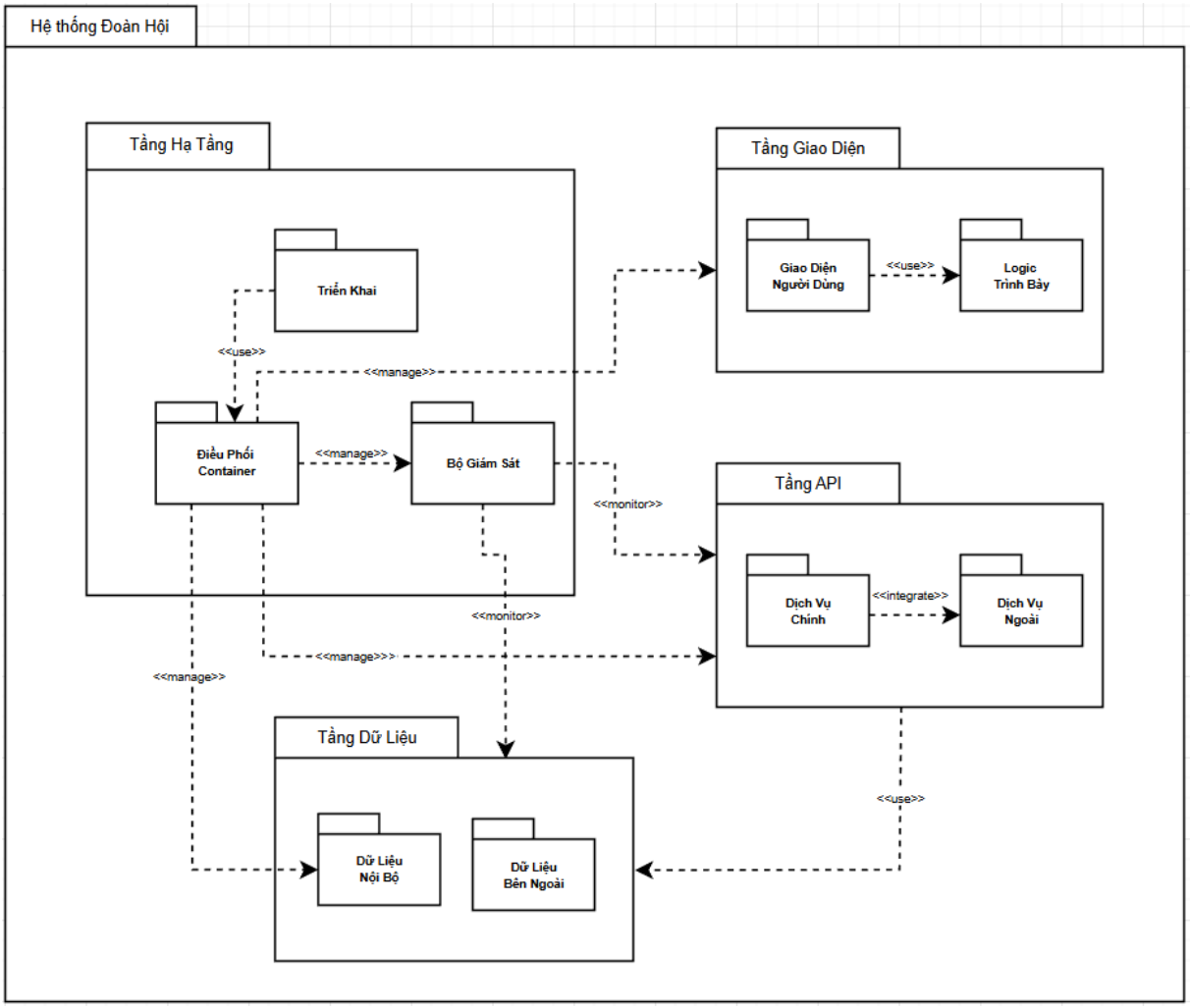
2.2.3 Quản lý dữ liệu tập trung

Hệ thống của đề tài được xây dựng theo hướng quản lý dữ liệu tập trung, trong đó thông tin người dùng, hoạt động công tác xã hội, quyền truy cập và dữ liệu tài chính được lưu trữ và truy xuất từ một nguồn dữ liệu thống nhất. Cách tiếp cận này giúp hạn chế tình trạng phân tán dữ liệu, giảm trùng lặp thông tin và hỗ trợ tra cứu nhanh hơn so với việc lưu trữ rời rạc trên nhiều file hoặc nhiều công cụ khác nhau.

Trong bối cảnh quản lý Đoàn – Hội, mô hình dữ liệu tập trung còn giúp tăng khả năng đồng bộ giữa các phân hệ, từ đó bảo đảm rằng khi một thông tin được cập nhật ở một nơi thì các thành phần liên quan trong hệ thống cũng có thể sử dụng ngay dữ liệu mới nhất.

2.2.4 Kiến trúc phân lớp

Để đảm bảo tính mở rộng (Scalability) và khả năng bảo trì (Maintainability) lâu dài, hệ thống được tổ chức theo mô hình **Layered Application Architecture** (kiến trúc phân tầng). Các thành phần được tách theo trách nhiệm rõ ràng, giúp giảm phụ thuộc chéo, thuận lợi cho kiểm thử và mở rộng theo từng phân hệ.



Hình 11: Kiến trúc phân tầng của hệ thống

Chi tiết các tầng trong kiến trúc hệ thống gồm:

- **Tầng Giao diện (Presentation Layer):** là điểm tiếp xúc trực tiếp với người dùng cuối (sinh viên, cán bộ Đoàn – Hội). Tầng này tập trung vào hiển thị thông tin và điều hướng thao tác.
  - *User Interface*: cung cấp giao diện trực quan cho tra cứu, đăng ký và theo dõi hoạt động.
  - *Presentation Logic*: xử lý trạng thái hiển thị, kiểm tra đầu vào ở mức giao diện và điều phối lời gọi xuống tầng API.
- **Tầng API (API Layer):** đóng vai trò trung tâm xử lý nghiệp vụ, cung cấp các điểm truy cập thống nhất cho frontend và các dịch vụ tích hợp.
  - *Core Services*: xử lý các luồng nghiệp vụ cốt lõi như người dùng, hoạt động CTXH, phân quyền, thanh toán và báo cáo.
  - *External Services*: thực hiện tích hợp với dịch vụ bên thứ ba (ví dụ cổng thanh toán, kênh xác thực dùng chung hoặc dịch vụ AI).
- **Tầng Dữ liệu (Data Layer):** chịu trách nhiệm lưu trữ và truy xuất dữ liệu bền vững, phục vụ cả nghiệp vụ giao dịch lẫn tra cứu.
  - *Internal Data*: cơ sở dữ liệu chính của hệ thống, lưu thông tin người dùng, tổ chức, hoạt động và giao dịch.

– *External Data*: kho lưu trữ mở rộng cho tệp đính kèm, hình ảnh minh chứng và dữ liệu phục vụ AI Chatbot.

- **Tầng Hạ tầng (Infrastructure Layer):** là nền tảng vận hành cho toàn bộ hệ thống, bảo đảm triển khai ổn định và giám sát liên tục.
  – *Container Orchestration*: quản lý vòng đời các dịch vụ, hỗ trợ triển khai linh hoạt theo môi trường.
  – *Observability Suite*: thu thập log, metrics và cảnh báo để phát hiện sớm sự cố, đặc biệt trong các giai đoạn tải cao.

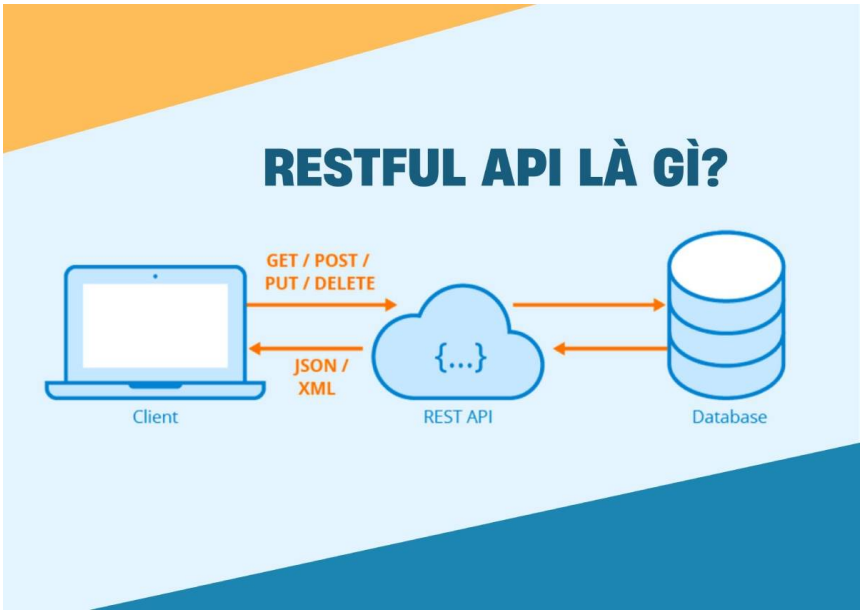
2.2.5 Quản lý biểu mẫu điện tử

Biểu mẫu điện tử là cách tiếp cận thay thế cho các biểu mẫu giấy hoặc file mẫu chỉnh sửa thủ công, trong đó dữ liệu đầu vào của người dùng được ràng buộc theo cấu trúc sẵn có của template. Lý thuyết này đặc biệt phù hợp với các quy trình cần đồng nhất thông tin, giảm lỗi nhập liệu và tự động sinh tài liệu đầu ra sau khi hoàn tất.

Trong đề tài, biểu mẫu điện tử được sử dụng cho các nghiệp vụ cần tạo hồ sơ và chứng từ theo mẫu. Nhờ việc gắn cấu trúc dữ liệu với template, hệ thống có thể hỗ trợ người dùng nhập thông tin một lần nhưng vẫn tạo ra tài liệu chuẩn hóa, phục vụ lưu trữ và tra cứu lâu dài.

2.2.6 RESTful API

RESTful API là kiến trúc giao tiếp giữa client và server dựa trên các nguyên tắc của REST, trong đó mỗi tài nguyên được định danh bằng URI và được thao tác thông qua các phương thức HTTP như GET, POST, PUT, PATCH và DELETE. Đây là nền tảng quan trọng để frontend và backend trong hệ thống có thể trao đổi dữ liệu một cách rõ ràng, thống nhất và độc lập với nhau.



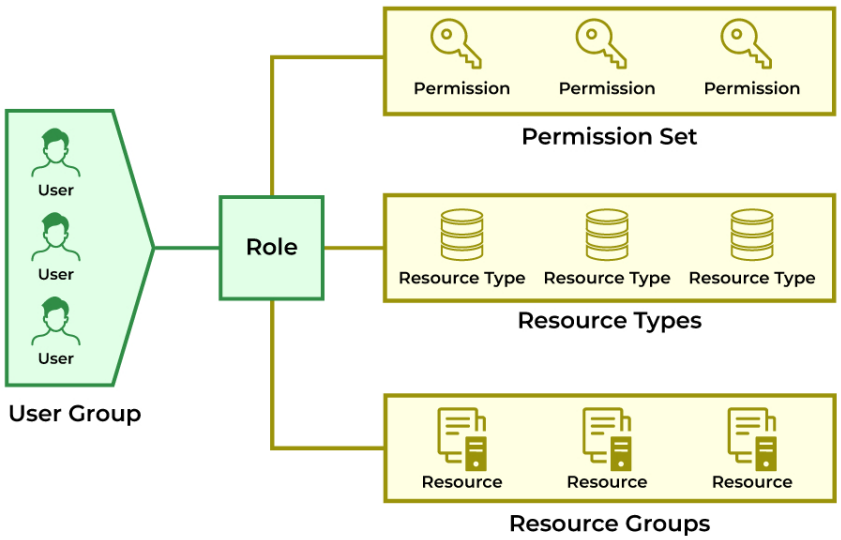
Hình 12: Minh họa nguyên lý giao tiếp của RESTful API

Đối với đề tài, RESTful API được sử dụng để xử lý các nghiệp vụ như quản lý người dùng, danh mục hoạt động, chấm ngày CTXH, cấu hình giới hạn, thanh toán và tra cứu chatbot. Cách thiết kế này giúp hệ thống dễ tích hợp với các màn hình giao diện khác nhau và cũng thuận tiện khi kiểm thử hoặc mở rộng về sau.

2.2.7 Xác thực và phân quyền

Xác thực và phân quyền là hai thành phần quan trọng trong hệ thống quản lý có nhiều nhóm người dùng khác nhau. Xác thực dùng để kiểm tra danh tính của người dùng khi đăng nhập, còn phân quyền dùng để quyết định

người dùng đó được phép truy cập hoặc thao tác trên những chức năng nào.



Hình 13: Mô hình phân quyền dựa trên vai trò (RBAC)

Trong hệ thống này, JSON Web Token (JWT) được sử dụng để duy trì trạng thái đăng nhập theo hướng an toàn và gọn nhẹ. Bên cạnh đó, cơ chế phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) giúp tách biệt quyền hạn giữa sinh viên, cán bộ và quản trị viên, từ đó hạn chế việc truy cập sai chức năng hoặc chỉnh sửa dữ liệu không phù hợp.

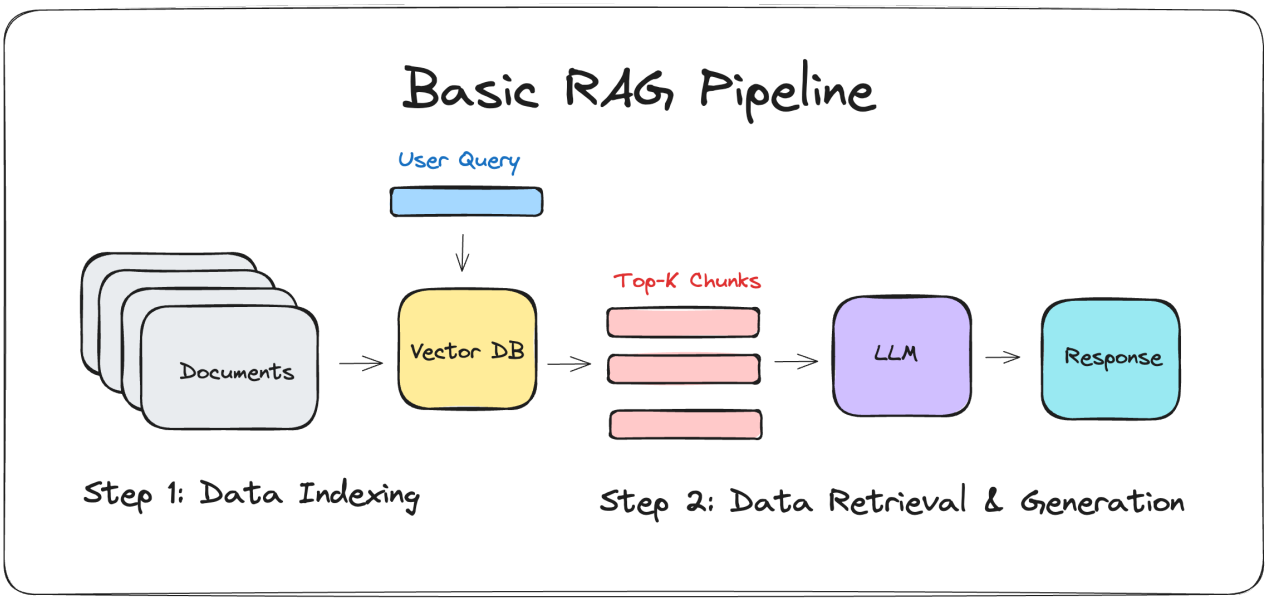
2.2.8 An toàn thông tin theo CIA Triad và accountability

Để bảo đảm an toàn thông tin ở mức hệ thống, đề tài tham chiếu mô hình CIA Triad gồm tính bảo mật (Confidentiality), toàn vẹn (Integrity) và sẵn sàng (Availability). Trong đó, Confidentiality được hỗ trợ bởi xác thực JWT và phân quyền RBAC; Integrity được tăng cường thông qua ràng buộc dữ liệu, quy tắc nghiệp vụ và cơ chế ghi vết; Availability được bảo đảm thông qua giám sát vận hành, cảnh báo và phương án duy trì dịch vụ.

Với các nghiệp vụ có yếu tố tài chính, nguyên tắc accountability đóng vai trò đặc biệt quan trọng. Mỗi thao tác cần có khả năng truy vết để phục vụ đối soát và kiểm tra khi cần thiết, từ đó tăng tính minh bạch dữ liệu và giảm rủi ro sai lệch trong quá trình vận hành thực tế.

2.2.9 RAG cho chatbot CTXH

Retrieval-Augmented Generation (RAG) là mô hình kết hợp giữa truy xuất thông tin và sinh nội dung bằng mô hình ngôn ngữ lớn. Thay vì chỉ dựa hoàn toàn vào kiến thức đã học, hệ thống RAG sẽ truy xuất các tài liệu hoặc dữ liệu liên quan trước, sau đó dùng mô hình ngôn ngữ để tạo ra câu trả lời phù hợp hơn với ngữ cảnh thực tế.



Hình 14: Mô hình Retrieval-Augmented Generation (RAG)

Trong đề tài, RAG được áp dụng cho AI Chatbot phục vụ công tác xã hội. Chatbot chỉ trả lời trong phạm vi thông tin CTXH đã được quản lý trên hệ thống, chẳng hạn như quy định, hoạt động, điều kiện tham gia, hoặc các nội dung tra cứu phổ biến. Cách làm này giúp câu trả lời có cơ sở dữ liệu rõ ràng, đồng thời giảm bớt nguy cơ trả lời ngoài phạm vi hoặc sai lệch nội dung.

2.2.10 Giám sát và ghi vết hệ thống

Trong các hệ thống thông tin có nhiều tác vụ quan trọng như đăng nhập, chấm điểm, thanh toán và sinh tài liệu, việc ghi log và giám sát trạng thái vận hành là một phần không thể tách rời của chất lượng hệ thống. Ghi vết đầy đủ giúp truy nguyên lỗi, phục vụ kiểm tra nghiệp vụ và hỗ trợ quản trị hệ thống trong quá trình vận hành thực tế.

Đối với đề tài, yếu tố này có ý nghĩa đặc biệt trong các phân hệ có tác động trực tiếp đến dữ liệu người dùng và dữ liệu tài chính. Khi có sự cố, log và metrics là cơ sở để xác định nguyên nhân, đánh giá mức độ ảnh hưởng và đưa ra biện pháp xử lý phù hợp.

2.3 Công nghệ sử dụng

2.3.1 Frontend

Frontend của hệ thống được xây dựng bằng React kết hợp TypeScript và Vite. React được dùng để phát triển giao diện theo hướng component, cho phép tái sử dụng các thành phần hiển thị trong nhiều màn hình khác nhau. TypeScript giúp bổ sung kiểu dữ liệu tĩnh, giảm lỗi trong quá trình phát triển, còn Vite hỗ trợ khởi tạo và biên dịch dự án nhanh hơn.

Ngoài các công nghệ nền tảng trên, giao diện còn sử dụng React Router để điều hướng giữa các trang, TanStack React Query để quản lý trạng thái dữ liệu từ server, i18next để hỗ trợ đa ngôn ngữ, Tailwind CSS để xây dựng giao diện nhanh và nhất quán, cùng với Radix UI và assistant-ui cho các thành phần giao diện tương tác, đặc biệt là phần chatbot.

- React: xây dựng giao diện theo hướng component và dễ tái sử dụng.
- TypeScript: tăng tính an toàn kiểu dữ liệu và khả năng bảo trì.
- React Router: quản lý điều hướng và phân trang trong ứng dụng.

- TanStack React Query: đồng bộ dữ liệu từ backend, giảm xử lý thủ công ở phía client.
- Tailwind CSS, Radix UI: hỗ trợ xây dựng giao diện nhanh, gọn và linh hoạt.
- i18next: hỗ trợ đa ngôn ngữ cho các màn hình người dùng.
- assistant-ui: hiển thị và tương tác với giao diện chatbot.

2.3.2 Backend

Backend của hệ thống được xây dựng bằng NestJS trên nền tảng Node.js và TypeScript. NestJS cung cấp kiến trúc module rõ ràng, phù hợp với các hệ thống có nhiều phân hệ như quản lý người dùng, CTXH, thanh toán, xuất báo cáo và chatbot. TypeORM được sử dụng để làm việc với cơ sở dữ liệu theo hướng ánh xạ đối tượng – quan hệ, giúp mã nguồn backend rõ ràng và dễ bảo trì hơn.

Hệ thống cũng sử dụng JWT và Passport cho xác thực, Swagger để sinh tài liệu API, Bull kết hợp Redis cho các tác vụ nền, PayOS cho thanh toán điện tử, và AWS S3 cho các nhu cầu lưu trữ tệp. Ngoài ra, NestJS Schedule và EventEmitter được dùng để hỗ trợ các tác vụ định kỳ và cơ chế phát sinh sự kiện trong hệ thống.

- NestJS: framework backend theo mô hình module, phù hợp cho hệ thống quy mô lớn.
- TypeORM: ánh xạ dữ liệu giữa entity và PostgreSQL.
- JWT, Passport: xác thực và bảo vệ các API quan trọng.
- Bull, Redis: xử lý hàng đợi và tác vụ nền cho các luồng cần bất đồng bộ.
- Swagger: tài liệu hóa API và hỗ trợ kiểm thử nhanh.
- PayOS: tích hợp thanh toán trực tuyến cho các nghiệp vụ có thu phí.
- AWS S3: lưu trữ tệp tin và tài liệu dùng chung.

2.3.3 Cơ sở dữ liệu

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ được sử dụng để lưu trữ toàn bộ dữ liệu nghiệp vụ của hệ thống. PostgreSQL phù hợp với các bài toán cần tính nhất quán cao, hỗ trợ truy vấn phức tạp và có khả năng mở rộng tốt.

Riêng với phần chatbot CTXH, hệ thống còn sử dụng cơ chế lưu vector embedding trong PostgreSQL để phục vụ truy xuất ngữ nghĩa. Cách tổ chức này cho phép chatbot tìm đúng nội dung liên quan từ dữ liệu CTXH trước khi tạo câu trả lời.

- PostgreSQL: lưu trữ dữ liệu quan hệ của người dùng, tổ chức, hoạt động và giao dịch.
- Vector embedding: phục vụ truy xuất ngữ nghĩa cho chatbot CTXH.
- TypeORM migrations và seed scripts: hỗ trợ tạo, cập nhật và khởi tạo dữ liệu.

2.3.4 AI Chatbot và LangChain

Phần AI Chatbot của hệ thống được xây dựng dựa trên LangChain.js, kết hợp với mô hình ngôn ngữ và cơ chế RAG để trả lời các câu hỏi trong phạm vi hoạt động công tác xã hội. Cụm xử lý này gồm bước nạp dữ liệu CTXH vào vector store, truy xuất các đoạn nội dung liên quan, rồi mới sinh câu trả lời cuối cùng.

Theo cấu hình hiện tại trong dự án, hệ thống hỗ trợ hai hướng triển khai mô hình: chạy cục bộ qua Ollama hoặc dùng dịch vụ Gemini tùy cấu hình môi trường. Nhờ đó, chatbot có thể được điều chỉnh theo điều kiện triển khai thực tế và yêu cầu vận hành của hệ thống.

Cách chọn hướng triển khai linh hoạt này giúp đề tài vừa có thể đáp ứng nhu cầu thử nghiệm và phát triển nội bộ, vừa có khả năng thích nghi với các môi trường vận hành khác nhau. Trong bối cảnh dữ liệu CTXH thường

xuyên thay đổi theo từng đợt hoạt động, kiến trúc này còn giúp hệ thống cập nhật nội dung tra cứu theo hướng chủ động hơn.

- LangChain.js: điều phối luồng RAG, truy xuất và sinh câu trả lời.
- Ollama hoặc Gemini: cung cấp mô hình ngôn ngữ và embeddings theo cấu hình.
- Vector store trên PostgreSQL: lưu trữ và tìm kiếm embedding của hoạt động CTXH.
- Bull: hỗ trợ nạp dữ liệu và đồng bộ embedding theo luồng nền.

2.3.5 Triển khai và giám sát

Hệ thống được đóng gói và triển khai bằng Docker, giúp thống nhất môi trường phát triển và môi trường chạy thật. Bên cạnh đó, dự án còn có cấu hình Prometheus và Grafana để theo dõi trạng thái hệ thống, phục vụ giám sát và phân tích vận hành.

- Docker: đóng gói và triển khai dịch vụ đồng nhất giữa các môi trường.
- Prometheus: thu thập chỉ số vận hành.
- Grafana: trực quan hóa dữ liệu giám sát.

Chương 3 Phân tích hệ thống

3.1 Các tính năng trong hệ thống

3.1.1 Yêu cầu chức năng

Trong phạm vi đề tài, nhóm xác định các chức năng theo hướng bám sát quy trình vận hành thực tế của công tác Đoàn – Hội trong trường. Thay vì tách rời từng màn hình, hệ thống được chia theo các cụm nghiệp vụ để thuận tiện triển khai và mở rộng.

Việc mô hình hóa yêu cầu theo cụm nghiệp vụ giúp phản ánh đúng cách tổ chức công việc trong thực tế: một cán bộ thường không thao tác trên một chức năng đơn lẻ mà xử lý theo chuỗi, ví dụ như quản lý hồ sơ người dùng đi cùng với phân quyền, hoặc tạo hoạt động đi cùng với phân công và chấm điểm. Cách trình bày này giúp phần yêu cầu chức năng sát thực hơn với quy trình vận hành và đồng thời tạo nền tảng cho phần phân tích use case ở các mục sau.

1. Nhóm quản lý người dùng và tổ chức

Nhóm chức năng này tập trung vào việc số hóa hồ sơ người dùng, bao gồm thông tin định danh, học vụ và trạng thái Đoàn viên/Hội viên. Bên cạnh đó, hệ thống quản lý cơ cấu Trường → Khoa → Lớp và các loại tổ chức như Đoàn, Hội, CLB. Để giảm thời gian nhập liệu, dữ liệu có thể được nạp hàng loạt từ Excel. Phần phân quyền sử dụng RBAC có ngữ cảnh tổ chức, cho phép một tài khoản đảm nhiệm nhiều vai trò ở các đơn vị khác nhau.

2. Nhóm quản lý hoạt động công tác xã hội (CTXH)

Đây là cụm nghiệp vụ trọng tâm của đề tài. Hệ thống hỗ trợ quản lý danh mục và danh sách hoạt động CTXH theo vòng đời trạng thái, quản lý danh sách người tham gia theo từng hoạt động, đồng thời cho phép cập nhật dữ liệu bằng cả thao tác thủ công và import từ file. Việc chấm ngày CTXH được hỗ trợ theo hai cách là chấm trực tiếp hoặc chấm hàng loạt bằng Excel. Kết quả sau cùng được đồng bộ về hồ sơ sinh viên để phục vụ báo cáo cá nhân và tổng hợp.

3. Nhóm quản lý thu phí

Đối với nghiệp vụ thu phí, hệ thống cho phép tạo đợt thu, cấu hình mức phí và phân công đối tượng cần đóng. Trạng thái thanh toán được theo dõi theo từng sinh viên và từng đợt để dễ đối soát. Hệ thống tích hợp PayOS cho thanh toán trực tuyến, sau đó nhận webhook để cập nhật trạng thái giao dịch. Dữ liệu lịch sử thanh toán cũng được lưu lại để phục vụ kiểm tra và thống kê.

4. Nhóm biểu mẫu và chứng từ

Nhóm chức năng này xử lý toàn bộ vòng đời biểu mẫu điện tử. Quản trị viên có thể upload template Word (`.docx`), hệ thống nhận diện placeholder để tạo field dữ liệu tương ứng. Người dùng điền thông tin qua form động, backend render tài liệu bằng Docxtemplater, lưu file trên S3 và lưu metadata vào cơ sở dữ liệu. Các hồ sơ đã nộp có thể được tra cứu và tải lại khi cần.

5. Nhóm hỗ trợ tra cứu bằng chatbot CTXH

Chatbot đóng vai trò kênh hỗ trợ tra cứu nhanh cho sinh viên. Người dùng có thể đặt câu hỏi tự nhiên, và hệ thống trả lời dựa trên dữ liệu CTXH hiện có, tập trung vào quy định và thông tin hoạt động liên quan.

6. Nhóm nền tảng vận hành

Ngoài các nghiệp vụ chính, hệ thống còn có các thành phần nền tảng phục vụ vận hành gồm dashboard tổng quan, màn hình cấu hình hệ thống tập trung (bao gồm S3 và feature flags), cùng với cơ chế xác thực, quản lý phiên và kiểm tra quyền xuyên suốt.

3.1.2 Yêu cầu phi chức năng

Ngoài chức năng nghiệp vụ, hệ thống cần đáp ứng một số yêu cầu phi chức năng để có thể đưa vào sử dụng ổn định.

Các yêu cầu phi chức năng được đặt ra không chỉ để bảo đảm hệ thống chạy được, mà còn để đáp ứng đúng bối cảnh sử dụng của một cổng thông tin có nhiều nhóm người dùng và nhiều loại dữ liệu khác nhau. Khi số lượng

sinh viên, hoạt động và giao dịch tăng lên, những yêu cầu như bảo mật, hiệu năng, khả năng mở rộng hay giám sát vận hành sẽ trở thành điều kiện bắt buộc để hệ thống duy trì chất lượng dịch vụ.

1. Bảo mật và an toàn dữ liệu

Về bảo mật, thông tin nhạy cảm được bảo vệ bằng cơ chế băm mật khẩu và xác thực JWT/refresh token. Các thao tác quan trọng đều phải đi qua lớp kiểm tra quyền RBAC. Dữ liệu đầu vào được kiểm tra ở cả frontend và backend để hạn chế lỗi cũng như rủi ro bảo mật. Ngoài ra, hệ thống có cơ chế kiểm soát upload tệp, CORS và HTTPS theo môi trường triển khai.

2. Hiệu năng xử lý

Về hiệu năng, truy vấn dữ liệu cần có phân trang và tối ưu index để tránh quá tải khi dữ liệu tăng. Các tác vụ nặng như import, sinh tài liệu hoặc đồng bộ embeddings được tổ chức theo luồng xử lý phù hợp. Ở phía giao diện, cơ chế cache giúp giảm số lần gọi API lặp lại.

3. Khả năng mở rộng và bảo trì

Về mở rộng và bảo trì, hệ thống được đóng gói bằng Docker và tổ chức theo kiến trúc module để dễ thêm chức năng mới. Việc tách lớp lưu trữ tệp lên S3 giúp giảm tải cho cơ sở dữ liệu giao dịch. Hai phần backend/frontend có thể phát triển tương đối độc lập theo từng nhóm nghiệp vụ.

4. Giám sát vận hành

Trong quá trình vận hành, hệ thống theo dõi metrics bằng Prometheus và trực quan hóa bằng Grafana. Song song đó, các thao tác chính được ghi log để hỗ trợ kiểm tra lỗi và truy vết.

5. Trải nghiệm sử dụng

Về trải nghiệm người dùng, giao diện cần đảm bảo responsive và thể hiện rõ trạng thái loading/error. Hệ thống hỗ trợ đa ngôn ngữ và tổ chức giao diện theo module để người dùng dễ làm quen. Mã nguồn được duy trì theo coding standards để thuận lợi cho bàn giao và bảo trì dài hạn.

3.2 Danh sách các Actor

Trong hệ thống cổng quản lý Đoàn – Hội, actor được xác định theo mô hình phân quyền và luồng nghiệp vụ đang hiện thực trong mã nguồn. Nhóm phân tách actor thành các nhóm người dùng nội bộ và actor hệ thống bên ngoài để thuận tiện cho việc đặc tả use-case.

a. Người dùng chưa đăng nhập

Chức năng	Mô tả
Truy cập màn hình đăng nhập	Người dùng chưa đăng nhập truy cập trang công khai `/login`; đường dẫn `/admin-doan-hoi/login` là điểm vào nội bộ cho cán bộ quản trị, không công khai cho người dùng phổ thông.
Xác thực tài khoản	Người dùng thực hiện đăng nhập bằng tài khoản nội bộ hoặc qua Google OAuth, sau đó hệ thống cấp phiên truy cập.
Nhận kết quả xác thực	Sau khi xác thực thành công, người dùng được chuyển qua luồng call-back và vào khu vực `/portal`.

Bảng 3: Chức năng của người dùng chưa đăng nhập

b. Thành viên (Sinh viên/Đoàn viên/Hội viên)

Chức năng	Mô tả
Xem báo cáo CTXH của bản thân	Thành viên có thể xem báo cáo cá nhân và dữ liệu CTXH của sinh viên, gồm thông tin sinh viên (họ tên, mã số sinh viên, email), tổng số ngày CTXH, tổng hợp kết quả (ngày hoạt động đã hoàn thành, ngày hoạt động trên hệ thống cũ, tổng số ngày hoạt động CTXH, trạng thái đạt/chưa đạt), cùng danh sách hoạt động với các cột tên hoạt động, danh mục, số ngày thực tế, trạng thái, ngày chấm điểm và ghi chú.
Tra cứu và thanh toán khoản phí của tôi	Thành viên xem danh sách các đợt thu được phân công, tạo giao dịch thanh toán và theo dõi kết quả thanh toán.
Điền và nộp biểu mẫu	Thành viên điền form động theo template đã cấu hình và lưu hồ sơ biểu mẫu trên hệ thống.
Tra cứu CTXH bằng chatbot	Thành viên đặt câu hỏi tự nhiên để tra cứu quy định và thông tin hoạt động CTXH.

Bảng 4: Chức năng của actor thành viên

c. Cán bộ đơn vị (cấp Trường/Khoa/Lớp/Chi đoàn/Chi hội)

Chức năng	Mô tả
Quản lý người dùng và cơ cấu tổ chức	Cán bộ đơn vị quản lý danh sách người dùng, tổ chức, khoa/phòng và lớp trong phạm vi đơn vị được phân quyền.
Quản lý hoạt động CTXH	Cán bộ đơn vị quản lý danh mục, hoạt động, người tham gia và nhập dữ liệu CTXH từ hệ thống cũ trong phạm vi đơn vị được phân quyền.
Quản lý thu phí theo đơn vị	Cán bộ đơn vị tạo đợt thu, phân công đối tượng đóng phí, theo dõi thống kê và xuất báo cáo thu phí.
Theo dõi trạng thái xử lý nghiệp vụ	Cán bộ đơn vị theo dõi tình trạng xử lý các hoạt động và giao dịch trong phạm vi đơn vị để phục vụ điều phối công việc hằng ngày.

Bảng 5: Chức năng của actor cán bộ đơn vị

d. Quản trị hệ thống (Super Admin)

Chức năng	Mô tả
Quản trị phân quyền toàn cục	Quản trị hệ thống cấu hình vai trò/quyền và kiểm soát truy cập ở mức hệ thống và mức tổ chức.
Quản trị cấu hình hệ thống	Quản trị hệ thống quản lý các cấu hình nền tảng như S3, feature flags và tham số vận hành chung.
Quản trị biểu mẫu	Quản trị hệ thống quản lý template biểu mẫu, upload và cấu hình place-holder cho toàn hệ thống.
Quản trị cấu hình giới hạn CTXH	Quản trị hệ thống thiết lập các giới hạn, điều kiện và tham số áp dụng cho hoạt động CTXH trên toàn hệ thống.
Giám sát và can thiệp liên module	Quản trị hệ thống có quyền truy cập đầy đủ các module để kiểm tra, đồng bộ và xử lý sự cố vận hành.

Bảng 6: Chức năng của actor quản trị hệ thống

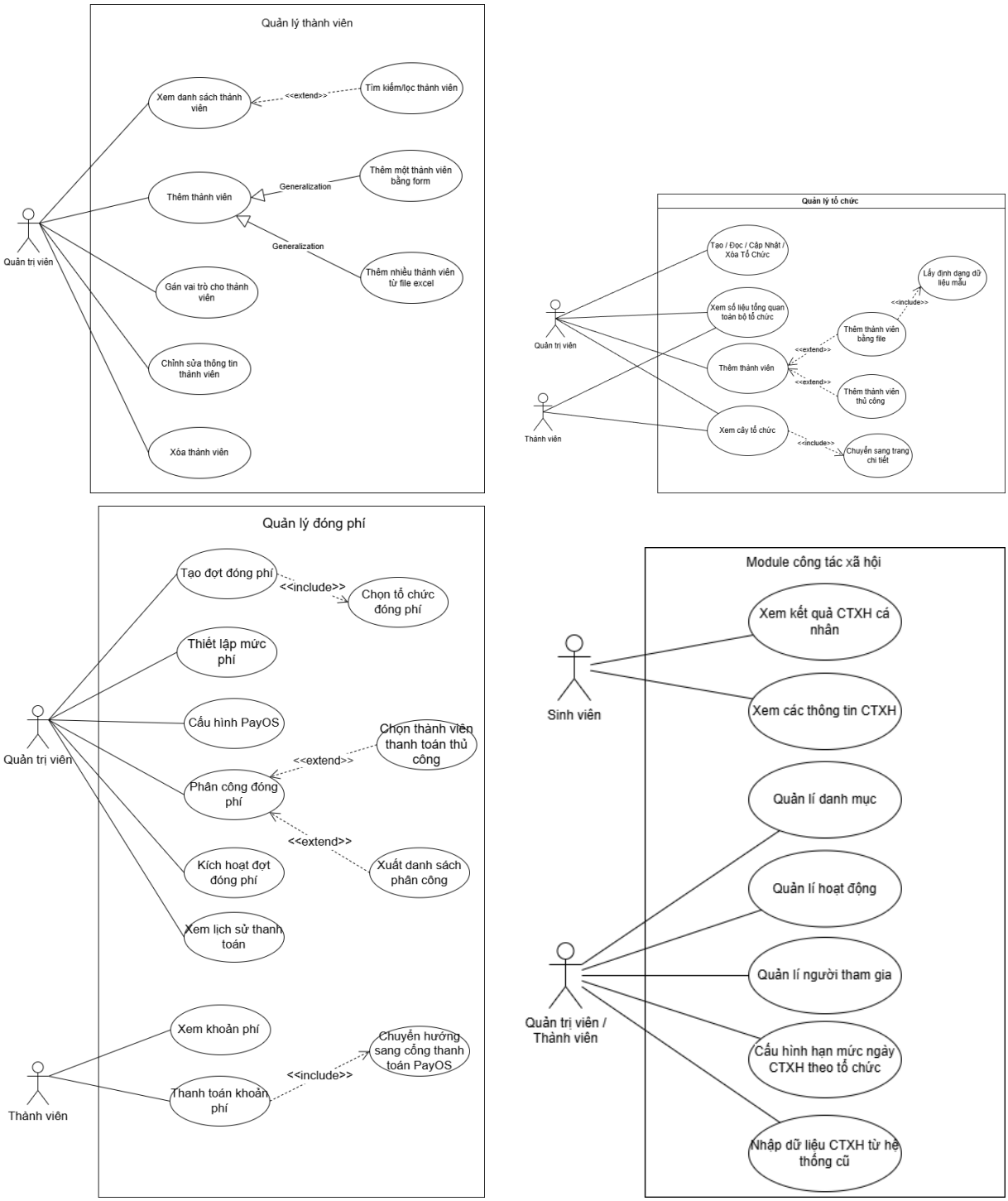
e. Cổng thanh toán PayOS (actor hệ thống ngoài)

Chức năng	Mô tả
Tiếp nhận yêu cầu tạo giao dịch	PayOS nhận thông tin đơn hàng từ hệ thống để tạo liên kết thanh toán cho người dùng.
Xử lý giao dịch thanh toán	PayOS thực hiện nghiệp vụ thanh toán và trả về trạng thái giao dịch tương ứng.
Đồng bộ trạng thái về hệ thống	Hệ thống truy vấn/nhận trạng thái từ PayOS để cập nhật lịch sử thanh toán và kết quả đối soát.

Bảng 7: Chức năng của actor PayOS

3.3 Sơ đồ Use-Case cho toàn bộ hệ thống

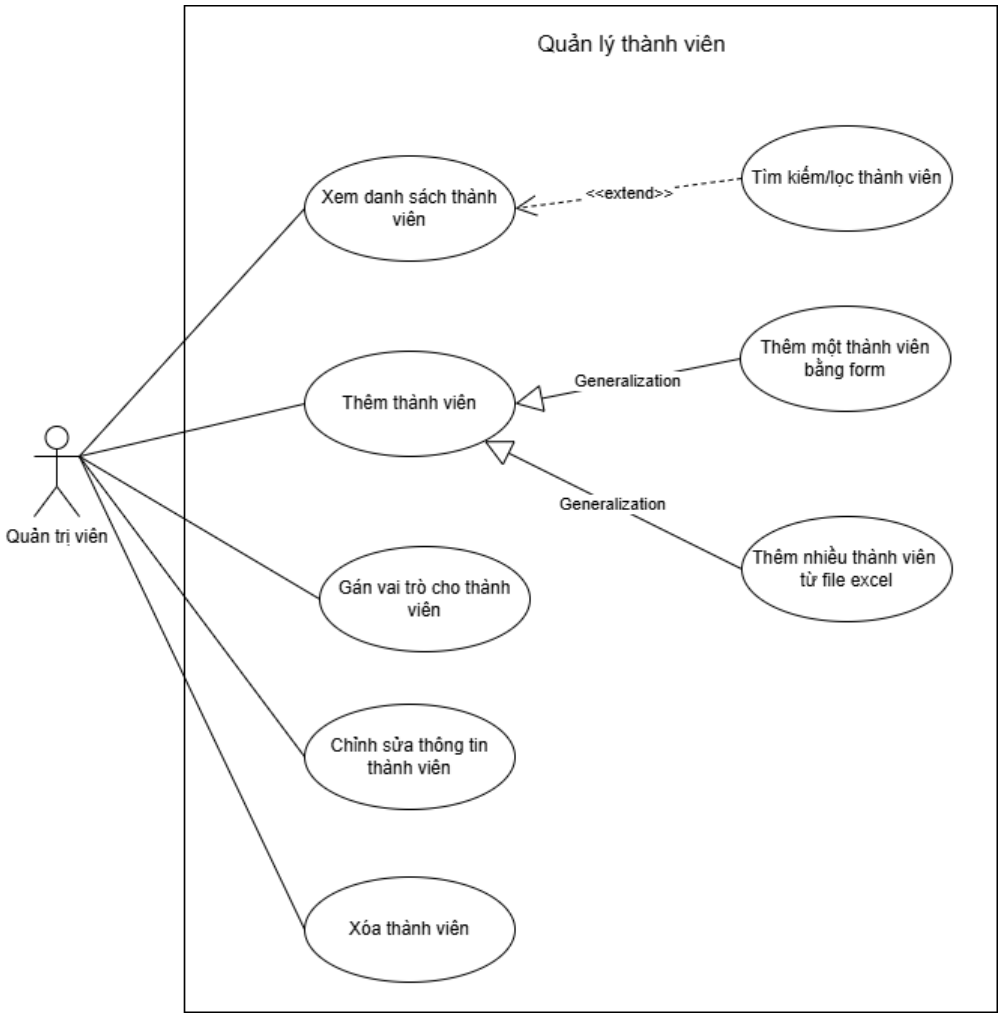
Do hệ thống bao gồm nhiều phân hệ nghiệp vụ độc lập (Người dùng, Tổ chức, Phân quyền, CTXH, Thu phí, Biểu mẫu, Chatbot), sơ đồ use-case toàn cục được trình bày theo dạng tổng hợp từ các phân hệ chính để đảm bảo dễ theo dõi trên khổ trang luận văn.



Hình 15: Sơ đồ Use-Case tổng quan toàn bộ hệ thống (ghép từ các phân hệ chính)

3.4 Sơ đồ Use-Case chi tiết

3.4.1 Phân hệ quản lý người dùng



Hình 16: UC01 - Quản lý thành viên

Phân hệ này tập trung vào các thao tác quản lý hồ sơ thành viên trong hệ thống gồm thêm/sửa/xóa, import từ Excel, tìm kiếm và xuất dữ liệu phục vụ vận hành ở cấp đơn vị.

Use Case Name	UC01 - Quản lý thành viên
Actor(s)	Quản trị viên
Description	Use case này cho phép quản trị viên thực hiện các thao tác quản lý thông tin thành viên trong hệ thống, bao gồm: – Xem danh sách thành viên – Tìm kiếm và lọc thành viên – Thêm, sửa, xóa thông tin thành viên – Gán vai trò (phân quyền) cho thành viên.
Trigger	Quản trị viên truy cập chức năng Quản lý thành viên từ giao diện hệ thống.
Pre-condition(s)	Quản trị viên đã đăng nhập vào hệ thống bằng tài khoản có quyền quản lý thành viên.
Post-condition(s)	Thông tin thành viên được hiển thị và cập nhật chính xác theo các thao tác đã thực hiện.
Main Flow	1. Quản trị viên truy cập module Quản lý thành viên. 2. Hệ thống hiển thị danh sách thành viên. 3. Quản trị viên chọn một trong các thao tác: xem chi tiết, tìm kiếm/lọc, thêm mới, chỉnh sửa, xóa, gán vai trò. 4. Hệ thống xử lý yêu cầu và cập nhật dữ liệu. 5. Hệ thống hiển thị kết quả thao tác cho quản trị viên.
Alternative Flow	Ở bước 3: – 3a. Quản trị viên thực hiện tìm kiếm/lọc, hệ thống hiển thị danh sách phù hợp. – 3b. Quản trị viên hủy thao tác thêm/sửa, hệ thống quay về danh sách mà không đổi dữ liệu.
Exception	Ở bước 1: nếu không có quyền, hệ thống chặn truy cập module. Ở bước 4: nếu dữ liệu không hợp lệ, hệ thống báo lỗi và yêu cầu nhập lại.

Bảng 8: Bảng mô tả Use Case UC01 - Quản lý thành viên

Use Case Name	UC01.1 - Thêm một thành viên từ Form
Actor(s)	Quản trị viên
Description	Quản trị viên thêm một thành viên mới vào hệ thống thông qua form nhập liệu.
Trigger	Quản trị viên bấm nút ”Thêm thành viên mới”trong trang quản lý người dùng.
Pre-condition(s)	Quản trị viên đã đăng nhập vào hệ thống.
Post-condition(s)	Thành viên mới được tạo và lưu vào hệ thống; thông tin được hiển thị lại đúng sau khi xử lý.
Main Flow	1. Quản trị viên bấm nút ”Thêm thành viên mới”. 2. Hệ thống hiển thị form nhập thông tin thành viên. 3. Quản trị viên điền thông tin vào form. 4. Quản trị viên bấm nút ”Tạo mới”. 5. Hệ thống kiểm tra tính hợp lệ của dữ liệu. 6. Hệ thống lưu thông tin thành viên và hiển thị thông báo thành công.
Alternative Flow	Ở bước 5, nếu người dùng không nhập mật khẩu thì hệ thống tự động tạo mật khẩu mặc định.
Exception	Ở bước 5, nếu dữ liệu nhập không hợp lệ hoặc thông tin đã tồn tại thì hệ thống hiển thị thông báo lỗi và yêu cầu nhập lại.

Bảng 9: Bảng mô tả Use Case UC01.1 - Thêm một thành viên từ Form

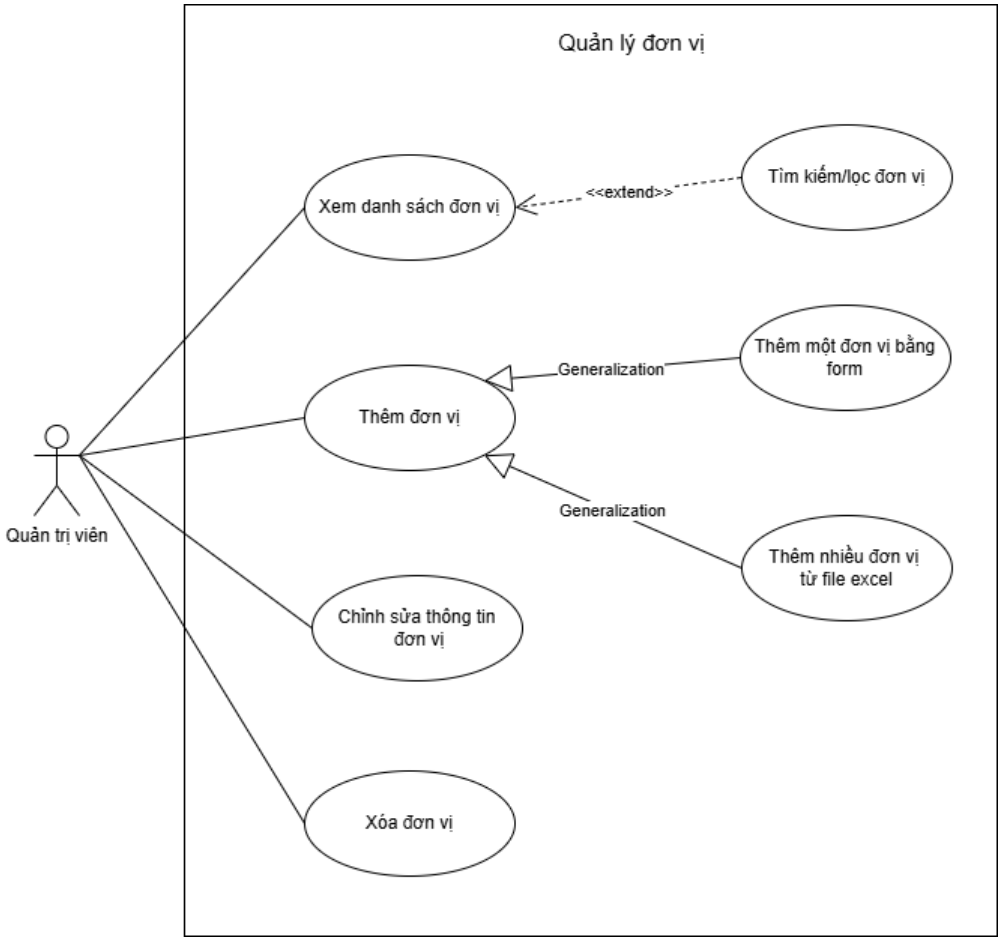
Use Case Name	**UC01.2 - Thêm nhiều thành viên từ file Excel**
Actor(s)	Quản trị viên
Description	Quản trị viên thêm hàng loạt thành viên từ file Excel theo template chuẩn của hệ thống.
Trigger	Quản trị viên cần import nhiều thành viên cùng lúc (đầu năm học, nhập dữ liệu chuyển tiếp).
Pre-condition(s)	Quản trị viên đã đăng nhập; có quyền import thành viên; file Excel được chuẩn bị theo mẫu.
Post-condition(s)	Danh sách thành viên được import thành công; dữ liệu được lưu và hiển thị lại đúng.
Main Flow	1. Quản trị viên chọn chức năng ”Thêm từ File”. 2. Hệ thống hiển thị hướng dẫn và file mẫu. 3. Quản trị viên tải lên file Excel. 4. Hệ thống tiếp nhận file và kiểm tra dữ liệu. 5. Quản trị viên xác nhận thực hiện import. 6. Hệ thống import dữ liệu và thông báo kết quả.
Alternative Flow	Nếu dữ liệu chưa phù hợp, hệ thống hiển thị danh sách bản ghi lỗi để người dùng điều chỉnh.
Exception	File sai định dạng, file sai mẫu hoặc dữ liệu nhập không hợp lệ thì hệ thống báo lỗi và không import.

Bảng 10: Bảng mô tả Use Case UC01.2 - Thêm nhiều thành viên từ file Excel

Use Case Name	**UC01.3 - Gán vai trò cho thành viên**
Actor(s)	Quản trị viên
Description	Quản trị viên gán một hoặc nhiều vai trò cho thành viên theo phạm vi toàn hệ thống hoặc theo tổ chức cụ thể.
Trigger	Quản trị viên cần phân quyền để thành viên thực hiện chức năng trong hệ thống.
Pre-condition(s)	Quản trị viên đã đăng nhập; có quyền gán vai trò; thành viên và danh sách vai trò đã tồn tại.
Post-condition(s)	Vai trò của thành viên được cập nhật; thông tin phân quyền được lưu lại.
Main Flow	1. Quản trị viên mở trang chi tiết thành viên. 2. Hệ thống hiển thị thông tin thành viên và danh sách vai trò hiện tại. 3. Quản trị viên chọn chức năng gán vai trò. 4. Hệ thống hiển thị form chọn vai trò. 5. Quản trị viên chọn vai trò và lưu thay đổi. 6. Hệ thống kiểm tra dữ liệu, cập nhật và thông báo thành công.
Alternative Flow	Quản trị viên có thể chọn phạm vi áp dụng vai trò nếu cần.
Exception	Nếu vai trò đã tồn tại, thiếu phạm vi áp dụng hoặc không đủ quyền thì hệ thống hiển thị thông báo lỗi.

Bảng 11: Bảng mô tả Use Case UC01.3 - Gán vai trò cho thành viên

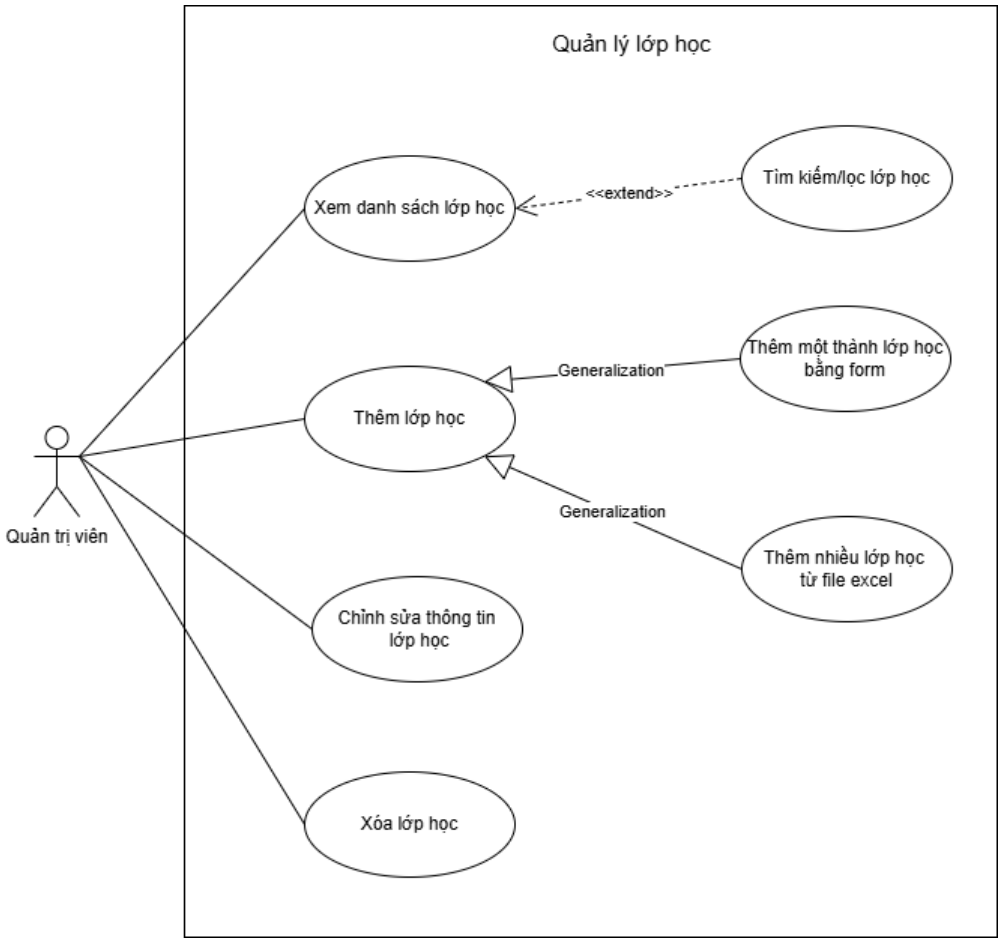
3.4.2 Phân hệ quản lý khoa, lớp và tổ chức



Hình 17: UC02 - Quản lý khoa/đơn vị

Use Case Name	**UC02 - Quản lý đơn vị**
Actor(s)	Quản trị viên
Description	Use case này cho phép quản trị viên thực hiện các thao tác quản lý thông tin đơn vị trong hệ thống bao gồm: – Xem danh sách đơn vị – Tìm và lọc đơn vị – Thêm, xóa, sửa thông tin đơn vị.
Trigger	Quản trị viên truy cập chức năng quản lý đơn vị từ giao diện hệ thống.
Pre-condition(s)	Quản trị viên đã đăng nhập bằng tài khoản có phân quyền quản lý đơn vị.
Post-condition(s)	Thông tin đơn vị được hiển thị và cập nhật đúng theo thao tác đã thực hiện.
Main Flow	1. Quản trị viên truy cập module Quản lý đơn vị. 2. Hệ thống hiển thị danh sách đơn vị. 3. Quản trị viên chọn xem/tìm/lọc/thêm/sửa/xóa đơn vị.
Alternative Flow	Ở bước 3, quản trị viên tìm kiếm/lọc; hệ thống trả về danh sách phù hợp điều kiện.
Exception	Ở bước 1, nếu không có quyền truy cập thì hệ thống chặn và không hiển thị module.

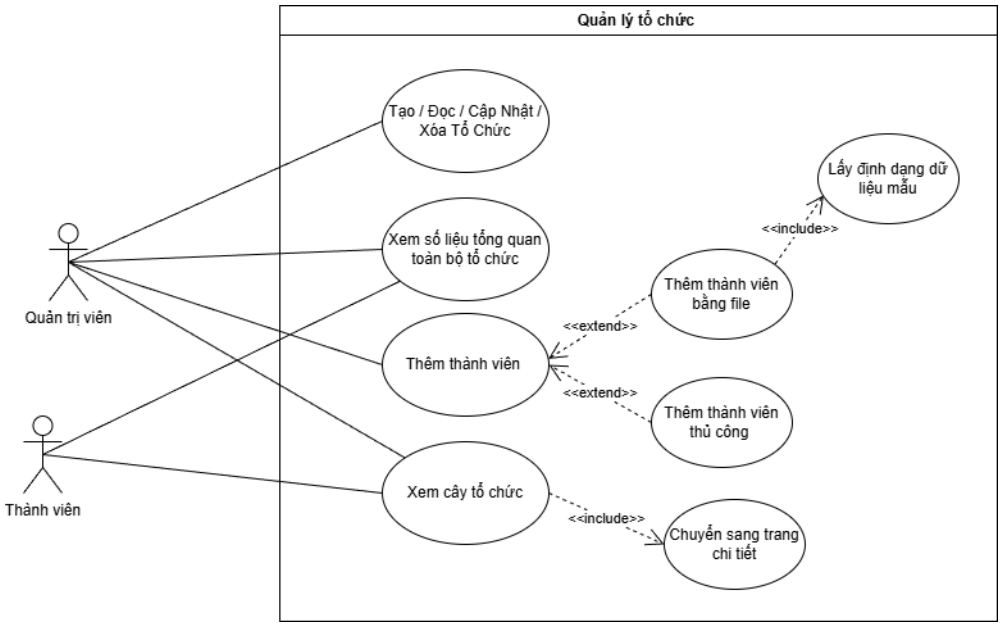
Bảng 12: Bảng mô tả Use Case UC02 - Quản lý đơn vị



Hình 18: UC03 - Quản lý lớp

Use Case Name	**UC03 - Quản lý lớp học**
Actor(s)	Quản trị viên
Description	Use case này cho phép quản trị viên thực hiện các thao tác quản lý thông tin lớp học trong hệ thống bao gồm: – Xem danh sách lớp học – Tìm và lọc lớp học – Thêm, xóa, sửa thông tin lớp học.
Trigger	Quản trị viên truy cập chức năng quản lý lớp học từ giao diện hệ thống.
Pre-condition(s)	Quản trị viên đã đăng nhập bằng tài khoản có phân quyền quản lý lớp học.
Post-condition(s)	Thông tin lớp học được hiển thị và cập nhật đúng theo thao tác đã thực hiện.
Main Flow	1. Quản trị viên truy cập module Quản lý lớp học. 2. Hệ thống hiển thị danh sách lớp học. 3. Quản trị viên chọn xem/tìm/lọc/thêm/sửa/xóa lớp học.
Alternative Flow	Ở bước 3, quản trị viên tìm kiếm/lọc lớp; hệ thống trả về danh sách phù hợp.
Exception	Ở bước 1, nếu không có phân quyền truy cập thì hệ thống chặn truy cập module.

Bảng 13: Bảng mô tả Use Case UC03 - Quản lý lớp học



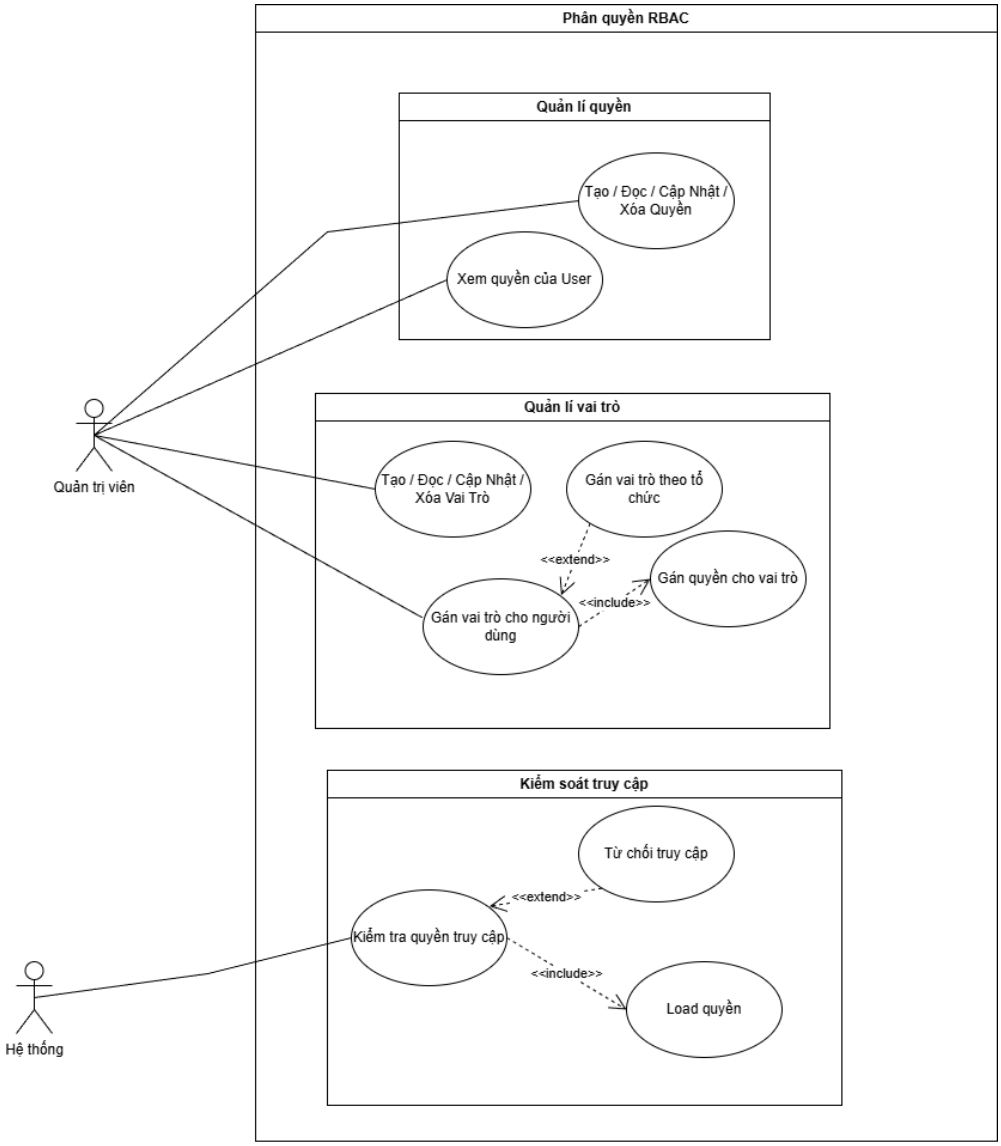
Hình 19: UC04 - Quản lý tổ chức Đoàn – Hội

Use Case Name	**UC04 - Quản lý tổ chức**
Actor(s)	Quản trị viên, Thành viên
Description	Cho phép xem cấu trúc cây tổ chức và số liệu tổng quan. Với Quản trị viên, hệ thống hỗ trợ thêm/sửa/xóa tổ chức và thêm thành viên (thủ công hoặc từ file).
Trigger	Người dùng truy cập trang Quản lý tổ chức từ thanh điều hướng chính.
Pre-condition(s)	Người dùng đã đăng nhập; dữ liệu tổ chức và thành viên đã tồn tại trong cơ sở dữ liệu.
Post-condition(s)	Cấu trúc tổ chức và thống kê được hiển thị chính xác; các thay đổi (nếu có) được lưu vào hệ thống.
Main Flow	1. Người dùng yêu cầu xem danh sách tổ chức. 2. Hệ thống hiển thị dữ liệu dạng cây phân cấp kèm thống kê. 3. Người dùng mở rộng/thu gọn các nhánh tổ chức. 4. Người dùng chọn một đơn vị cụ thể. 5. Hệ thống chuyển sang trang chi tiết tổ chức.
Alternative Flow	Ở bước 3 (quản trị): thêm thành viên thủ công hoặc bằng file; thực hiện CRUD tổ chức theo phân quyền.
Exception	File import sai định dạng mẫu; dữ liệu tạo/sửa tổ chức bị trùng hoặc thiếu thông tin bắt buộc.

Bảng 14: Bảng mô tả Use Case UC04 - Quản lý tổ chức

Nhóm use-case này mô tả luồng quản trị cơ cấu tổ chức nhiều cấp, đồng thời là nền tảng để gắn quyền theo ngữ cảnh đơn vị.

3.4.3 Phân hệ phân quyền và cấu hình truy cập



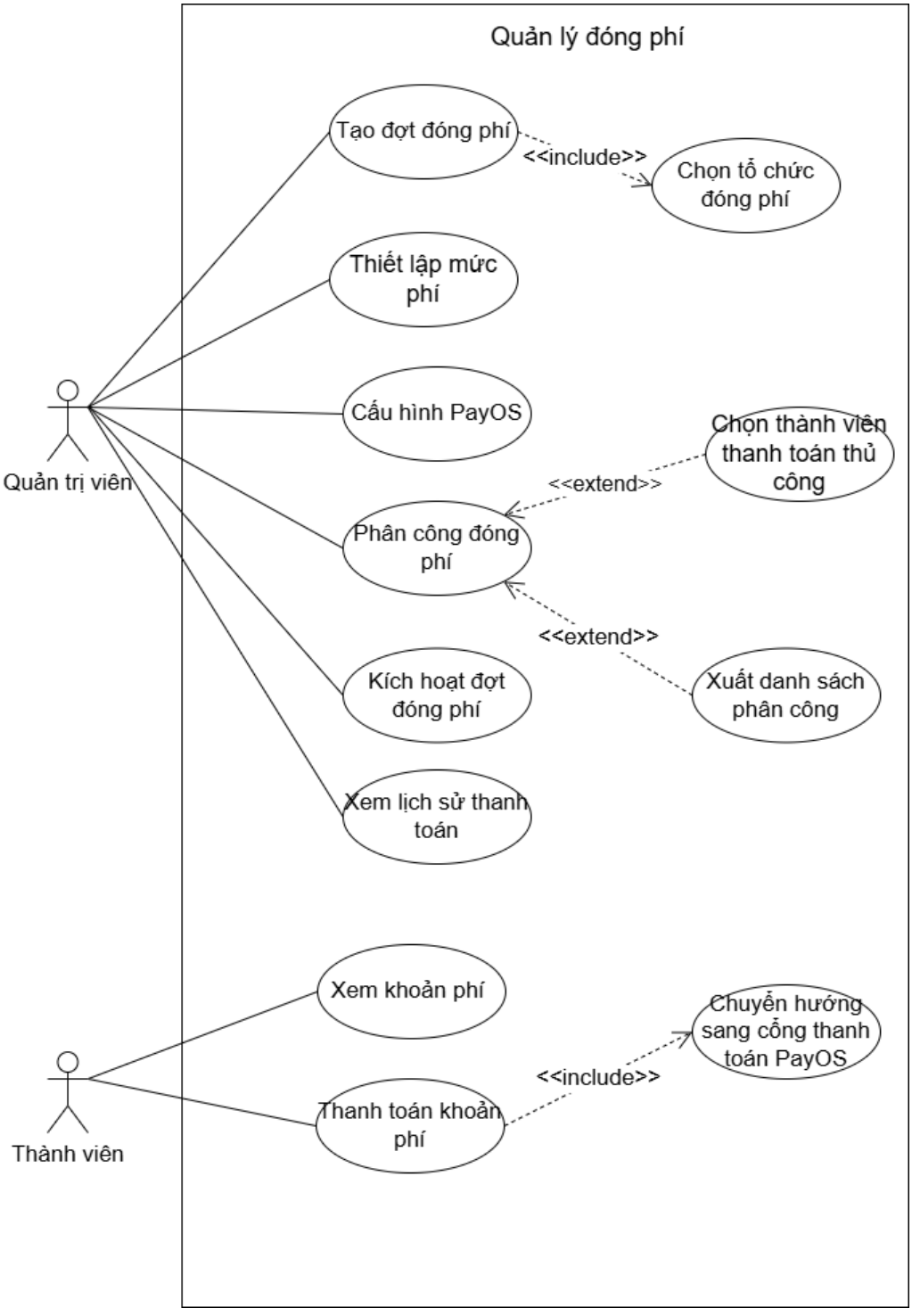
Hình 20: UC05 - Gán vai trò và phân quyền

Use Case Name	**UC05 - Gán vai trò và phân quyền**
Actor(s)	Quản trị viên, Hệ thống
Description	Quản trị viên cấp quyền cho người dùng bằng cách gán vai trò; hệ thống tự động liên kết các quyền tương ứng và hỗ trợ phạm vi theo tổ chức khi cần.
Trigger	Quản trị viên chọn tài khoản người dùng và mở màn hình gán vai trò.
Pre-condition(s)	Quản trị viên đã đăng nhập và có quyền quản lý hệ thống; vai trò và tài khoản người dùng đã tồn tại.
Post-condition(s)	Người dùng được cập nhật vai trò/quyền tương ứng; dữ liệu phân quyền được lưu vào cơ sở dữ liệu.
Main Flow	1. Quản trị viên chọn người dùng cần phân quyền. 2. Hệ thống hiển thị danh sách vai trò khả dụng. 3. Quản trị viên chọn vai trò cần gán. 4. Hệ thống gắn các quyền tương ứng theo vai trò. 5. Quản trị viên xác nhận lưu thay đổi. 6. Hệ thống cập nhật và thông báo thành công.
Alternative Flow	Trường hợp vai trò có phạm vi tổ chức, quản trị viên chọn thêm tổ chức áp dụng trước khi lưu.
Exception	Lỗi kết nối cơ sở dữ liệu; tài khoản người dùng bị khóa/không hợp lệ nên hệ thống từ chối gán quyền.

Bảng 15: Bảng mô tả Use Case UC05 - Gán vai trò và phân quyền

Phân hệ này cho phép gán vai trò và quyền theo tổ chức, hỗ trợ kiểm soát truy cập linh hoạt giữa các cấp quản trị.

3.4.4 Phân hệ thu phí



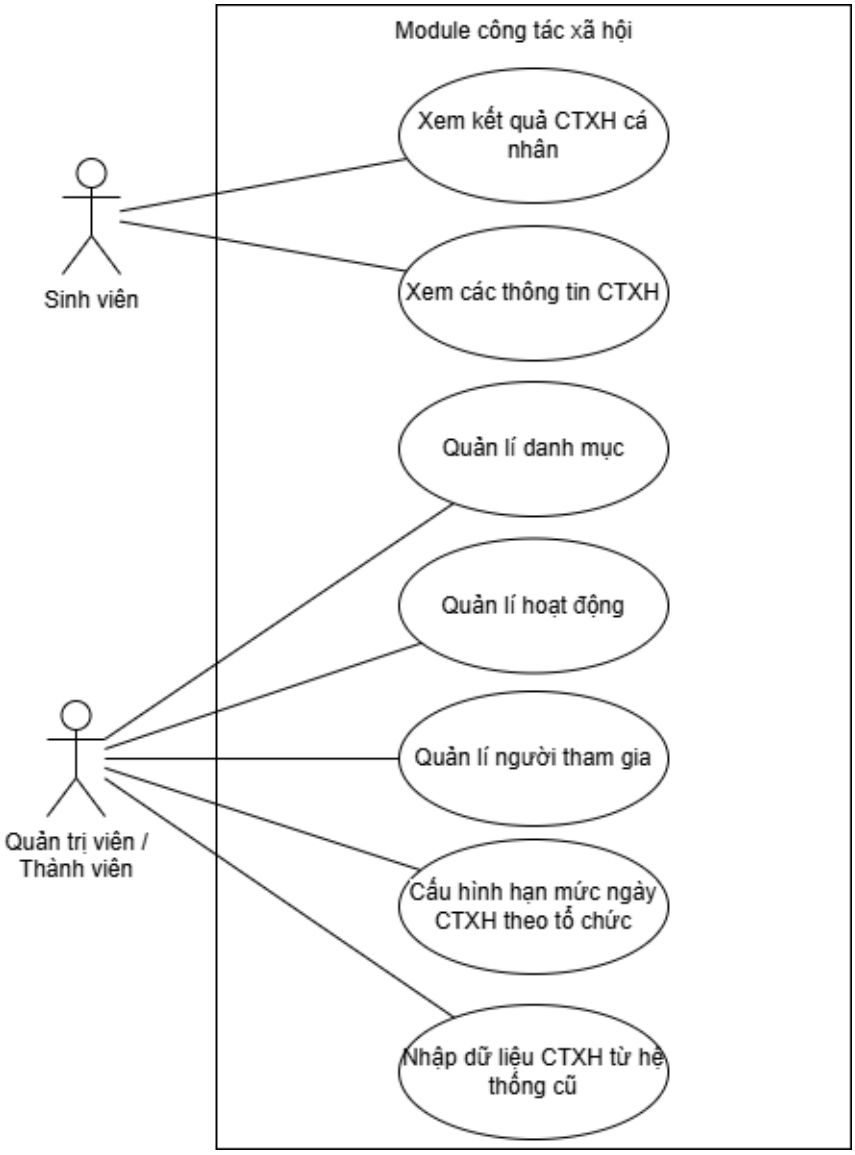
Hình 21: UC06 - Quản lý đóng phí

Phân hệ thu phí mô tả đầy đủ các use-case từ tạo đợt thu, phân công đối tượng, thanh toán và đối soát trạng thái giao dịch với cổng thanh toán.

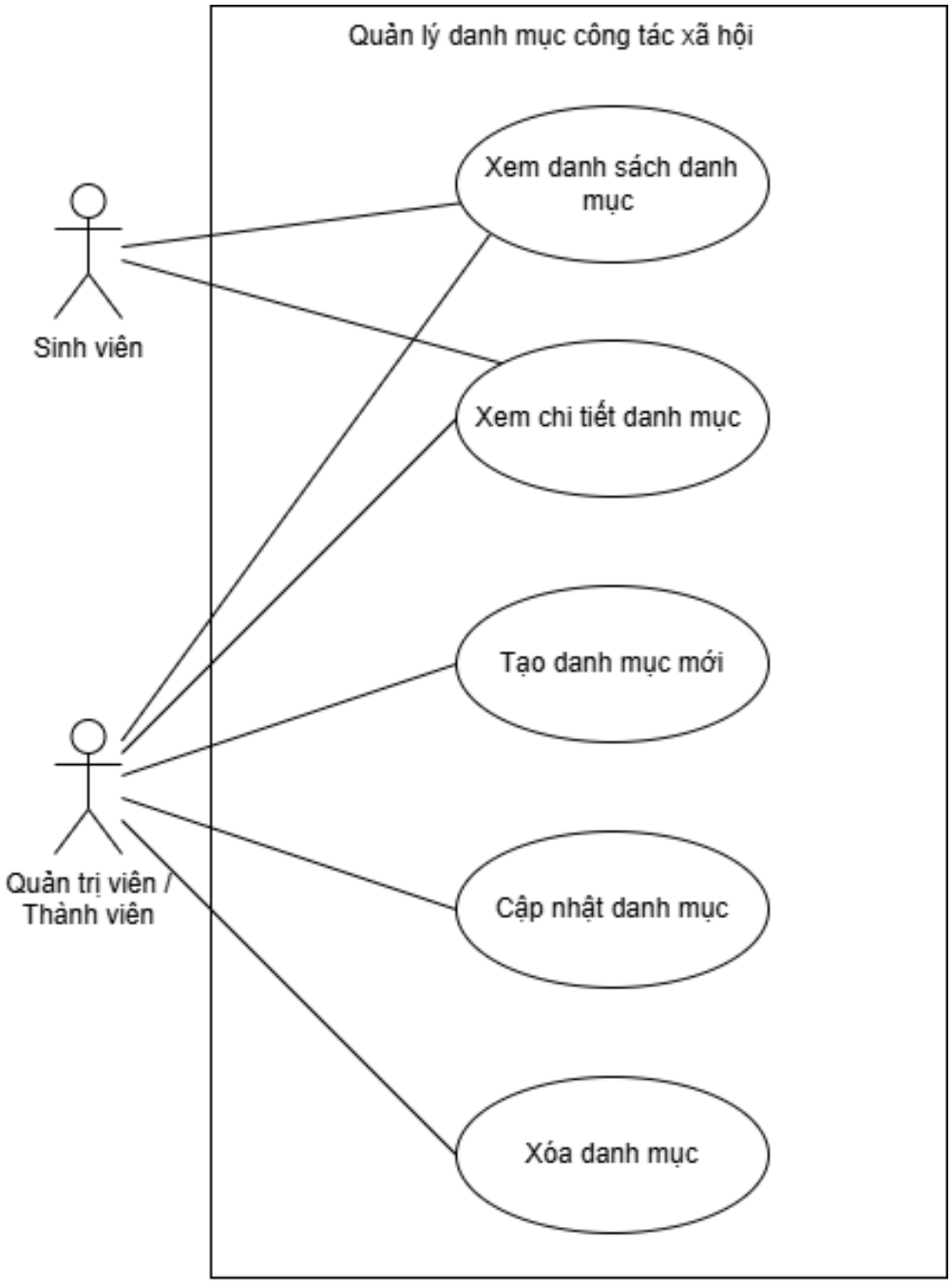
Use Case Name	UC06 - Quản lý đóng phí
Actor(s)	Quản trị viên, Thành viên
Description	Quản lý trọn luồng thu phí: tạo đợt thu, cấu hình mức phí, phân công đối tượng, kích hoạt và theo dõi thanh toán.
Trigger	Quản trị viên tạo đợt thu mới hoặc thành viên truy cập chức năng đóng phí.
Pre-condition(s)	Có quyền quản lý đợt thu; đợt thu có mức phí và danh sách phân công hợp lệ; cổng thanh toán sẵn sàng khi thu online.
Post-condition(s)	Đợt thu được kích hoạt; giao dịch thanh toán được ghi nhận và cập nhật trạng thái trong lịch sử thu phí.
Main Flow	1. Quản trị viên tạo đợt thu và nhập thông tin cơ bản. 2. Cấu hình mức phí, phân công thành viên và kích hoạt đợt thu. 3. Thành viên truy cập khoản phí của tôi và tạo thanh toán. 4. Hệ thống tạo liên kết thanh toán. 5. Hệ thống nhận/truy vấn trạng thái giao dịch và cập nhật kết quả.
Alternative Flow	Ở bước 3, thành viên có thể đóng thủ công; quản trị viên cập nhật trạng thái đã đóng trực tiếp trên hệ thống.
Exception	Thiếu cấu hình trước khi kích hoạt; lỗi kết nối cổng thanh toán; giao dịch bị hủy hoặc hết hạn.

Bảng 16: Bảng mô tả Use Case UC06 - Quản lý đóng phí

3.4.5 Phân hệ CTXH



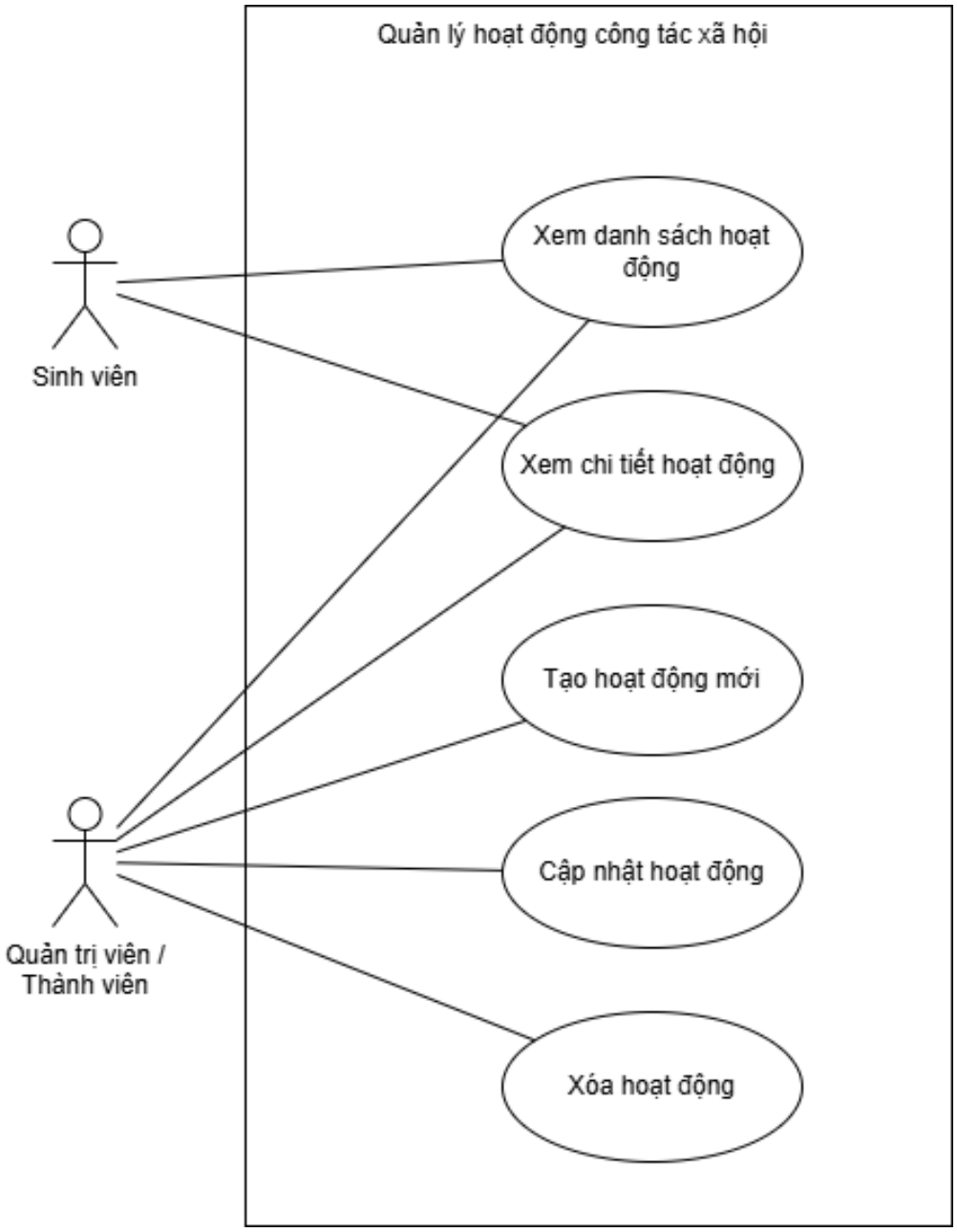
Hình 22: UC07 - Quản lý công tác xã hội (tổng quát)



Hình 23: UC07.1 - Quản lý danh mục CTXH

Use Case Name	**UC07.1 - Quản lý danh mục CTXH**
Actor(s)	Quản trị viên / Thành viên, Sinh viên
Description	Quản lý danh mục phân loại hoạt động CTXH, bao gồm tạo mới, cập nhật, xem chi tiết và xóa theo điều kiện dữ liệu; hỗ trợ truy cập trực tiếp (direct view) từ danh mục sang danh sách hoạt động và chi tiết hoạt động tương ứng.
Trigger	Người dùng truy cập chức năng danh mục CTXH.
Pre-condition(s)	Người dùng đã đăng nhập và có quyền quản lý danh mục CTXH.
Post-condition(s)	Danh mục CTXH được tạo mới, cập nhật hoặc xóa theo thao tác đã thực hiện.
Main Flow	1. Người dùng mở danh sách danh mục. 2. Hệ thống hiển thị danh sách danh mục và thông tin liên quan. 3. Người dùng chọn xem chi tiết danh mục để mở danh sách hoạt động thuộc danh mục (direct view). 4. Từ danh sách hoạt động, người dùng mở chi tiết hoạt động và chuyển sang màn hình danh sách người tham gia của hoạt động đã chọn. 5. Người dùng có quyền phù hợp có thể tạo mới, chỉnh sửa hoặc xóa danh mục. 6. Hệ thống kiểm tra dữ liệu và lưu thay đổi.
Alternative Flow	Sinh viên có thể xin tham gia hoạt động trực tiếp từ màn hình chi tiết hoạt động khi đi từ danh mục.
Exception	Nếu danh mục đã có hoạt động liên kết thì hệ thống không cho xóa.

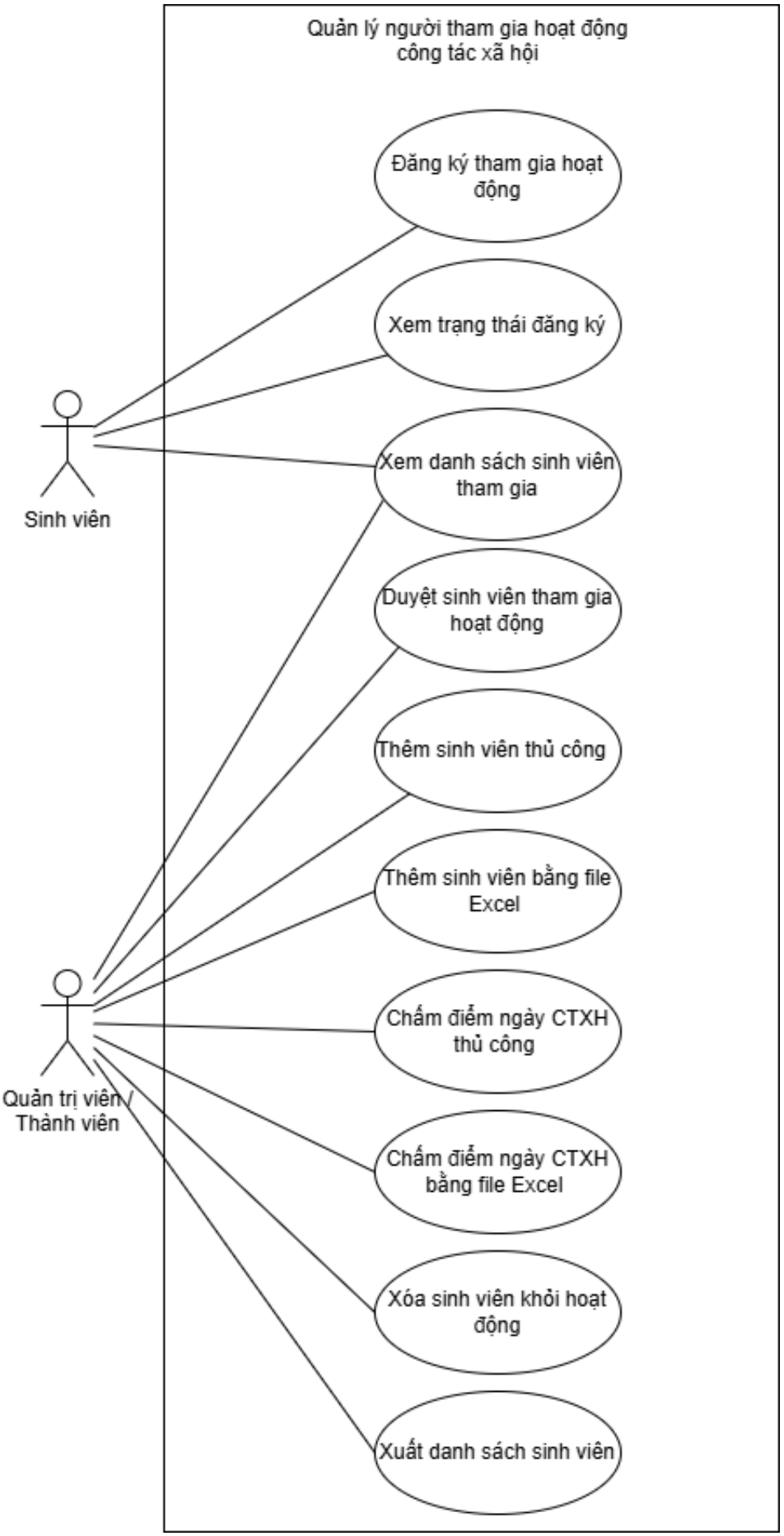
Bảng 17: Bảng mô tả Use Case UC07.1 - Quản lý danh mục CTXH



Hình 24: UC07.2 - Quản lý hoạt động CTXH

Use Case Name	**UC07.2 - Quản lý hoạt động CTXH**
Actor(s)	Quản trị viên / Thành viên, Sinh viên
Description	Quản lý hoạt động CTXH gồm xem danh sách/chi tiết, tạo mới, cập nhật và xóa hoạt động theo quyền; hỗ trợ direct view từ danh mục sang chi tiết hoạt động, danh sách người tham gia và luồng xin tham gia hoạt động.
Trigger	Người dùng truy cập chức năng hoạt động CTXH.
Pre-condition(s)	Người dùng đã đăng nhập và có quyền quản lý hoạt động CTXH.
Post-condition(s)	Hoạt động được tạo mới/cập nhật/xóa đúng theo thao tác và hiển thị lại trên danh sách.
Main Flow	1. Người dùng mở danh sách hoạt động CTXH. 2. Hệ thống hiển thị danh sách và thông tin chi tiết hoạt động được chọn (bao gồm truy cập từ danh mục). 3. Người dùng mở danh sách người tham gia của hoạt động để theo dõi hoặc quản lý theo quyền. 4. Sinh viên có thể xin tham gia hoạt động từ màn hình chi tiết hoạt động. 5. Người dùng có quyền quản lý tạo mới hoặc cập nhật hoạt động. 6. Hệ thống kiểm tra dữ liệu và lưu thay đổi.
Alternative Flow	Người dùng có quyền quản lý có thể xóa hoạt động khi thỏa điều kiện nghiệp vụ; nếu không có quyền thì chỉ xem chi tiết và danh sách người tham gia.
Exception	Nếu dữ liệu thời gian hoặc trạng thái không hợp lệ thì hệ thống hiển thị thông báo lỗi.

Bảng 18: Bảng mô tả Use Case UC07.2 - Quản lý hoạt động CTXH



Hình 25: UC07.3 - Quản lý người tham gia hoạt động CTXH

Use Case Name	**UC07.3 - Quản lý người tham gia hoạt động CTXH**
Actor(s)	Quản trị viên / Thành viên, Sinh viên
Description	Quản lý toàn bộ vòng đời tham gia CTXH: đăng ký, theo dõi trạng thái, duyệt sinh viên tham gia, thêm/xóa danh sách và chấm điểm kết quả theo từng hoạt động; hỗ trợ cấu hình duyệt tự động (auto-accept) và giới hạn số lượng tối đa người tham gia (max participants).
Trigger	Người dùng truy cập màn hình người tham gia của một hoạt động.
Pre-condition(s)	Hoạt động đã tồn tại; người dùng có quyền quản lý người tham gia.
Post-condition(s)	Trạng thái đăng ký, danh sách tham gia và kết quả CTXH được cập nhật; dữ liệu phản ánh lên báo cáo cá nhân.
Main Flow	1. Sinh viên đăng ký tham gia hoạt động và theo dõi trạng thái đăng ký. 2. Hệ thống kiểm tra số lượng hiện tại so với max participants trước khi ghi nhận đăng ký hoặc thêm vào danh sách. 3. Nếu hoạt động bật auto-accept thì hệ thống tự động chuyển trạng thái sang đã duyệt; nếu không thì chuyển sang chờ duyệt. 4. Quản trị viên/thành viên phụ trách mở danh sách đăng ký và duyệt các trường hợp chờ duyệt (hệ thống kiểm tra lại max participants trước khi xác nhận duyệt). 5. Cập nhật danh sách tham gia (thêm thủ công hoặc import file Excel) và chỉ lưu các bản ghi hợp lệ theo giới hạn số lượng. 6. Chấm điểm ngày CTXH (thủ công hoặc bằng file Excel). 7. Hệ thống lưu thay đổi và cập nhật thống kê.
Alternative Flow	Ở bước quản lý danh sách, quản trị viên/thành viên phụ trách có thể: – Xóa sinh viên khỏi hoạt động (hệ thống kiểm tra nếu còn chỗ thì tự động duyệt đăng ký chờ). – Xuất danh sách sinh viên tham gia ra file Excel để phục vụ vận hành hoặc in biểu mẫu.
Exception	Nếu dữ liệu đăng ký không hợp lệ, sinh viên trùng lặp, vượt max participants, hoặc file nhập/chấm điểm sai định dạng thì hệ thống báo lỗi và không cập nhật bản ghi lỗi.

Bảng 19: Bảng mô tả Use Case UC07.3 - Quản lý người tham gia hoạt động CTXH

Use Case Name	**UC07.4 - Quản lý hạn mức ngày CTXH theo tổ chức**
Actor(s)	Quản trị viên
Description	Quản trị viên thiết lập số ngày CTXH tối đa được phép cho từng tổ chức; cấu hình có thể tự động áp dụng cho tổ chức con.
Trigger	Quản trị viên muốn giới hạn số ngày CTXH theo tổ chức.
Pre-condition(s)	Quản trị viên đã đăng nhập; có quyền cấu hình giới hạn; tổ chức tồn tại trong hệ thống.
Post-condition(s)	Cấu hình giới hạn được tạo và lưu lại; thông tin cấu hình được phản ánh trên hệ thống.
Main Flow	1. Quản trị viên truy cập chức năng cấu hình giới hạn. 2. Chọn tổ chức, nhập số ngày tối đa và trạng thái. 3. Hệ thống kiểm tra tính hợp lệ của dữ liệu. 4. Hệ thống lưu cấu hình cho tổ chức. 5. Hệ thống áp dụng cho các tổ chức con phù hợp.
Alternative Flow	Nếu số ngày không hợp lệ hoặc tổ chức đã có cấu hình đang hoạt động, hệ thống hiển thị thông báo phù hợp.
Exception	Tổ chức không tồn tại thì hệ thống báo lỗi và không lưu cấu hình.

Bảng 20: Bảng mô tả Use Case UC07.4 - Quản lý hạn mức ngày CTXH theo tổ chức

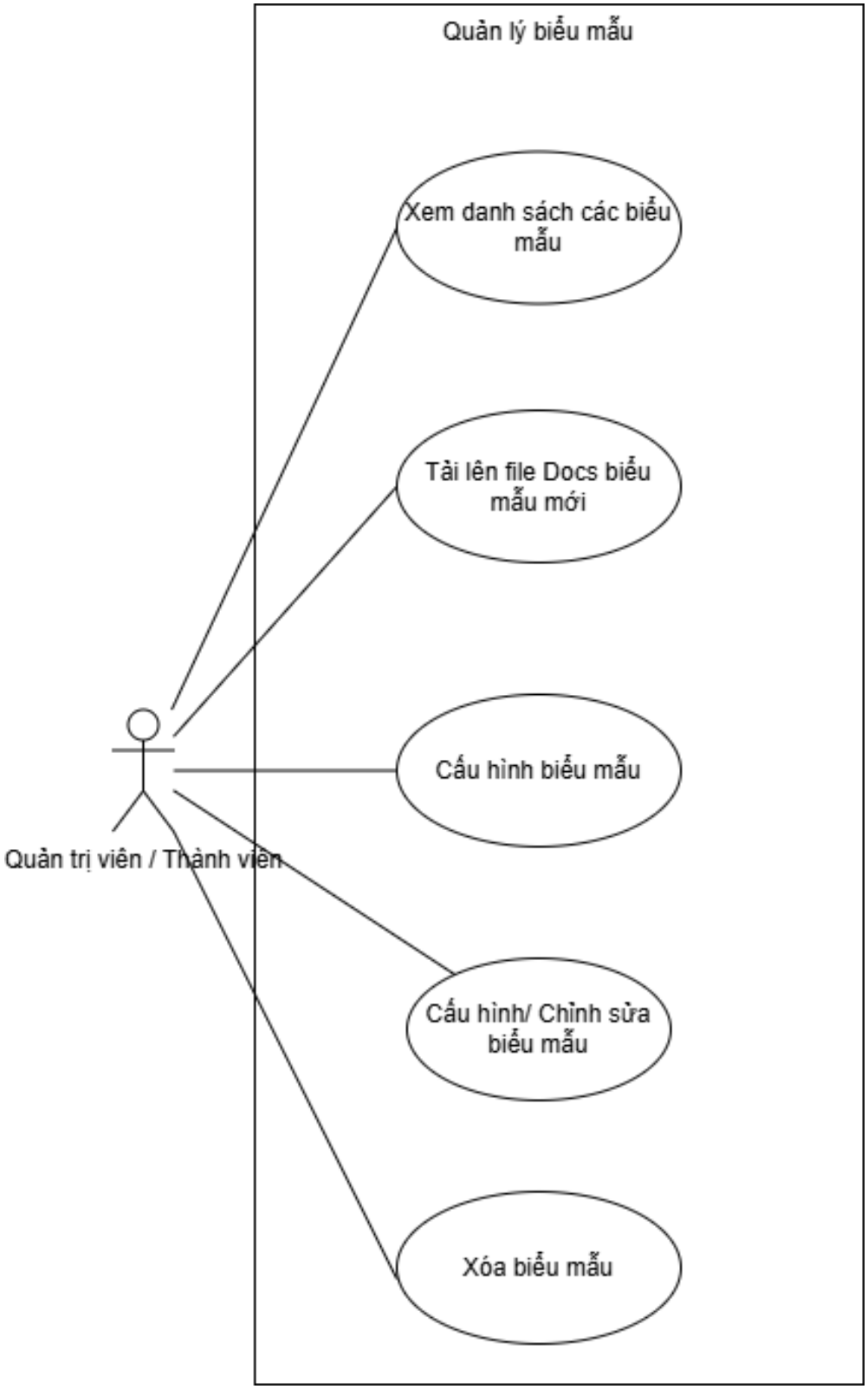
Use Case Name	**UC07.5 - Xem báo cáo ngày công tác xã hội cá nhân**
Actor(s)	Sinh viên
Description	Sinh viên xem báo cáo CTXH cá nhân bao gồm tổng số ngày đã tích lũy, trạng thái đạt/chưa đạt và danh sách chi tiết các hoạt động đã tham gia.
Trigger	Sinh viên muốn kiểm tra tiến độ CTXH của mình.
Pre-condition(s)	Người dùng đã đăng nhập; tài khoản sinh viên có mã số sinh viên trong hệ thống.
Post-condition(s)	Báo cáo CTXH cá nhân được hiển thị đầy đủ thông tin.
Main Flow	1. Sinh viên chọn chức năng xem báo cáo CTXH. 2. Hệ thống truy vấn dữ liệu và tính tổng số ngày CTXH. 3. Hệ thống kiểm tra trạng thái đạt/chưa đạt. 4. Hệ thống hiển thị báo cáo cho sinh viên.
Alternative Flow	Nếu sinh viên chưa tham gia hoạt động nào, hệ thống thông báo tương ứng và gợi ý xem danh sách hoạt động.
Exception	Nếu không tìm thấy thông tin sinh viên thì hệ thống hiển thị lỗi tài khoản không hợp lệ.

Bảng 21: Bảng mô tả Use Case UC07.5 - Xem báo cáo ngày công tác xã hội cá nhân

Use Case Name	**UC07.6 - Nhập dữ liệu ngày CTXH từ hệ thống cũ**
Actor(s)	Quản trị viên
Description	Quản trị viên nhập dữ liệu số ngày CTXH từ hệ thống cũ vào hồ sơ sinh viên bằng file Excel.
Trigger	Quản trị viên muốn nhập dữ liệu CTXH từ hệ thống cũ khi triển khai hệ thống mới.
Pre-condition(s)	Quản trị viên đã đăng nhập; có quyền nhập dữ liệu; file Excel được chuẩn bị theo mẫu.
Post-condition(s)	Số ngày CTXH của sinh viên được cập nhật đúng theo dữ liệu nhập.
Main Flow	1. Quản trị viên vào chức năng nhập dữ liệu hệ thống cũ. 2. Tải mẫu Excel, điền dữ liệu và tải lên. 3. Hệ thống kiểm tra dữ liệu trong file. 4. Hệ thống hiển thị kết quả kiểm tra. 5. Quản trị viên xác nhận nhập dữ liệu. 6. Hệ thống cập nhật dữ liệu và thông báo kết quả.
Alternative Flow	Nếu có dòng dữ liệu không hợp lệ, hệ thống đánh dấu lỗi và tiếp tục các dòng còn lại.
Exception	Nếu file sai định dạng hoặc xảy ra lỗi xử lý hàng loạt thì hệ thống báo lỗi và dừng thao tác.

Bảng 22: Bảng mô tả Use Case UC07.6 - Nhập dữ liệu ngày CTXH từ hệ thống cũ

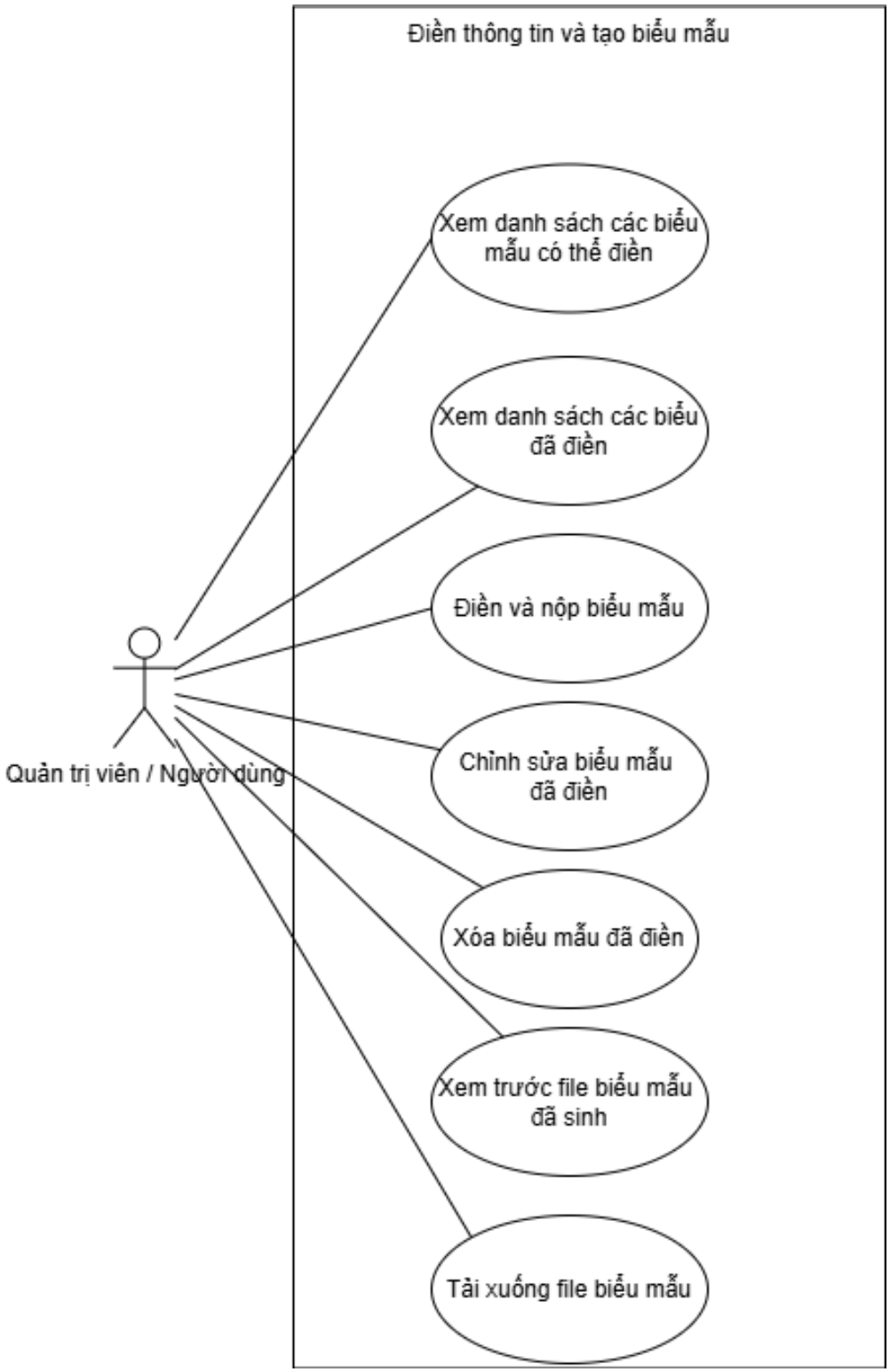
3.4.6 Phân hệ biểu mẫu



Hình 26: UC08.1 - Quản lý biểu mẫu

Use Case Name	**UC08.1 - Quản lý biểu mẫu**
Actor(s)	Quản trị viên / Thành viên
Description	Quản lý biểu mẫu Word động, bao gồm tải lên template, cấu hình trường dữ liệu và quản lý danh sách biểu mẫu.
Trigger	Người dùng truy cập trang quản lý biểu mẫu.
Pre-condition(s)	Người dùng đã đăng nhập và có quyền quản lý biểu mẫu.
Post-condition(s)	Biểu mẫu được tạo mới, cấu hình hoặc xóa trong hệ thống.
Main Flow	1. Người dùng tải lên biểu mẫu mới. 2. Hệ thống phân tích template và hiển thị các trường động. 3. Người dùng cấu hình các trường dữ liệu. 4. Hệ thống lưu template và cấu hình.
Alternative Flow	Người dùng có thể chỉnh sửa cấu hình biểu mẫu đã tồn tại.
Exception	Nếu template không đúng định dạng hoặc hệ thống gặp lỗi lưu trữ thì báo lỗi tương ứng.

Bảng 23: Bảng mô tả Use Case UC08.1 - Quản lý biểu mẫu

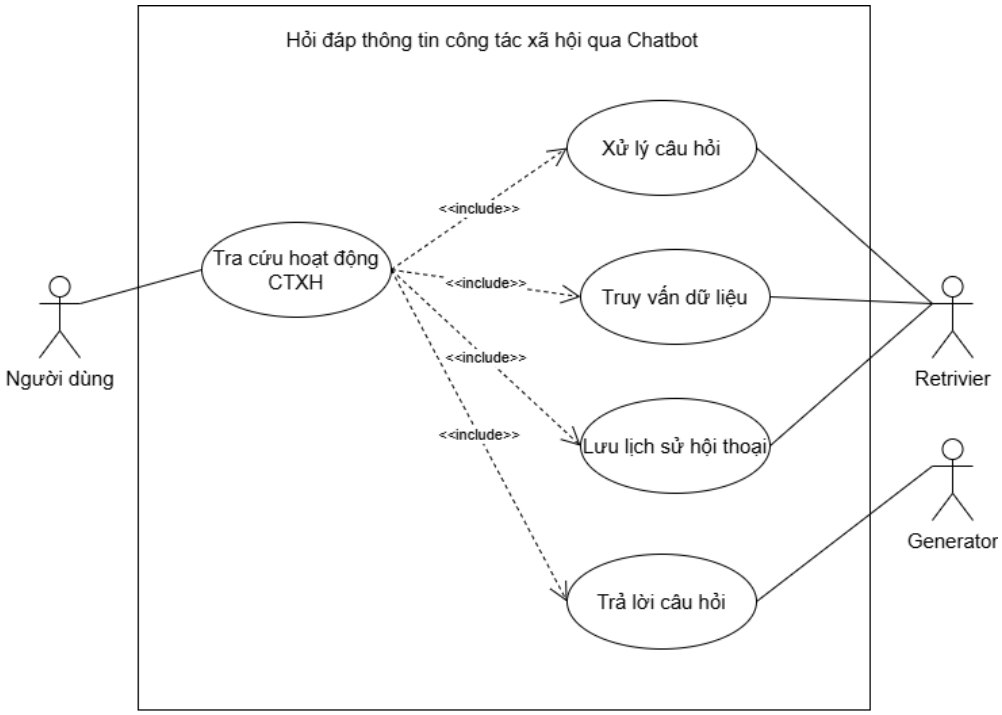


Hình 27: UC08.2 - Điền và nộp biểu mẫu

Use Case Name	**UC08.2 - Điền và nộp biểu mẫu**
Actor(s)	Quản trị viên / Người dùng
Description	Người dùng xem danh sách biểu mẫu có thể điền/đã điền, điền và nộp biểu mẫu, sau đó xem trước hoặc tải xuống file kết quả.
Trigger	Người dùng muốn điền biểu mẫu hoặc quản lý các biểu mẫu đã nộp.
Pre-condition(s)	Người dùng đã đăng nhập và có ít nhất một biểu mẫu trong hệ thống.
Post-condition(s)	Bản ghi nộp được tạo/cập nhật/xóa theo thao tác; file kết quả được sinh và lưu lại để xem trước hoặc tải xuống.
Main Flow	1. Người dùng xem danh sách biểu mẫu có thể điền hoặc danh sách biểu mẫu đã điền. 2. Người dùng chọn biểu mẫu và nhập thông tin. 3. Hệ thống kiểm tra dữ liệu, lưu bản nộp và sinh file kết quả. 4. Người dùng xem trước hoặc tải xuống file biểu mẫu đã sinh.
Alternative Flow	Người dùng có thể chỉnh sửa hoặc xóa biểu mẫu đã điền trước khi nộp phiên bản mới.
Exception	Nếu thiếu trường bắt buộc hoặc file kết quả không thể tạo thì hệ thống báo lỗi.

Bảng 24: Bảng mô tả Use Case UC08.2 - Điền và nộp biểu mẫu

3.4.7 Phân hệ chatbot CTXH



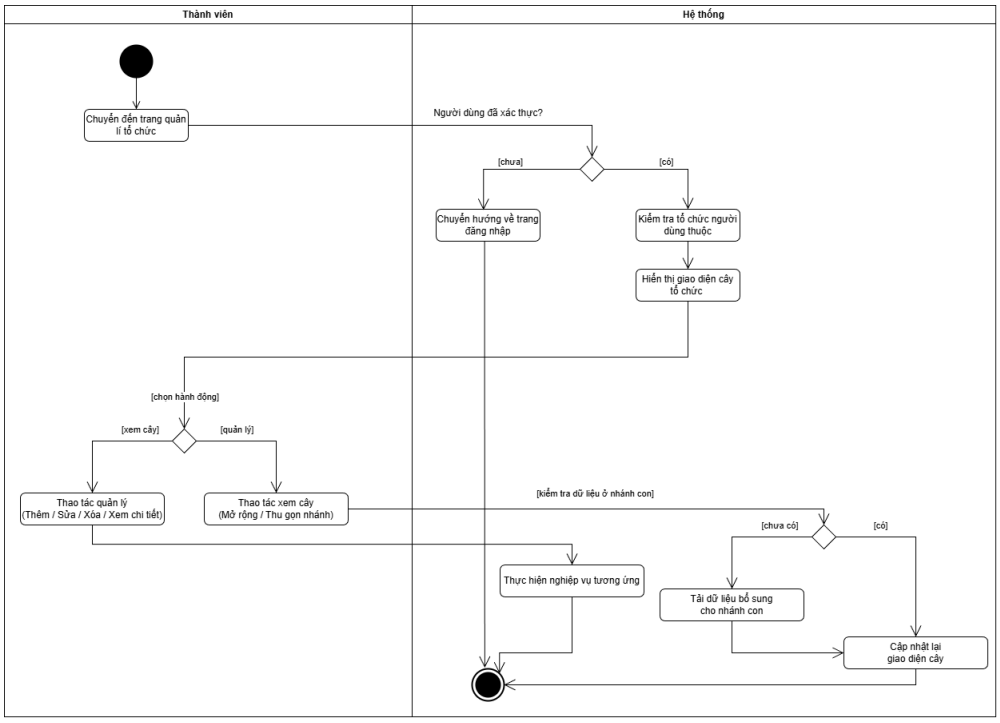
Hình 28: UC09 - Tra cứu CTXH bằng chatbot

Use Case Name	UC09 - Tra cứu CTXH bằng Chatbot
Actor(s)	Người dùng, Retriever, Generator
Description	Hỗ trợ hỏi đáp CTXH bằng ngôn ngữ tự nhiên dựa trên dữ liệu hoạt động đã được hệ thống thu thập và chuẩn bị trước.
Trigger	Người dùng truy cập màn hình chatbot và gửi câu hỏi.
Pre-condition(s)	Người dùng đã đăng nhập; hệ thống chatbot và dữ liệu kiến thức đang sẵn sàng.
Post-condition(s)	Người dùng nhận được câu trả lời phù hợp; lịch sử hội thoại được lưu lại.
Main Flow	1. Người dùng nhập câu hỏi về CTXH. 2. Hệ thống xử lý câu hỏi và truy vấn dữ liệu liên quan. 3. Hệ thống tổng hợp thông tin và tạo câu trả lời. 4. Hệ thống hiển thị kết quả cho người dùng.
Alternative Flow	Nếu không tìm thấy thông tin phù hợp, hệ thống thông báo và gợi ý người dùng đặt câu hỏi khác.
Exception	Nếu dịch vụ truy xuất hoặc sinh nội dung gặp sự cố thì hệ thống hiển thị thông báo lỗi và yêu cầu thử lại sau.

Bảng 25: Bảng mô tả Use Case UC09 - Tra cứu CTXH bằng Chatbot

3.5 Mô hình hóa Activity Diagram

3.5.1 Activity Diagram - Quản lý tổ chức



Hình 29: Activity Diagram - Quản lý tổ chức

Biểu đồ hoạt động tại hình trên mô tả luồng tương tác nghiệp vụ trên trang Quản lý tổ chức. Quy trình này không chỉ bao gồm các thao tác quản lý dữ liệu (CRUD) mà còn thể hiện chi tiết cơ chế tối ưu hóa trải nghiệm người dùng thông qua việc tải dữ liệu cây phân cấp theo nhu cầu. Quy trình được chia thành hai làn xử lý (swimlanes) là Thành viên và Hệ thống, bao gồm các giai đoạn chính sau:

Khởi tạo và kiểm soát truy cập Khi người dùng truy cập vào trang, hệ thống thực hiện chuỗi kiểm tra tự động trước khi hiển thị dữ liệu:

- Xác thực người dùng: Hệ thống kiểm tra trạng thái đăng nhập. Nếu chưa xác thực, người dùng bị chuyển hướng về trang đăng nhập.
- Khởi tạo ngữ cảnh: Nếu đã xác thực, hệ thống thực hiện bước kiểm tra tổ chức người dùng thuộc. Bước này giúp hệ thống xác định và ưu tiên hiển thị các đơn vị mà người dùng đang là thành viên lên đầu danh sách, sau đó mới hiển thị giao diện cây tổ chức hoàn chỉnh.

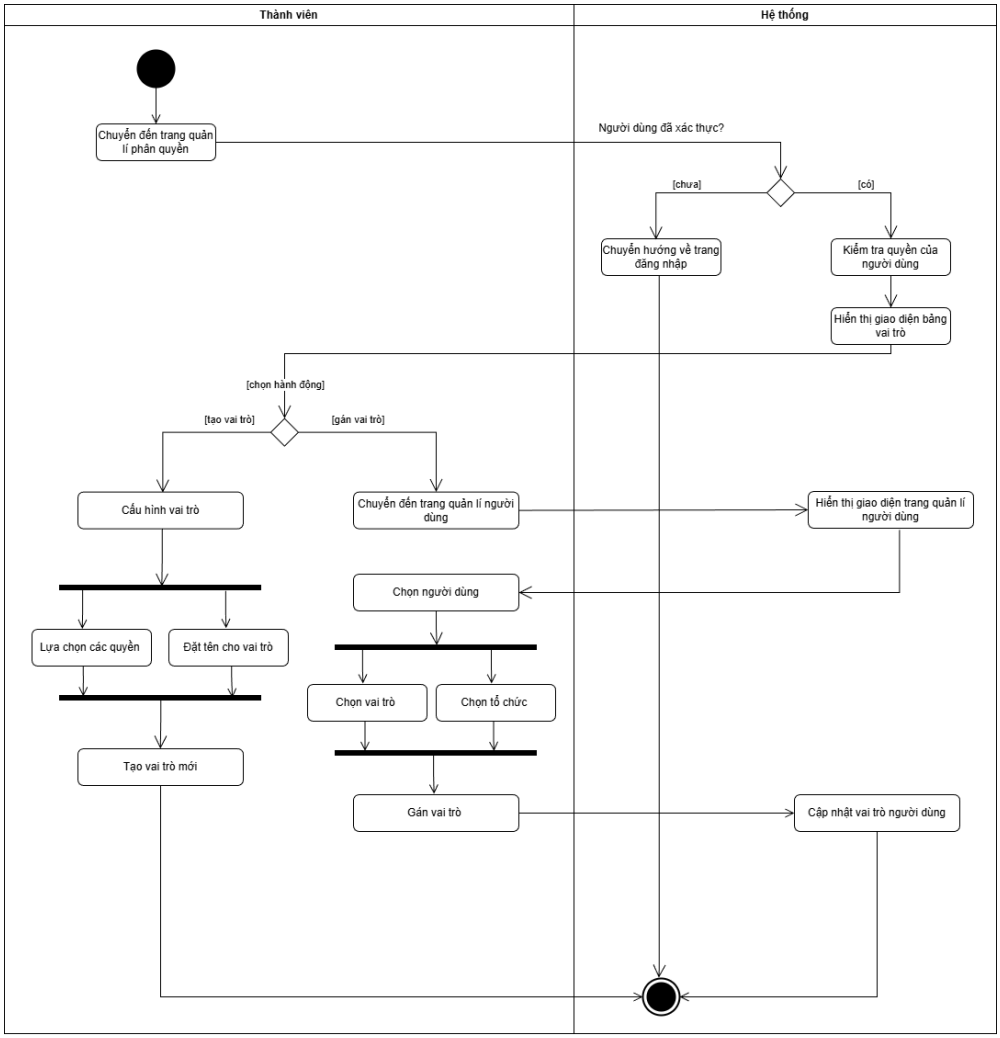
Phân loại hành động người dùng Tại giao diện chính, người dùng có thể thực hiện hai nhóm tác vụ, dẫn đến hai nhánh xử lý riêng biệt:

- Nhánh quản lý (CRUD): Bao gồm các thao tác thêm, sửa, xóa hoặc xem chi tiết. Hệ thống sẽ tiếp nhận và thực hiện các nghiệp vụ backend tương ứng rồi kết thúc luồng.
- Nhánh xem cây (View Tree): Bao gồm thao tác mở rộng (+) hoặc thu gọn (-) các nhánh con của cây tổ chức.

Cơ chế hiển thị và Lazy Loading Khi người dùng chọn thao tác mở rộng nhánh (thuộc nhóm thao tác xem cây), hệ thống áp dụng thuật toán tối ưu hóa dữ liệu:

1. Kiểm tra dữ liệu cục bộ: Hệ thống kiểm tra xem dữ liệu của nhánh con đó đã được tải về trình duyệt chưa (client-side check).
2. Quyết định tải dữ liệu:
    - Trường hợp chưa có (cache miss): Hệ thống thực hiện hành động tải dữ liệu bổ sung cho nhánh con từ máy chủ.
    - Trường hợp đã có (cache hit): Hệ thống bỏ qua bước tải mạng và sử dụng ngay dữ liệu có sẵn.
3. Cập nhật giao diện: Cuối cùng, hệ thống cập nhật lại hiển thị cây (re-render) để mở rộng nhánh mà người dùng vừa chọn.

3.5.2 Activity Diagram - Quản lý phân quyền



Hình 30: Activity Diagram - Quản lý phân quyền

Biểu đồ hoạt động tại hình trên mô tả chi tiết quy trình nghiệp vụ quản lý phân quyền, bao gồm các bước kiểm soát truy cập và hai luồng chức năng chính: cấu hình vai trò mới và gán vai trò cho người dùng. Quy trình được chia thành hai làn xử lý (swimlanes) là Thành viên (Quản trị viên) và Hệ thống, với các bước xử lý cụ thể như sau:

Kiểm soát truy cập ban đầu Khi người dùng truy cập vào trang quản lý phân quyền, hệ thống thực hiện lớp bảo mật đầu tiên:

- Kiểm tra xác thực: Hệ thống xác minh xem phiên làm việc của người dùng có hợp lệ không.
- Điều hướng:
  - Nếu chưa xác thực: Hệ thống tự động chuyển hướng về trang đăng nhập.
  - Nếu đã xác thực: Hệ thống tiếp tục kiểm tra quyền hạn cụ thể của người dùng trước khi hiển thị giao diện bảng danh sách vai trò.

Các luồng nghiệp vụ chính Tại giao diện quản lý, quản trị viên có thể lựa chọn một trong hai hành động, dẫn đến hai nhánh xử lý khác nhau:

- Nhánh 1: Tạo vai trò mới (Create Role):

1. Quản trị viên vào màn hình cấu hình vai trò.
2. Hệ thống cho phép thực hiện đồng thời hai tác vụ: lựa chọn các quyền hạn và đặt tên cho vai trò.
3. Sau khi hoàn tất cả hai thông tin trên, hệ thống tổng hợp và tạo vai trò mới trong cơ sở dữ liệu.

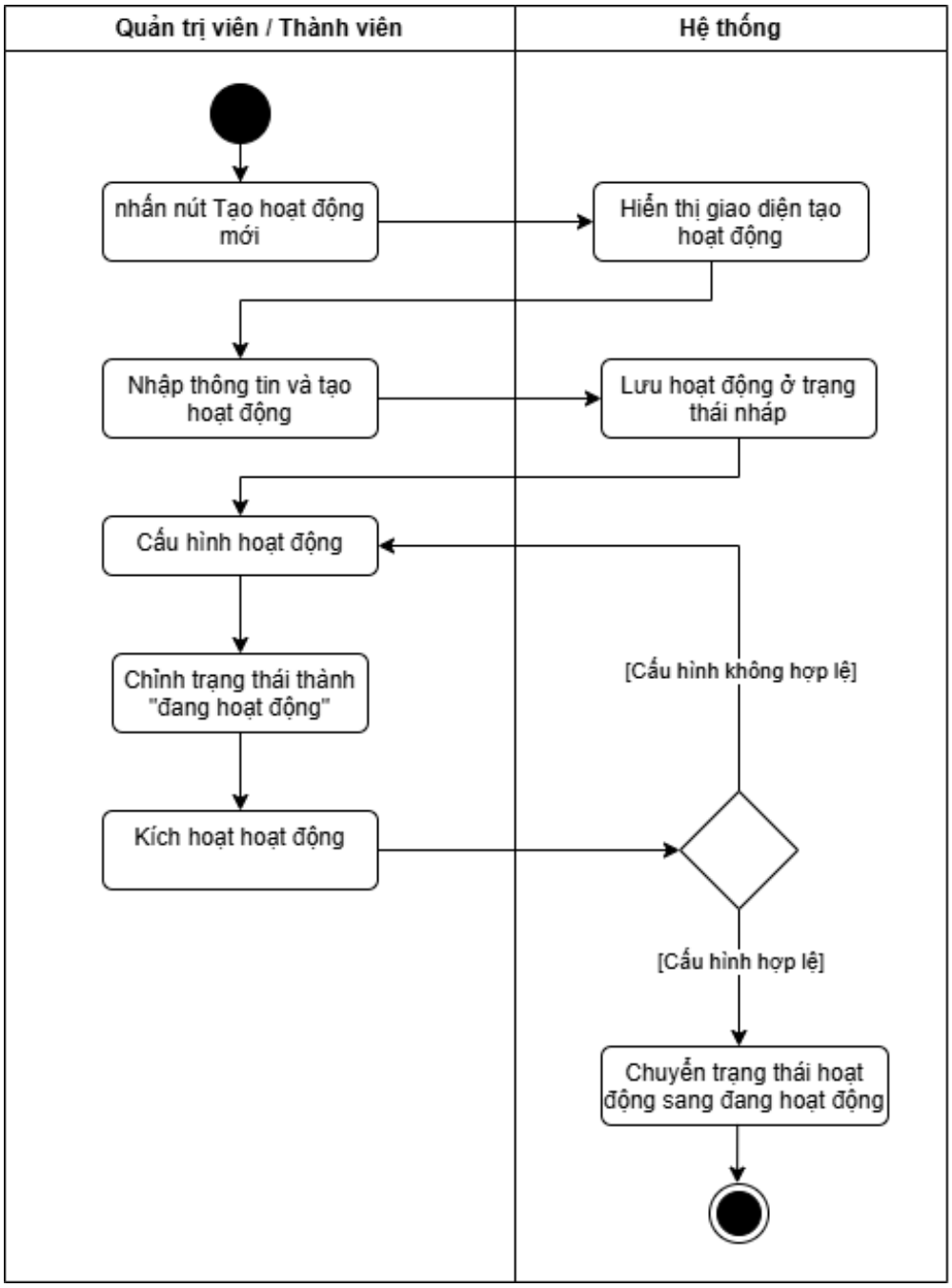
- Nhánh 2: Gán vai trò cho người dùng (Assign Role):

1. Hệ thống chuyển hướng và hiển thị danh sách người dùng.
2. Quản trị viên chọn một tài khoản người dùng cụ thể.
3. Hệ thống yêu cầu xác định song song hai yếu tố: vai trò cần gán và tổ chức áp dụng (scope).
4. Hệ thống cập nhật thông tin phân quyền vào cơ sở dữ liệu và kết thúc quy trình.

3.5.3 Activity Diagram - Module Công tác Xã hội

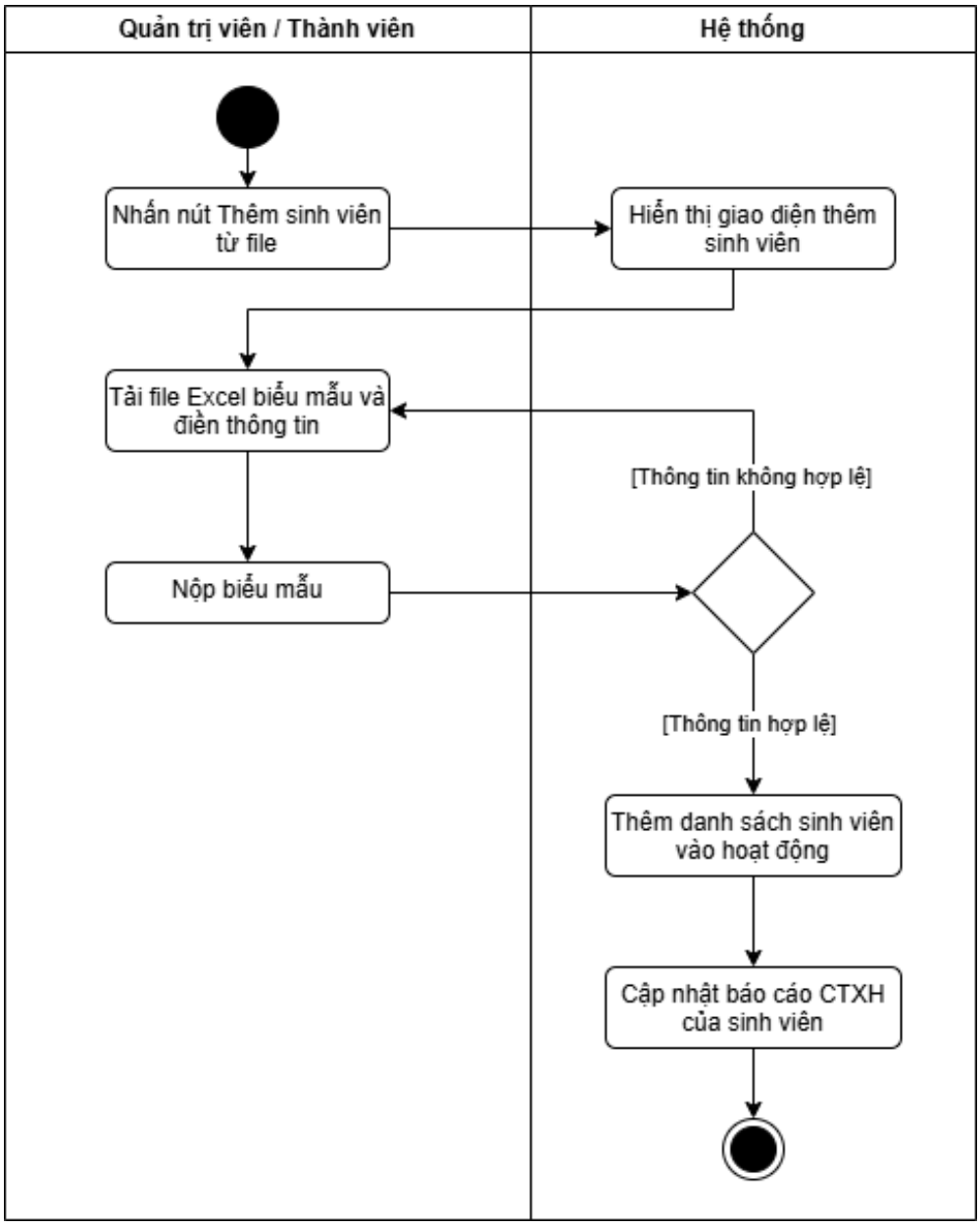
Để minh họa chi tiết luồng xử lý của các chức năng chính trong module Công tác Xã hội, báo cáo trình bày ba activity diagram đại diện cho các pattern quan trọng trong hệ thống:

- Quản lý trạng thái: tạo và chỉnh sửa hoạt động với logic chuyển đổi giữa trạng thái nháp và đang hoạt động.
- Xử lý hàng loạt: thêm danh sách sinh viên từ file Excel với các bước tải file, kiểm tra hợp lệ và cập nhật danh sách tham gia hoạt động.
- Cấu hình cascade: tạo cấu hình giới hạn tự động áp dụng cho tổ chức con theo cấp bậc.



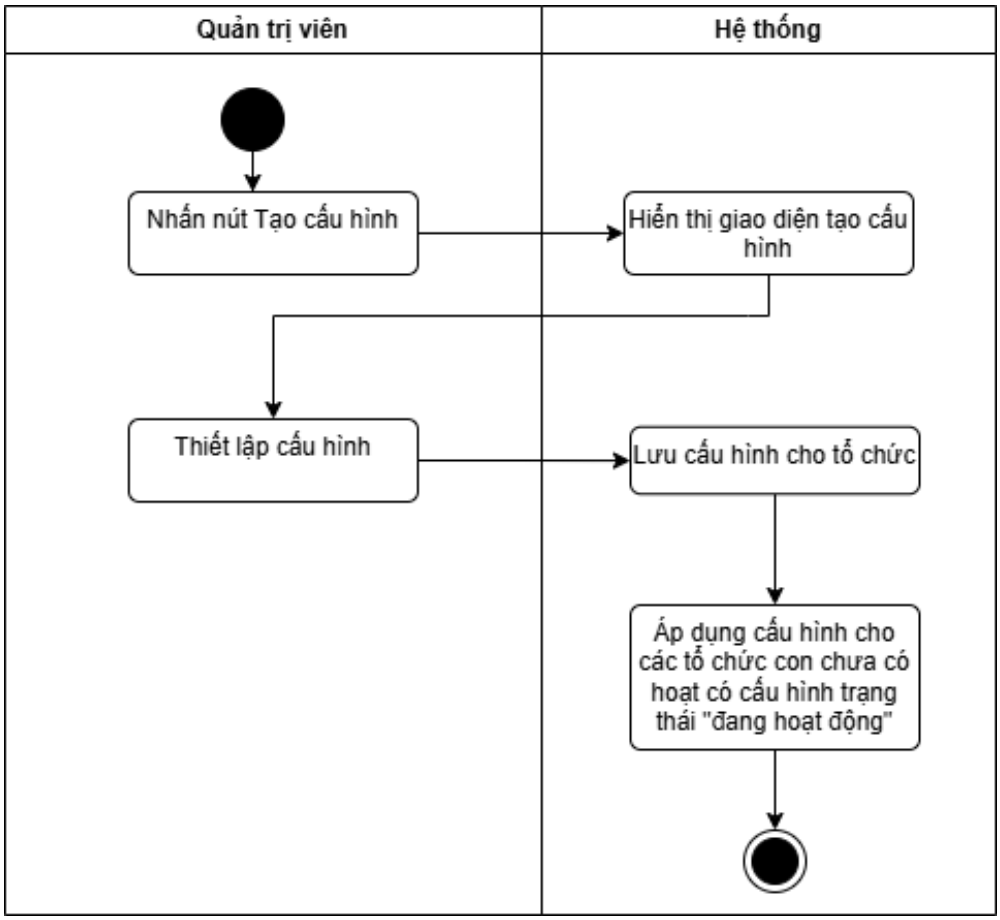
Hình 31: Activity Diagram - Tạo và chỉnh sửa hoạt động

Activity diagram trên minh họa luồng tạo và chỉnh sửa hoạt động CTXH với logic quản lý trạng thái. Điểm quan trọng là khi chọn trạng thái đang hoạt động, hệ thống yêu cầu validate đầy đủ thông tin như địa điểm, thời gian và số ngày; trong khi trạng thái nháp cho phép lưu chưa đầy đủ để tiếp tục sau. Hệ thống kiểm tra cấu hình giới hạn số ngày theo tổ chức nhưng chỉ hiển thị cảnh báo, vẫn cho phép tạo hoạt động vượt giới hạn để linh hoạt với các trường hợp đặc biệt.



Hình 32: Activity Diagram - Thêm danh sách sinh viên từ file Excel

Activity diagram này thể hiện luồng thêm danh sách sinh viên tham gia hoạt động từ file Excel. Quản trị viên/thành viên tải file theo biểu mẫu, hệ thống kiểm tra tính hợp lệ dữ liệu và phản hồi kết quả kiểm tra để người dùng chỉnh sửa nếu cần. Khi dữ liệu hợp lệ, hệ thống thêm danh sách sinh viên vào hoạt động và cập nhật báo cáo CTXH tương ứng. Luồng này nhấn mạnh bước kiểm tra dữ liệu trước khi ghi nhận chính thức để bảo đảm tính nhất quán.



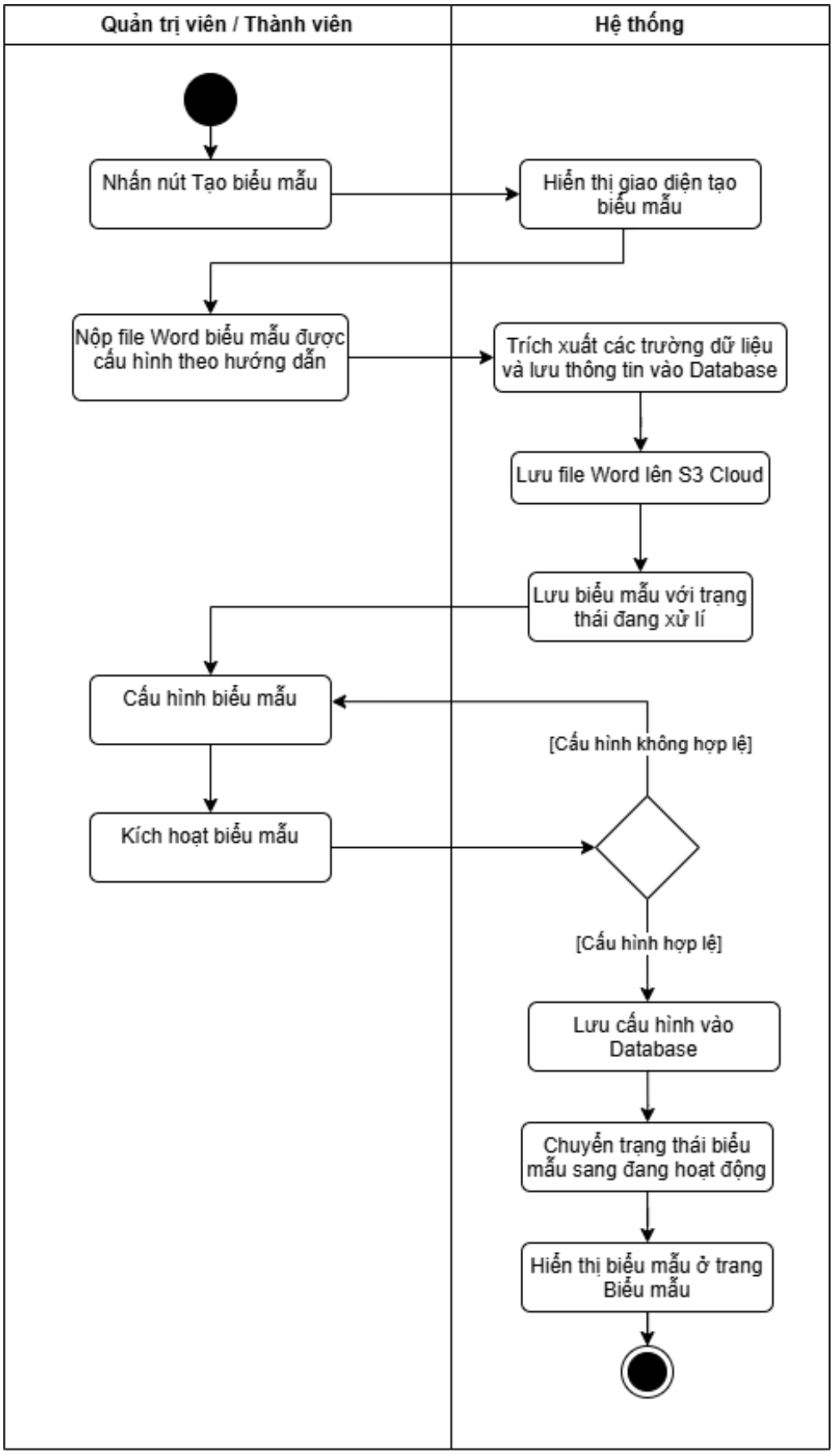
Hình 33: Activity Diagram - Tạo cấu hình giới hạn ngày CTXH

Activity diagram này minh họa luồng tạo cấu hình giới hạn số ngày CTXH với cơ chế cascade tự động. Sau khi tạo cấu hình cho tổ chức cha, hệ thống tự động duyệt qua các tổ chức con theo cấu trúc phân cấp. Với mỗi tổ chức con, hệ thống kiểm tra: nếu chưa có cấu hình thì tạo mới, nếu đã có cấu hình đang hoạt động thì cập nhật số ngày tối đa. Logic này giúp cấu hình được áp dụng nhất quán cho toàn bộ cây tổ chức mà không cần thiết lập thủ công từng tổ chức con.

3.5.4 Activity Diagram - Module Biểu mẫu

Để minh họa chi tiết luồng xử lý của các chức năng chính trong module Biểu mẫu, báo cáo trình bày hai activity diagram đại diện cho các pattern quan trọng trong hệ thống:

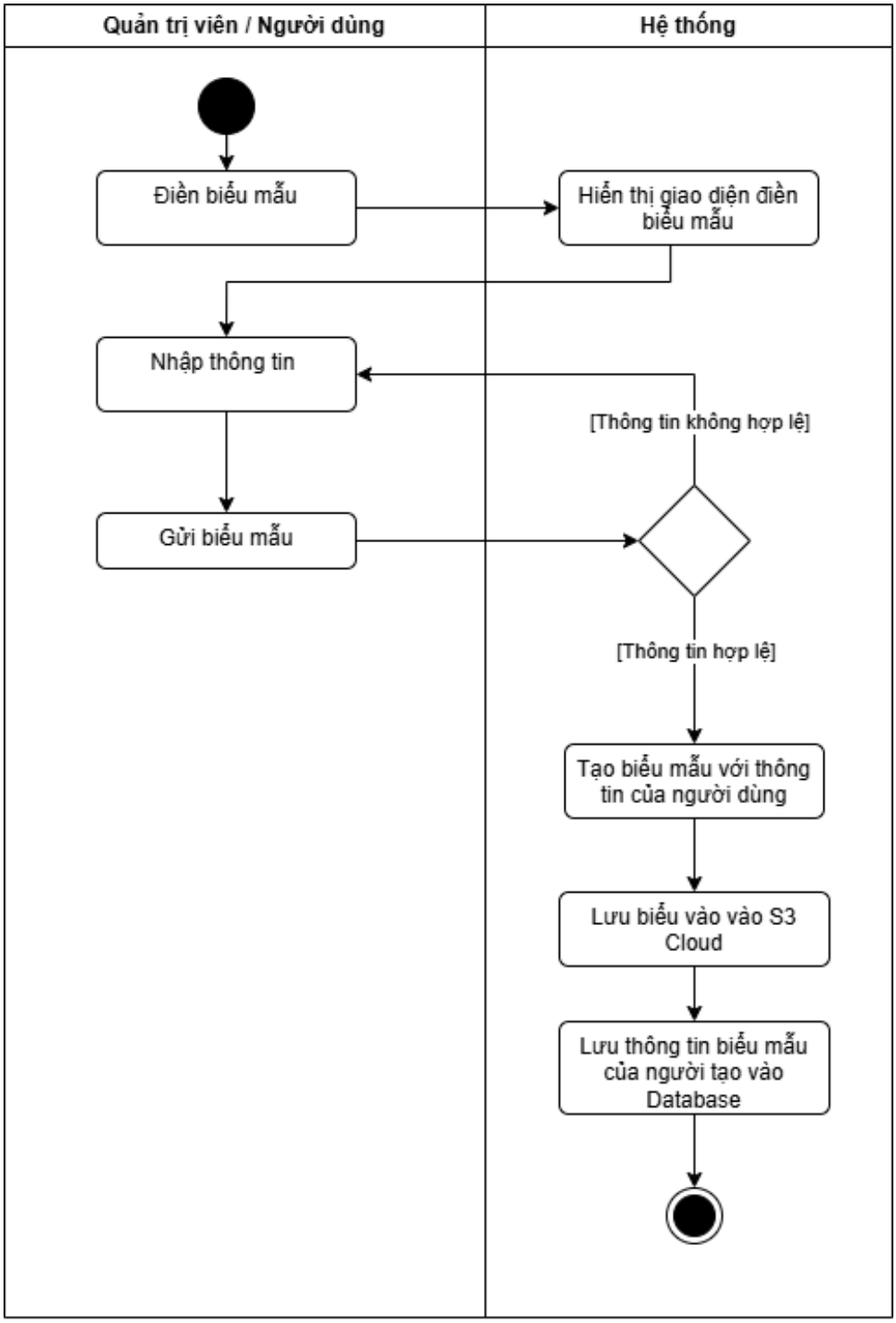
- Template parsing and configuration: tải lên và phân tích file Word template, trích xuất placeholder động, cấu hình trường dữ liệu.
- Dynamic form generation: sinh form động từ cấu hình, điền dữ liệu, thay thế placeholder và tạo file tự động.



Hình 34: Activity Diagram - Tạo và cấu hình biểu mẫu

Activity diagram trên minh họa luồng tải lên và cấu hình biểu mẫu Word động. Hệ thống nhận file `.docx`, parse nội dung và trích xuất các placeholder theo cú pháp `{tên_trường}`. Nếu không tìm thấy placeholder nào, hệ thống

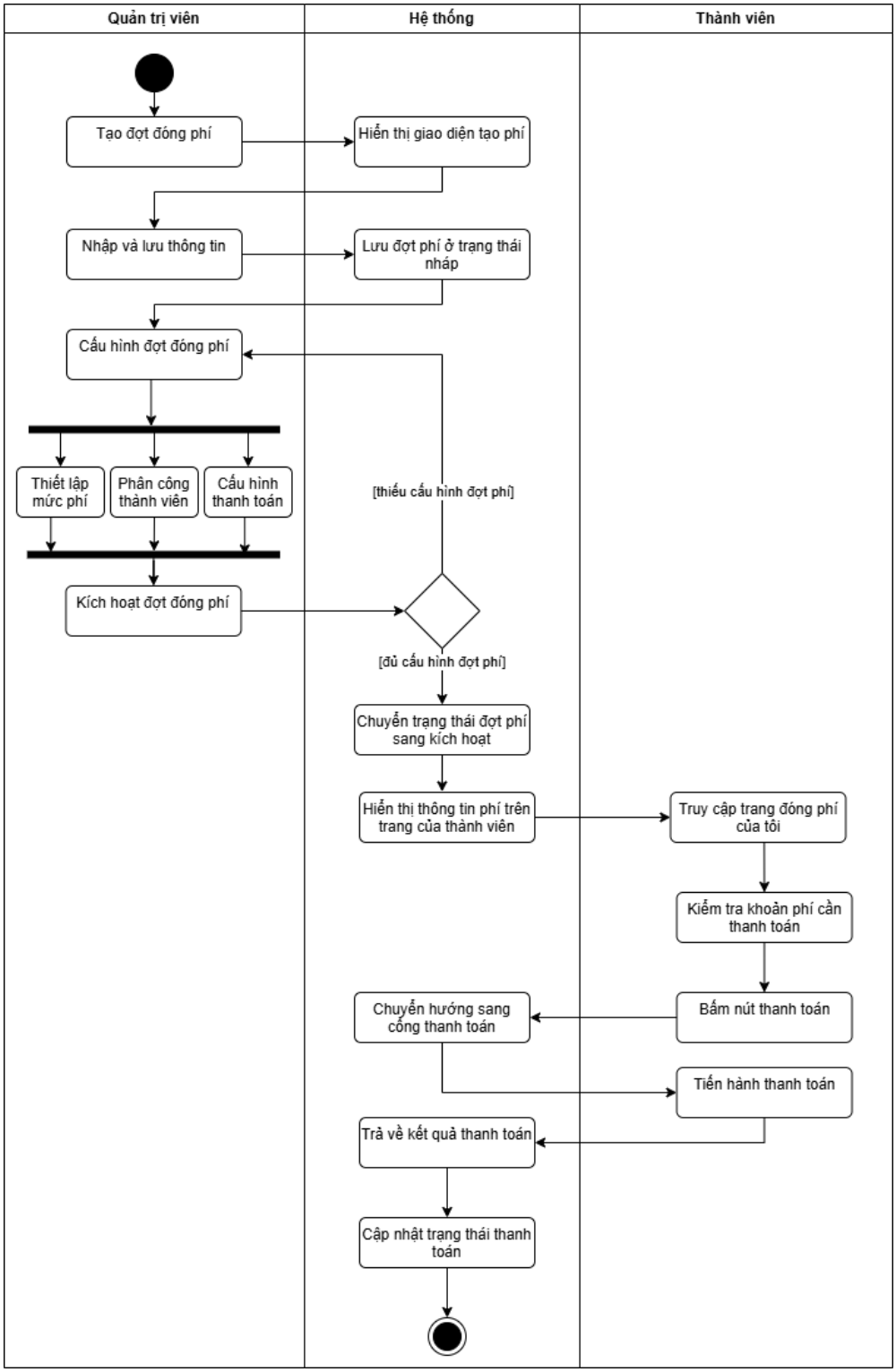
vẫn cho phép lưu template nhưng hiển thị cảnh báo. Sau khi trích xuất thành công, người dùng cấu hình cho từng placeholder gồm loại dữ liệu, nhãn hiển thị và trường bắt buộc hoặc tùy chọn. File template được lưu trên S3 và cấu hình trường được lưu vào cơ sở dữ liệu để sử dụng khi sinh form.



Hình 35: Activity Diagram - Điền thông tin vào biểu mẫu

Activity diagram này thể hiện luồng điền biểu mẫu động và sinh file tự động. Hệ thống lấy cấu hình trường từ cơ sở dữ liệu để sinh form với các input phù hợp. Sau khi người dùng điền thông tin, hệ thống validate theo cấu hình. Nếu hợp lệ, hệ thống tải template từ S3, thay thế placeholder bằng dữ liệu người dùng nhập và sinh file đầu ra. File được lưu lại để người dùng có thể tải xuống hoặc xem trước trong trình duyệt. Toàn bộ quá trình được lưu thành bản ghi nộp trong cơ sở dữ liệu để tra cứu sau này.

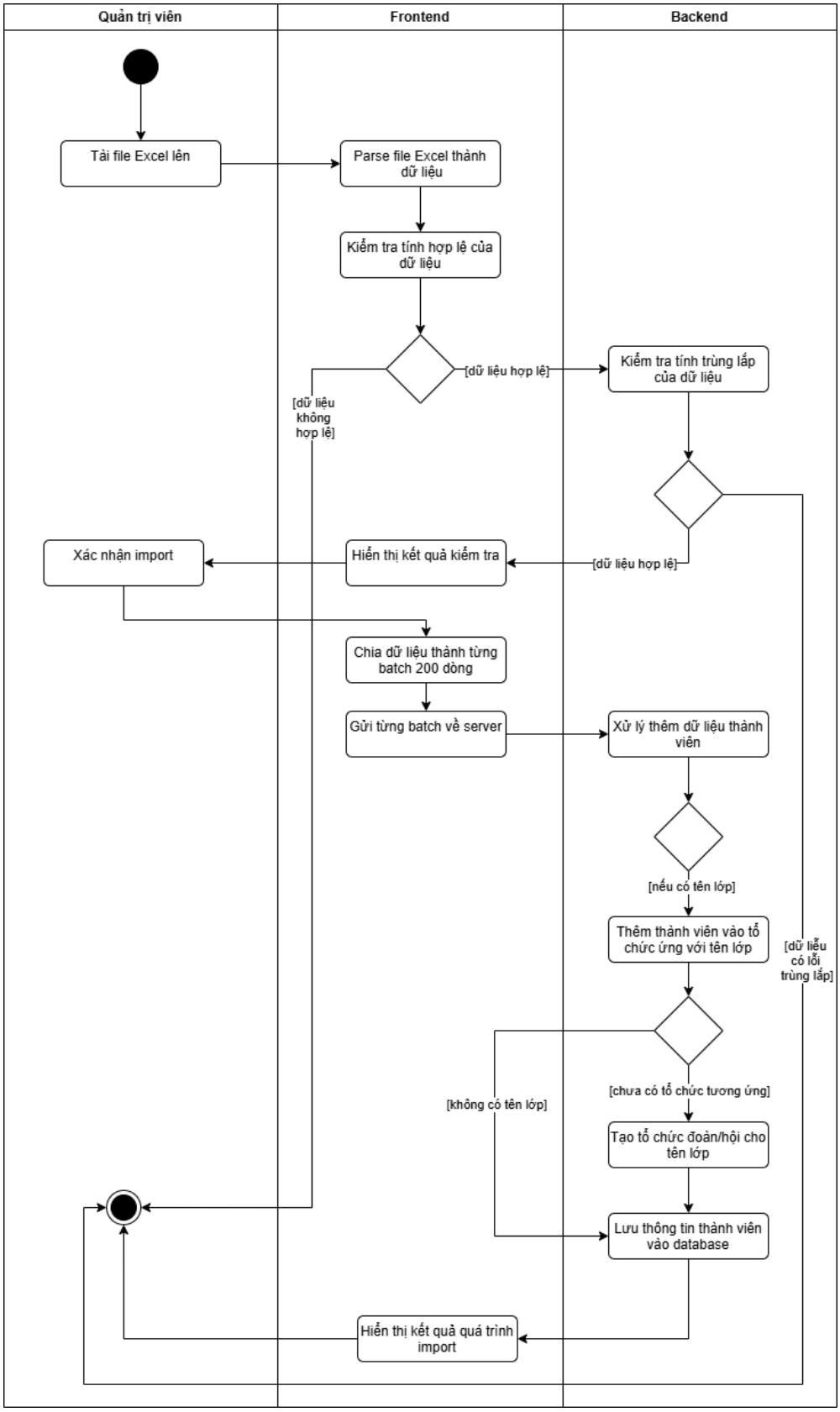
3.5.5 Activity Diagram - Chức năng tạo đợt đóng phí



Hình 36: Activity Diagram - Quản lý đóng phí

Đầu tiên quản trị viên tạo đợt đóng phí theo tổ chức; sau khi bấm lưu, đợt đóng phí sẽ được tạo mới ở trạng thái nháp. Để kích hoạt, quản trị viên cần thiết lập mức phí và phân công thành viên. Sau khi đã thiết lập đầy đủ, hệ thống sẽ cho phép kích hoạt. Khi đợt đóng phí ở trạng thái kích hoạt thì đợt đóng phí được công bố cho toàn bộ thành viên được phân công để xem và thực hiện thanh toán.

3.5.6 Activity Diagram - Chức năng thêm dữ liệu từ file Excel



Hình 37: Activity Diagram - Thêm nhiều thành viên từ file Excel

Chức năng thêm dữ liệu từ file Excel trong module người dùng được xây dựng nhằm hỗ trợ nhập nhanh số lượng lớn dữ liệu sinh viên vào hệ thống, đặc biệt trong giai đoạn đầu mỗi năm học khi cần cập nhật thông tin cho nhiều khóa và lớp khác nhau. Trong phạm vi chức năng này, báo cáo tập trung mô tả quy trình import dữ liệu tại module người dùng do đây là module có tần suất sử dụng cao và đóng vai trò trung tâm trong việc quản lý thành viên.

Khi quản trị viên tải lên file Excel, hệ thống tiến hành đọc, kiểm tra dữ liệu đầu vào và xử lý từng dòng tương ứng với một người dùng. Với mỗi bản ghi, hệ thống tự động xác định và tạo cấu trúc tổ chức phù hợp dựa trên thông tin lớp học và khoa đào tạo. Cụ thể, hệ thống kiểm tra sự tồn tại của các tổ chức cấp khoa và cấp lớp; trong trường hợp chưa tồn tại, hệ thống sẽ tự động khởi tạo mới.

Sau khi hoàn tất việc xác định và tạo tổ chức, người dùng được tự động gán vào các tổ chức liên quan với vai trò mặc định phù hợp. Cách xử lý này giúp giảm đáng kể thao tác thủ công trong quá trình khởi tạo dữ liệu, đồng thời bảo đảm tính nhất quán của mô hình tổ chức trong toàn hệ thống.

3.6 Sơ đồ Sequence Diagram

Sơ đồ sequence được sử dụng để làm rõ thứ tự trao đổi thông điệp giữa các thành phần trong hệ thống và cơ chế phối hợp giữa frontend, backend, service layer, cơ sở dữ liệu cùng các hệ thống ngoài.

3.6.1 Sequence Diagram - Module Quản lý tổ chức

Quy trình hiển thị và tương tác với cấu trúc tổ chức (Organization Tree) là một trong những nghiệp vụ quan trọng, yêu cầu tốc độ phản hồi nhanh và trải nghiệm người dùng mượt mà. Biểu đồ tuần tự tại hình dưới mô tả chi tiết luồng tương tác giữa Thành viên, Hệ thống Frontend và Backend Controller. Quy trình xử lý được chia thành hai giai đoạn chính nhằm tối ưu hóa hiệu năng hệ thống:

Giai đoạn 1: Khởi tạo và tải dữ liệu song song Khi người dùng truy cập vào trang quản lý, hệ thống không thực hiện các lệnh gọi API tuần tự mà sử dụng kỹ thuật parallel processing để giảm thiểu tổng thời gian phản hồi.

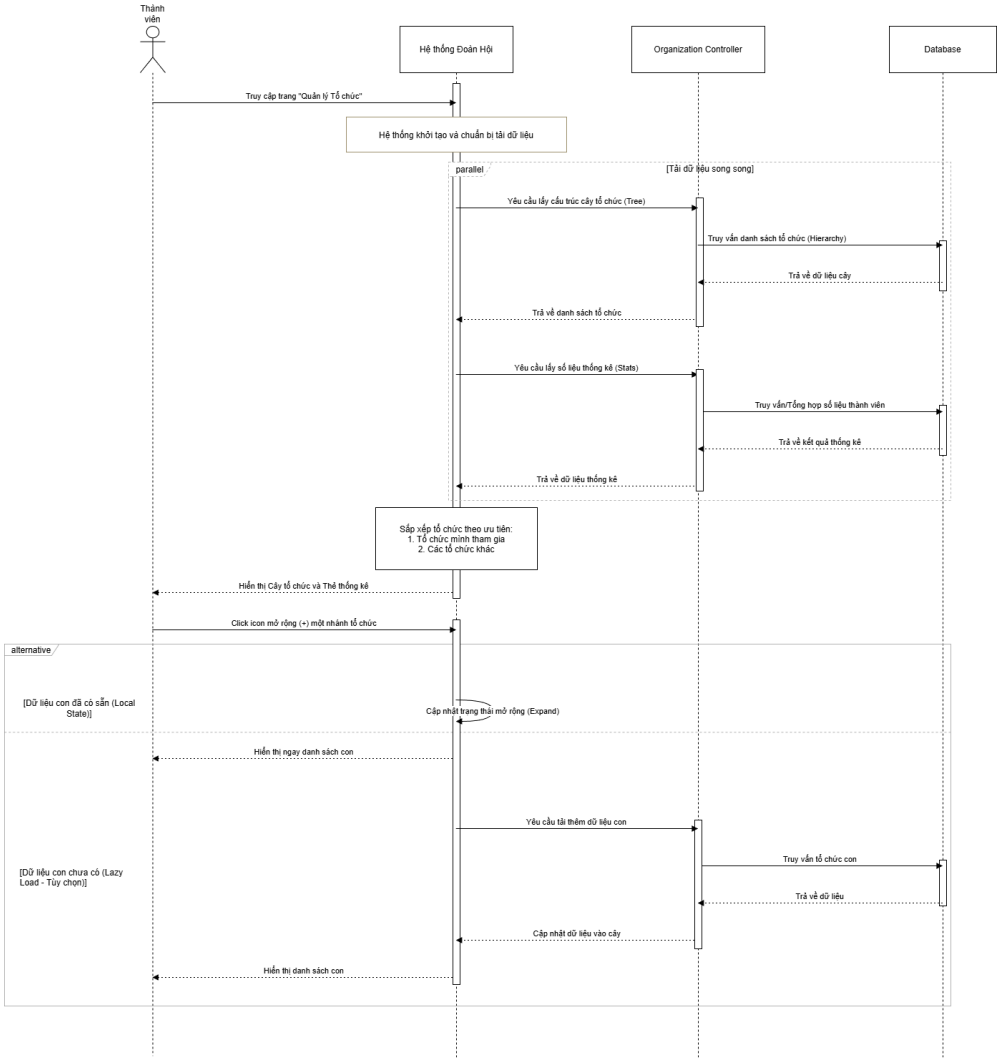
- Truy vấn cấu trúc cây (Tree Structure): Hệ thống gửi yêu cầu đến Organization Controller để lấy danh sách tổ chức phân cấp.
- Truy vấn thống kê (Statistics): Đồng thời, một luồng xử lý khác truy vấn số liệu tổng hợp về thành viên từ cơ sở dữ liệu.

Sau khi nhận đủ dữ liệu từ cả hai luồng, Frontend thực hiện thuật toán sắp xếp cục bộ (client-side sorting). Các tổ chức mà người dùng đang là thành viên sẽ được ưu tiên hiển thị lên đầu danh sách (Priority 1), giúp người dùng tiếp cận thông tin quan trọng nhanh nhất.

Giai đoạn 2: Tương tác và cơ chế Lazy Loading Để giảm tải băng thông và dung lượng bộ nhớ ban đầu, hệ thống áp dụng cơ chế lazy loading cho các thao tác mở rộng nhánh cây:

1. Khi người dùng kích hoạt mở rộng một nhánh tổ chức, hệ thống kiểm tra trạng thái dữ liệu cục bộ (local state).
2. Trường hợp cache hit: Nếu dữ liệu nhánh con đã tồn tại, giao diện được cập nhật tức thì mà không cần gọi về server.
3. Trường hợp cache miss: Nếu dữ liệu chưa có, hệ thống mới gửi yêu cầu tải bổ sung (fetch additional data) tới controller. Dữ liệu mới sau đó được cập nhật vào cây và hiển thị cho người dùng.

Giải pháp này giúp hệ thống đảm bảo khả năng mở rộng tốt ngay cả khi số lượng tổ chức tăng lên quy mô rất lớn, đồng thời duy trì trải nghiệm mượt mà cho người dùng cuối.



Hình 38: Sequence Diagram - Luồng hiển thị và tương tác Organization Tree

3.6.2 Sequence Diagram - Module Phân quyền

Đảm bảo an toàn bảo mật và phân quyền chính xác là yêu cầu cốt lõi của hệ thống. Biểu đồ tuần tự tại hình dưới mô tả chi tiết quy trình đăng nhập và cơ chế tự động tải quyền hạn (permissions) kết hợp với chiến lược caching để tối ưu hiệu năng.

Giai đoạn 1: Xác thực người dùng (Authentication) Khi thành viên nhập thông tin đăng nhập, hệ thống thực hiện các bước kiểm tra bảo mật nghiêm ngặt:

- Kiểm tra thông tin: Auth Controller truy vấn thông tin từ cơ sở dữ liệu và thực hiện so khớp mật khẩu đã được mã hóa.
- Cấp phát phiên làm việc: Nếu thông tin chính xác, hệ thống khởi tạo phiên làm việc bằng cách cấp phát cặp Access Token và Refresh Token. Thông tin phiên này cũng được lưu vết để quản lý bảo mật.

Giai đoạn 2: Tải quyền hạn và tối ưu hóa với cache Sau khi đăng nhập thành công, hệ thống tự động thực hiện quy trình phân quyền (authorization) để xác định phạm vi truy cập của người dùng:

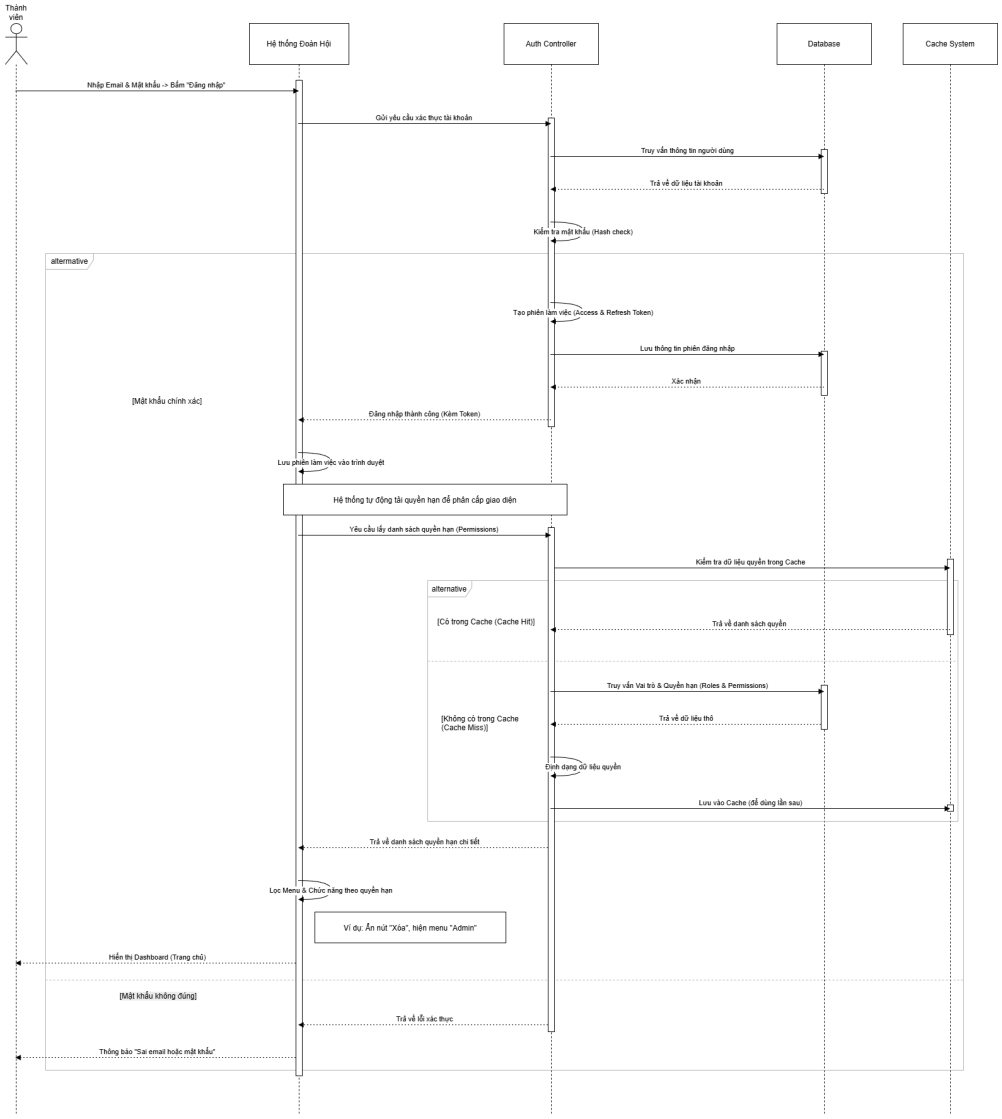
1. Hệ thống kiểm tra danh sách quyền hạn của người dùng trong cache system.
2. Trường hợp cache hit: Nếu dữ liệu quyền đã có sẵn trong bộ nhớ đệm, hệ thống trả về ngay lập tức, giúp giảm độ trễ phản hồi.

3. Trường hợp cache miss: Nếu chưa có trong cache, hệ thống thực hiện truy vấn từ cơ sở dữ liệu, sau đó định dạng dữ liệu và lưu ngược vào cache cho các lần truy cập sau.

Giai đoạn 3: Phân cấp giao diện (UI Adaptation) Dựa trên danh sách quyền hạn nhận được, frontend thực hiện lọc và hiển thị giao diện động:

- Các chức năng không thuộc quyền hạn của người dùng sẽ bị ẩn.
- Người dùng được chuyển hướng đến trang chủ với giao diện được cá nhân hóa theo đúng vai trò.

Quy trình này đảm bảo tính toàn vẹn dữ liệu và ngăn chặn các truy cập trái phép vào các chức năng nhạy cảm của hệ thống.



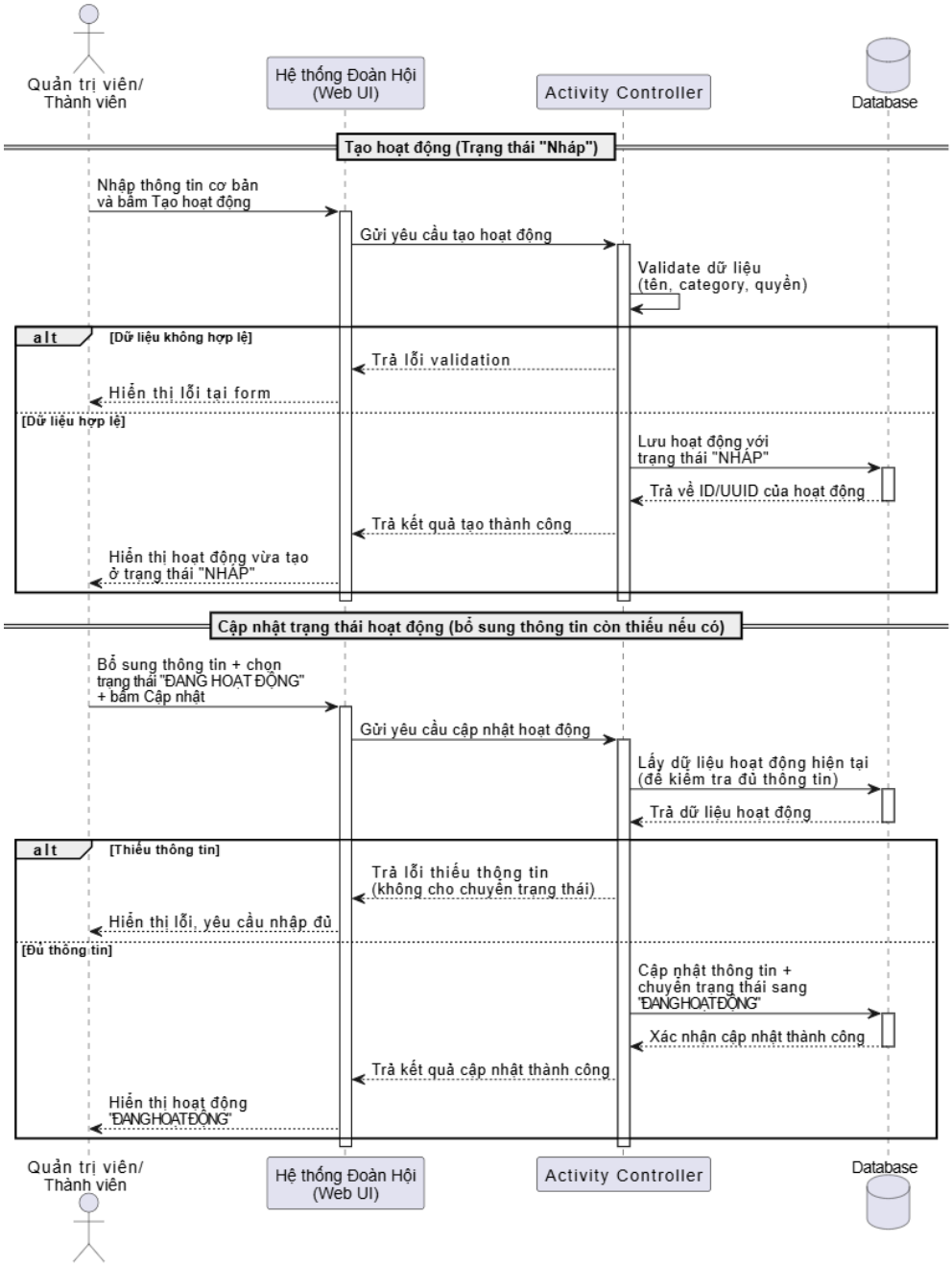
Hình 39: Sequence Diagram - Luồng hoạt động cơ chế phân quyền

3.6.3 Sequence Diagram - Module Công tác Xã hội

Để minh họa chi tiết tương tác giữa các thành phần trong hệ thống, báo cáo trình bày hai sequence diagram đại diện cho các luồng xử lý quan trọng trong module Công tác Xã hội:

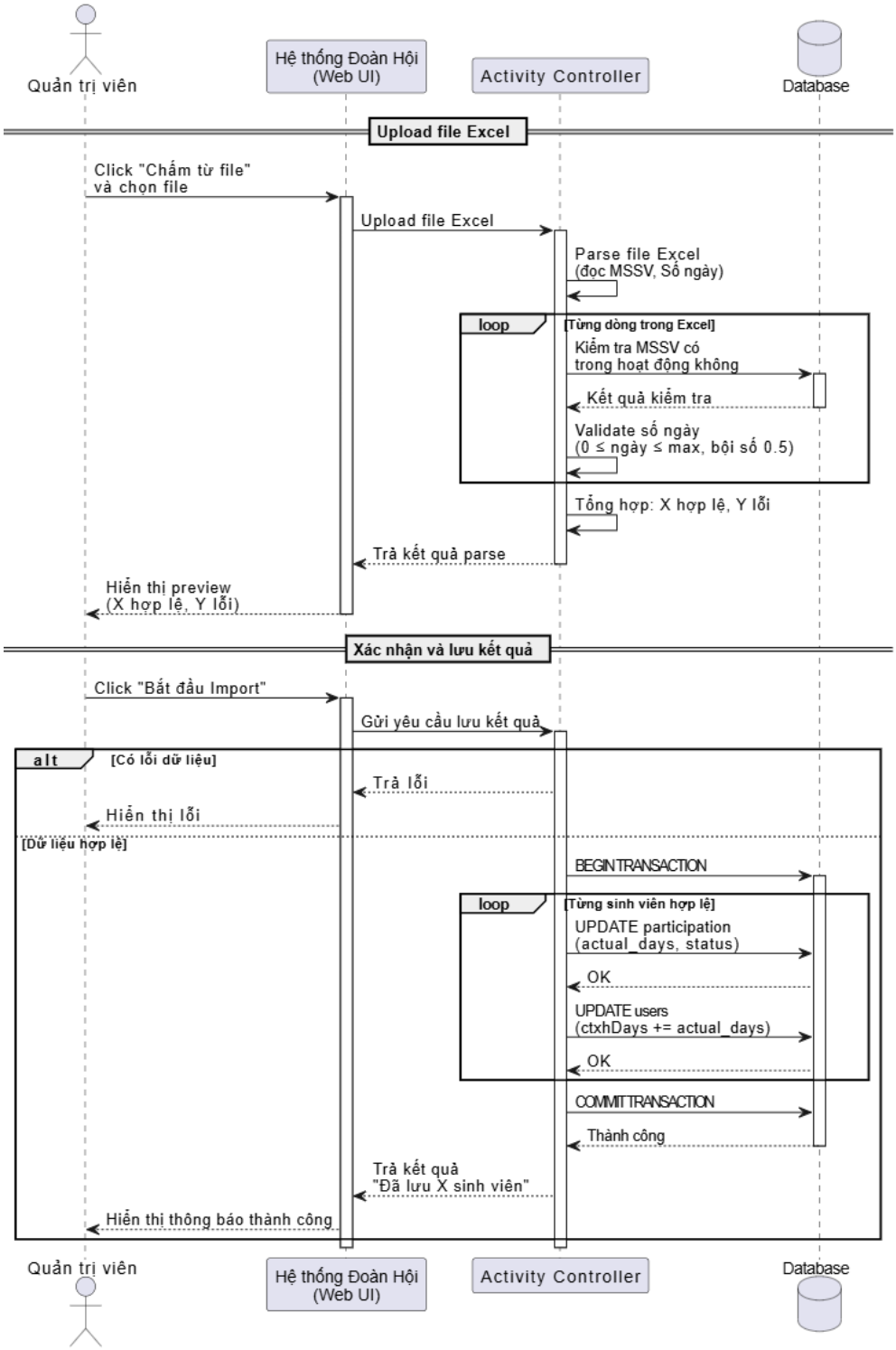
- Tạo và chỉnh sửa hoạt động: tương tác giữa người dùng, controller, service layer và database khi tạo hoặc cập nhật hoạt động với logic quản lý trạng thái.

- Chấm điểm từ Excel: tương tác phức tạp giữa nhiều component trong quá trình xử lý file Excel hàng loạt, bao gồm file parsing, validation và batch update.



Hình 40: Sequence Diagram - Tạo và chỉnh sửa hoạt động CTXH

Sequence diagram trên mô tả tương tác giữa các thành phần khi tạo hoặc chỉnh sửa hoạt động CTXH. Actor gửi request qua Controller, Controller gọi Service Layer để xử lý business logic. Service kiểm tra quyền, validate dữ liệu đầu vào (tên không trống, thời gian hợp lệ, số ngày là bội của 0.5), và kiểm tra cấu hình giới hạn từ ConfigService. Nếu trạng thái là đang hoạt động, Service validate đầy đủ thông tin bắt buộc. Sau khi validation thành công, Service gọi Repository để lưu vào cơ sở dữ liệu và trả kết quả cho người dùng. Nếu có lỗi validation, hệ thống trả về thông báo lỗi cụ thể để người dùng chỉnh sửa.



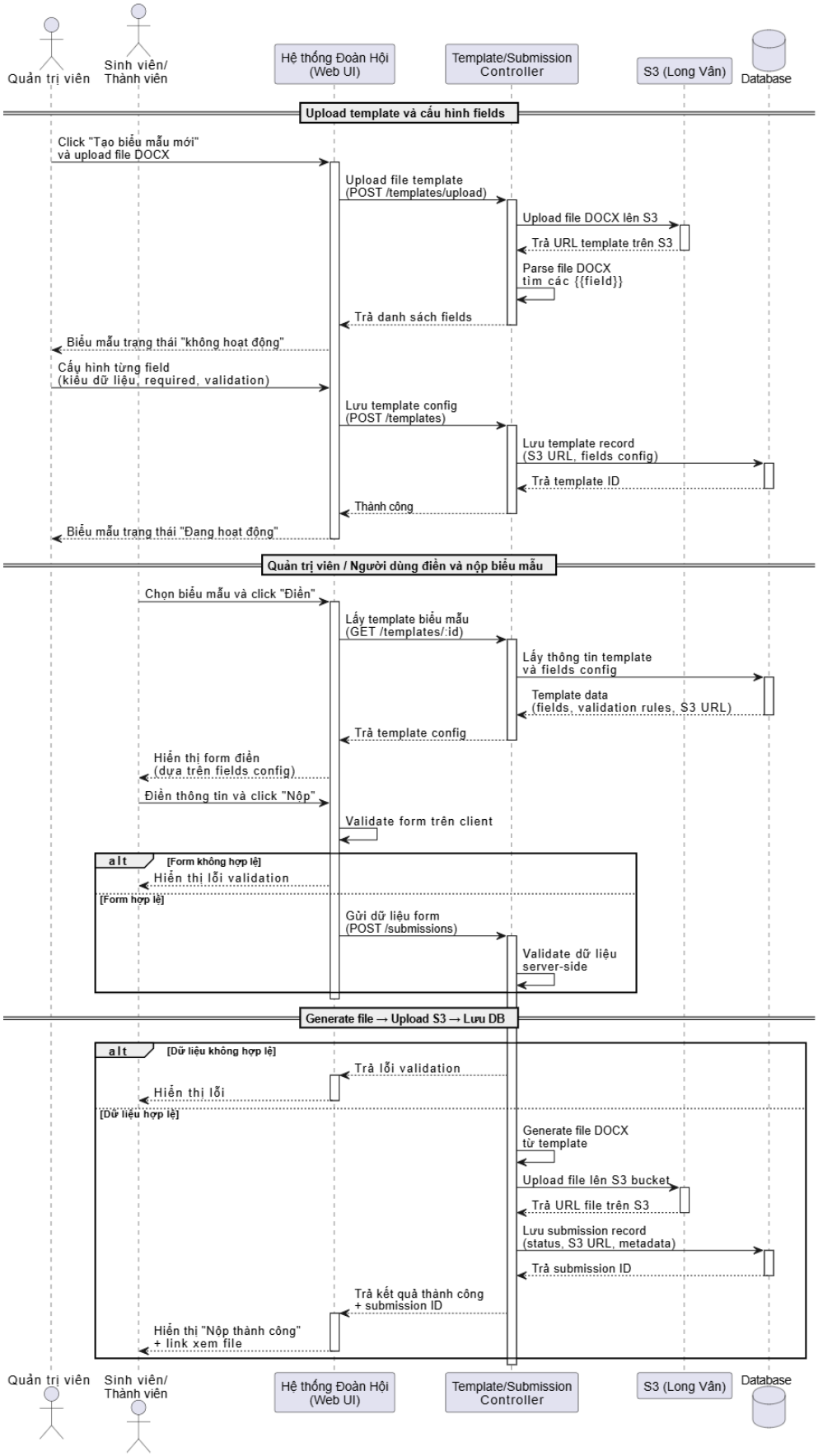
Hình 41: Sequence Diagram - Chấm điểm sinh viên từ file Excel

Sequence diagram này thể hiện luồng xử lý phức tạp khi chấm điểm hàng loạt từ Excel. Controller nhận file upload, gọi parser service để parse file thành các chunk. Với mỗi chunk, validation service kiểm tra từng dòng: sinh viên tồn tại trong cơ sở dữ liệu, số ngày hợp lệ (bội 0.5, không âm) và không vượt giới hạn hoạt động. Kết quả validation được trả về dạng preview gồm danh sách dòng hợp lệ và dòng lỗi kèm lý do. Khi người dùng xác nhận, service bắt đầu batch update trong transaction: với mỗi dòng hợp lệ, cập nhật điểm tham gia và cộng vào tổng CTXH của sinh viên. Nếu có lỗi trong quá trình update, toàn bộ transaction được rollback để đảm bảo tính nhất quán dữ liệu.

3.6.4 Sequence Diagram - Module Biểu mẫu

Để minh họa chi tiết tương tác giữa các thành phần trong hệ thống, báo cáo trình bày sequence diagram đại diện cho luồng xử lý end-to-end trong module Biểu mẫu:

- Tạo, cấu hình, điền và nộp biểu mẫu: tương tác giữa người dùng, controller, parser service, S3 storage và database từ lúc tải lên template đến khi sinh file và lưu bản nộp.

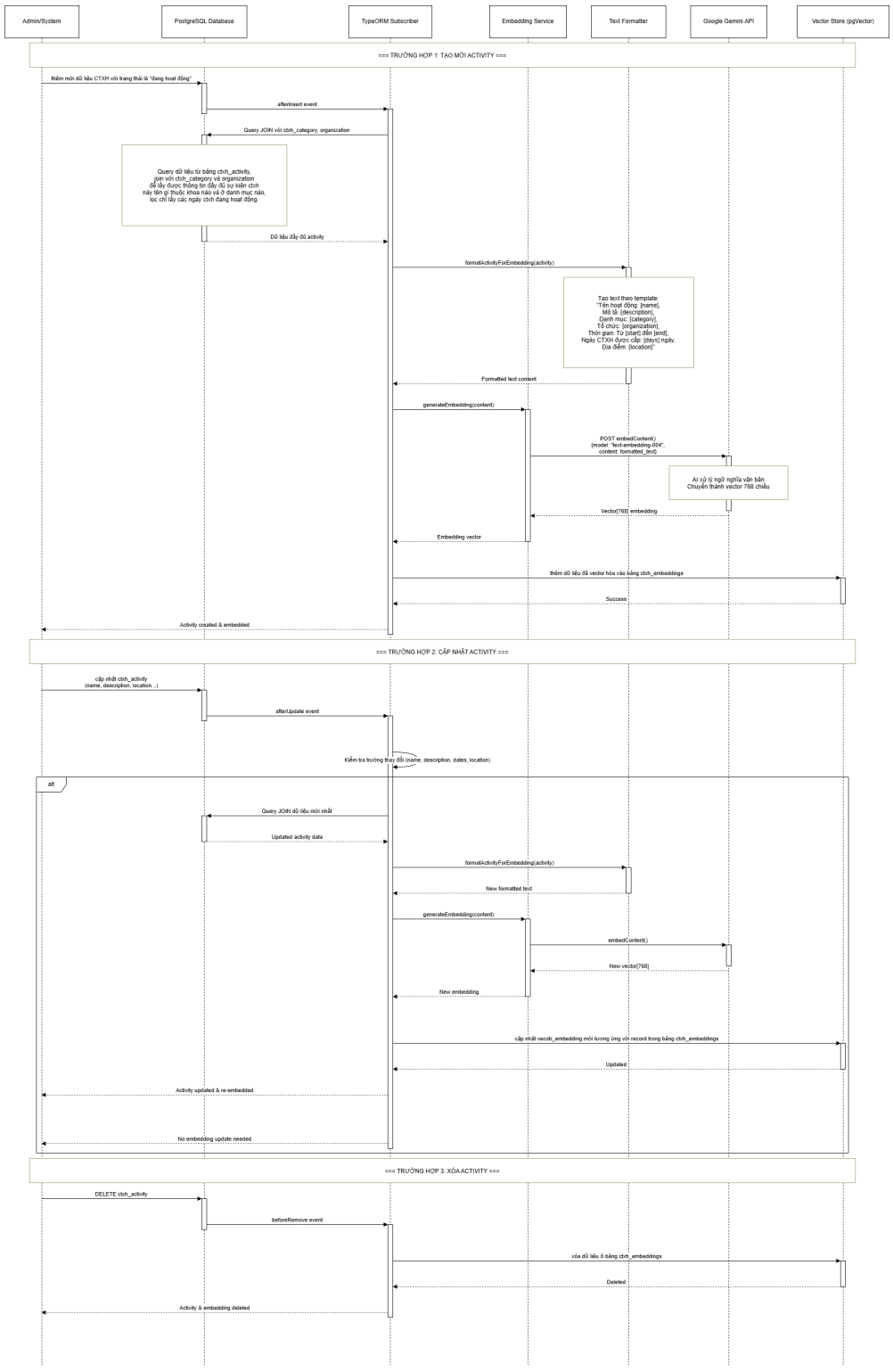


Hình 42: Sequence Diagram - Tạo, cấu hình, điền và nộp biểu mẫu

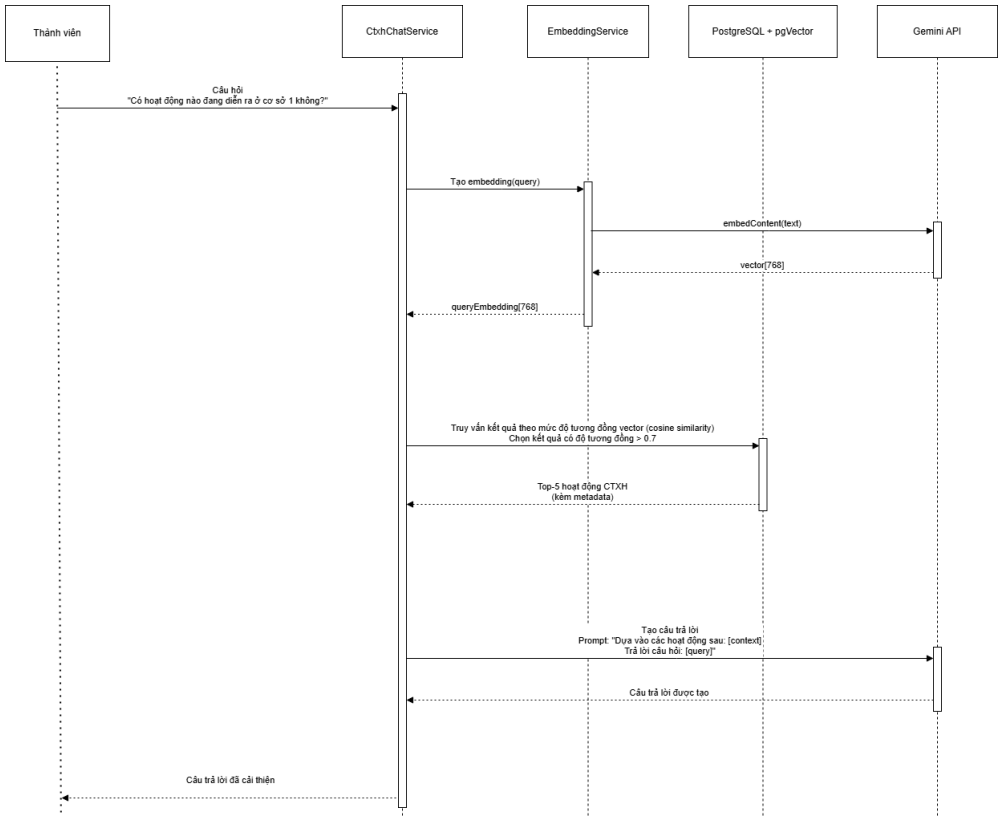
Sequence diagram trên mô tả tương tác end-to-end của module biểu mẫu. Ở pha đầu, người dùng upload file

Word (`.docx`), hệ thống parse template để trích xuất placeholder và lưu cấu hình fields cùng metadata. Ở pha tiếp theo, người dùng chọn biểu mẫu, điền dữ liệu và nộp; controller thực hiện validate, sinh file từ template, upload lên S3, sau đó lưu bản ghi submission vào cơ sở dữ liệu. Luồng này phản ánh đầy đủ cả bước cấu hình template lẫn bước sử dụng template để tạo biểu mẫu đã nộp.

3.6.5 Sequence Diagram - Chatbot hỏi-đáp hoạt động CTXH



Hình 43: Sequence Diagram - Pipeline tự động chuyển đổi dữ liệu CTXH sang dạng vector embedding



Hình 44: Sequence Diagram - Luồng hoạt động của Chatbot RAG

Mô tả trong phần này sẽ được làm rõ ở phần hiện thực hệ thống khi trình bày pipeline đồng bộ dữ liệu và cơ chế truy vấn theo ngữ cảnh.

3.7 Một số bài toán và giải pháp

3.7.1 Bài toán quản lý và chấm điểm CTXH theo hoạt động

a. Khái niệm

Trong mô hình cũ, dữ liệu hoạt động và dữ liệu chấm ngày thường nằm rời rạc, nên khó kiểm soát tiến độ thật của từng sinh viên. Vì vậy, bài toán đặt ra là phải quản lý trọn vòng đời hoạt động CTXH, từ lúc mở đăng ký đến lúc chấm điểm và tổng hợp kết quả.

b. Các phương pháp hiện có

Nhóm khảo sát thấy có hai hướng xử lý phổ biến:

Thứ nhất là quản lý thủ công bằng bảng tính, cập nhật từng dòng theo từng đợt.

Thứ hai là quản lý trên hệ thống có trạng thái nghiệp vụ rõ ràng và chấm theo danh sách participant.

Hướng thứ hai phù hợp hơn vì giúp hạn chế chấm trùng, kiểm soát trạng thái hoạt động và dễ tổng hợp báo cáo.

c. Hiện thực trong hệ thống

Trong hệ thống hiện tại, module CTXH đã bao gồm quản lý danh mục, quản lý hoạt động, quản lý người tham gia, chấm trực tiếp và chấm bằng Excel. Mỗi hoạt động đi qua các trạng thái `DRAFT`, `ACTIVE`, `COMPLETED`, `CANCELLED`; quyền thao tác và bước chấm được ràng buộc theo trạng thái này. Kết quả chấm hợp lệ sẽ được cộng dồn vào hồ sơ sinh viên để dùng cho báo cáo cá nhân và báo cáo tổng hợp.

3.7.2 Bài toán thu phí trực tuyến và đồng bộ trạng thái thanh toán

a. Khái niệm

Bài toán thu phí không chỉ dừng ở việc tạo danh sách phải đóng, mà còn phải theo dõi trạng thái giao dịch đủ tin cậy để đối soát. Nếu cập nhật thủ công, khả năng lệch dữ liệu giữa thực tế và hệ thống là rất cao.

b. Các phương pháp hiện có

Hai cách làm thường gặp là:

Một là ghi nhận thủ công sau khi nhận chuyển khoản.

Hai là tích hợp cổng thanh toán và cập nhật trạng thái tự động qua callback/webhook.

Cách thứ hai phù hợp hơn cho môi trường có số lượng giao dịch lớn vì giảm nhập tay và rút ngắn thời gian đối soát.

c. Hiện thực trong hệ thống

Hệ thống hiện thực module Fee Collection gồm *Payment Campaign*, *Fee Level*, *Payment Assignment* và luồng thanh toán qua PayOS. Khi sinh viên thanh toán, kết quả được đẩy về qua webhook để cập nhật trạng thái giao dịch trong hệ thống. Toàn bộ lịch sử thanh toán được lưu lại để phục vụ đối soát và kiểm tra khi cần.

3.7.3 Bài toán sinh biểu mẫu tự động từ template Word

a. Khái niệm

Trong công tác Đoàn – Hội, nhiều biểu mẫu có cấu trúc lặp lại. Nếu nhập tay từng hồ sơ sẽ tốn thời gian và dễ sai định dạng. Vì vậy cần một cơ chế sinh biểu mẫu tự động từ template chuẩn.

b. Các phương pháp hiện có

Hai hướng xử lý dễ gặp là:

Hướng thứ nhất là người dùng tải mẫu, điền thủ công và nộp file.

Hướng thứ hai là hệ thống parse placeholder và render tự động từ dữ liệu form.

Cách sinh tự động có ưu điểm rõ hơn: dữ liệu đồng nhất, giảm lỗi nhập và thuận tiện lưu trữ lâu dài.

c. Hiện thực trong hệ thống

Với hệ thống này, quản trị viên có thể upload template `.docx`, hệ thống tự đọc placeholder và sinh cấu hình field tương ứng. Người dùng chỉ cần điền form động trên giao diện, backend sẽ render tài liệu bằng Docxtemplater. File kết quả được lưu trên S3, metadata được lưu trong database để tra cứu và tải lại khi cần.

3.7.4 Bài toán chatbot hỏi đáp CTXH bằng RAG

a. Khái niệm

Sinh viên thường cần tra cứu nhanh thông tin CTXH nhưng không muốn đọc nhiều trang hướng dẫn. Bài toán đặt ra là xây dựng chatbot trả lời nhanh, đồng thời vẫn bám dữ liệu thực của hệ thống để tránh trả lời sai lệch.

b. Các phương pháp hiện có

Qua khảo sát, có ba hướng phổ biến:

Hướng đầu tiên là FAQ tĩnh theo từ khóa.

Hướng thứ hai là chatbot dùng mô hình ngôn ngữ nhưng không truy xuất dữ liệu hệ thống.

Hướng thứ ba là RAG, tức truy xuất dữ liệu liên quan trước khi sinh câu trả lời.

Trong bối cảnh CTXH thay đổi theo từng đợt hoạt động, RAG là hướng phù hợp hơn vì giữ được độ bám dữ liệu và cập nhật tốt hơn FAQ tĩnh.

c. Hiện thực trong hệ thống

Hệ thống triển khai chatbot CTXH theo hướng truy xuất thông tin trước khi sinh câu trả lời. Dữ liệu CTXH được chuẩn bị để hệ thống có thể tìm kiếm ngữ cảnh phù hợp trước khi phản hồi cho người dùng. Cách tổ chức này giúp câu trả lời bám sát nội dung hoạt động và thuận tiện cập nhật khi dữ liệu thay đổi.

Chương 4 Thiết kế hệ thống

4.1 Kiến trúc tổng thể

Từ kết quả phân tích ở Chương 3, hệ thống được thiết kế theo hướng mô-đun hóa nghiệp vụ, tách biệt rõ lớp giao diện, lớp dịch vụ, lớp truy cập dữ liệu và lớp tích hợp hệ thống ngoài. Cách tiếp cận này giúp hệ thống vừa đáp ứng tốt yêu cầu hiện tại của cổng thông tin Đoàn – Hội, vừa thuận lợi khi mở rộng thêm chức năng trong các giai đoạn tiếp theo.

4.1.1 Kiến trúc logic nhiều lớp

Về mặt logic, hệ thống được tổ chức theo bốn lớp chính:

- **Lớp trình bày (Presentation Layer):** frontend sử dụng React + TypeScript, tổ chức theo các module chức năng như Người dùng, Tổ chức, CTXH, Thu phí, Biểu mẫu, Chatbot. Lớp này chịu trách nhiệm điều hướng, hiển thị dữ liệu, kiểm tra đầu vào phía client và tương tác API.
- **Lớp dịch vụ nghiệp vụ (Application/Business Layer):** backend sử dụng NestJS, triển khai theo module domain. Tại đây xử lý toàn bộ logic nghiệp vụ: xác thực, phân quyền, quản lý trạng thái nghiệp vụ, kiểm tra ràng buộc dữ liệu, và phối hợp các tiến trình nền.
- **Lớp dữ liệu (Data Layer):** sử dụng PostgreSQL làm kho dữ liệu giao dịch chính, truy cập thông qua TypeORM. Các quan hệ thực thể và ràng buộc toàn vẹn được quản lý tập trung tại backend.
- **Lớp tích hợp (Integration Layer):** tích hợp các dịch vụ ngoài như PayOS (thanh toán), S3-compatible storage (lưu file), dịch vụ mô hình ngôn ngữ và vector store cho chatbot RAG.

Việc phân lớp theo trách nhiệm giúp giảm phụ thuộc chéo giữa các thành phần, tăng khả năng kiểm thử độc lập và hạn chế rủi ro khi thay đổi một module cụ thể.

4.1.2 Thiết kế backend theo module

Backend được thiết kế theo mô hình module của NestJS, trong đó mỗi nhóm nghiệp vụ được đóng gói thành một module độc lập để dễ phát triển song song và tái sử dụng:

- **Nhóm nền tảng:** `AuthModule`, `RolesModule`, `MetricsModule`, `AwsS3ConfigModule`.
- **Nhóm quản trị tổ chức – nhân sự:** `UsersModule`, `OrganizationsModule`, `DepartmentsModule`, `ClassesModule`.
- **Nhóm dashboard và báo cáo điều hành:** `DashboardModule`.
- **Nhóm CTXH:** `CtxhModule` cho quản lý hoạt động/chấm điểm và `CtxhChatModule` cho tra cứu bằng RAG.
- **Nhóm thu phí:** `PaymentModule`, `PayosModule` xử lý campaign, assignment, fee level và đồng bộ trạng thái giao dịch.
- **Nhóm biểu mẫu:** `TemplatesModule` quản lý template Word, cấu hình field động và sinh tài liệu đầu ra.

Trong mỗi module, luồng xử lý tuân theo chuỗi Controller → Service → Repository/Entity. Cấu trúc này giúp phân tách rõ vai trò: controller nhận request, service hiện thực nghiệp vụ, repository thao tác dữ liệu.

4.1.3 Thiết kế frontend và cơ chế giao tiếp

Frontend sử dụng React + Vite, tổ chức theo hướng feature-based để bám theo các phân hệ nghiệp vụ. Quản lý trạng thái truy vấn và cache dữ liệu sử dụng `@tanstack/react-query`, nhờ đó giảm số lần gọi API lặp lại và tăng độ mượt của giao diện.

Giao tiếp frontend – backend sử dụng REST API qua `axios`. Cơ chế xác thực dựa trên JWT và token refresh. Tại phía giao diện, các thành phần hiển thị được điều chỉnh theo danh sách quyền nhận từ backend, đảm bảo người dùng chỉ nhìn thấy chức năng nằm trong phạm vi vai trò được cấp.

4.1.4 Kiến trúc triển khai và vận hành

Hệ thống được đóng gói container và triển khai theo kiến trúc dịch vụ:

- **Frontend container:** phục vụ ứng dụng React thông qua Nginx.
- **Backend container:** chạy ứng dụng NestJS, cung cấp API và xử lý nghiệp vụ.
- **PostgreSQL + pgvector:** lưu dữ liệu giao dịch và dữ liệu vector phục vụ RAG.
- **Redis/Bull Queue:** hỗ trợ các tác vụ nền và hàng đợi xử lý batch.
- **Prometheus + Grafana:** thu thập và trực quan hóa metrics vận hành.

Ở môi trường phát triển, hệ thống hỗ trợ thêm các thành phần tiện ích như pgAdmin và Ollama để kiểm thử cục bộ. Ở môi trường triển khai chính thức, cấu hình được tách riêng theo biến môi trường nhằm bảo đảm an toàn thông tin truy cập và thuận lợi khi chuyển môi trường.

4.2 Thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu của hệ thống bám theo hướng domain-driven: mỗi phân hệ nghiệp vụ sở hữu nhóm bảng riêng, nhưng vẫn liên kết thống nhất thông qua các thực thể lõi như `users`, `organizations`. Cách tổ chức này giúp hệ thống vừa nhất quán về mặt dữ liệu, vừa giảm độ phức tạp khi truy vấn liên miền nghiệp vụ.

4.2.1 Thiết kế cơ sở dữ liệu ý niệm

Ở mức ý niệm, mô hình dữ liệu được chia thành sáu cụm nghiệp vụ chính:

1. Cụm người dùng – tổ chức

- `users` lưu thông tin định danh, học vụ, trạng thái thành viên, thông tin xác thực.
- `organizations` mô hình hóa cây tổ chức phân cấp (Trường/Khoa/Lớp/Đoàn/Hội/CLB).
- `user_organizations` biểu diễn quan hệ nhiều-nhiều giữa người dùng và tổ chức, kèm vai trò thành viên trong đơn vị.
- `departments`, `classes` biểu diễn khoa và lớp phục vụ quản lý học vụ.

2. Cụm RBAC phân quyền

- `permissions` định nghĩa đơn vị quyền thao tác.
- `roles` gom nhóm các quyền theo vai trò nghiệp vụ.
- `user_roles` gán vai trò cho người dùng theo ngữ cảnh toàn cục hoặc theo tổ chức.

Mô hình này cho phép một người dùng giữ nhiều vai trò khác nhau ở các đơn vị khác nhau, đúng với thực tế vận hành Đoàn – Hội đa cấp.

3. Cụm CTXH

- `ctxh_category` quản lý danh mục hoạt động.
- `ctxh_activity` quản lý thông tin hoạt động và vòng đời trạng thái.
- `ctxh_participation` lưu kết quả tham gia/chấm điểm của từng sinh viên theo hoạt động.
- `ctxh_limit_config` lưu cấu hình giới hạn số ngày CTXH theo tổ chức.

4. Cụm thu phí

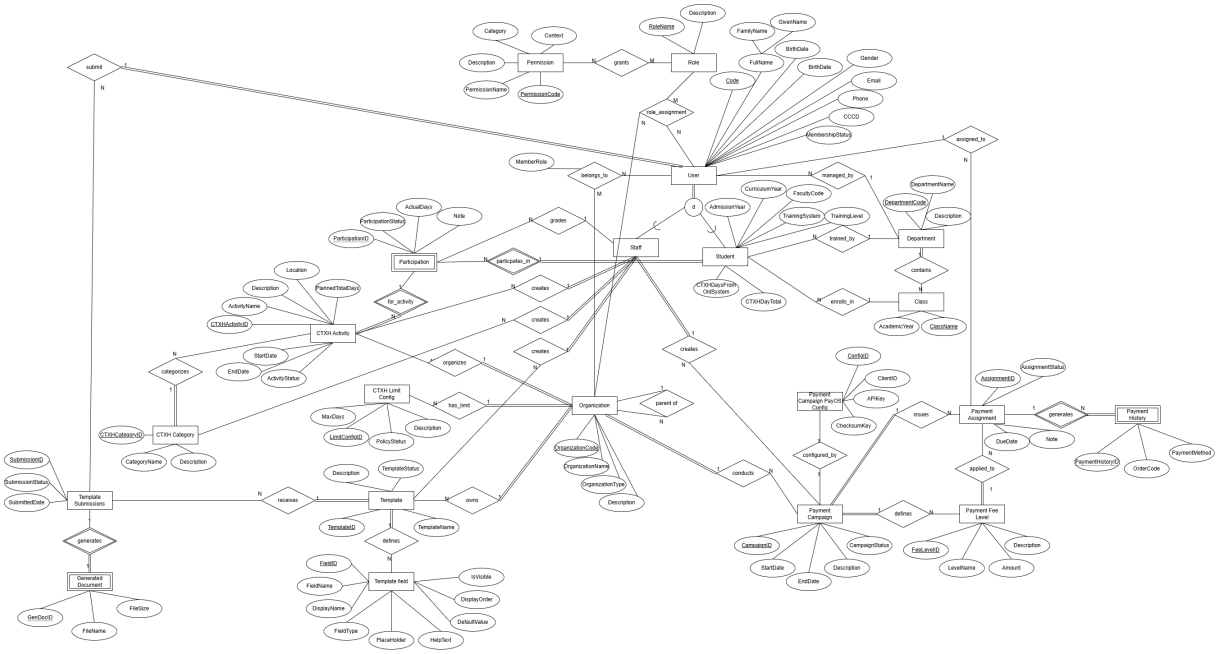
- `payment_campaigns` đại diện các đợt thu theo tổ chức.
- `payment_fee_levels` cấu hình mức phí áp dụng cho campaign.
- `payment_assignments` gán nghĩa vụ đóng phí cho từng người dùng.
- `payment_history` lưu lịch sử giao dịch và trạng thái thanh toán.
- `payment_campaign_payos_configs` lưu cấu hình PayOS theo chiến dịch/tổ chức.

5. Cụm biểu mẫu

- `templates` lưu metadata của mẫu `.docx` và kết quả parse schema.
- `template_fields` lưu cấu hình trường động, validation, thứ tự hiển thị.
- `template_submissions` lưu dữ liệu người dùng nộp và file đầu ra đã render.

6. Cụm dữ liệu RAG

- `ctxh_embeddings` lưu các vector embedding theo từng hoạt động và từng chunk nội dung.
- Bảng này liên kết về `ctxh_activity`, là cầu nối giữa dữ liệu nghiệp vụ CTXH và lớp truy xuất ngữ nghĩa.



Hình 45: Entity-Relationship Model cho toàn bộ hệ thống

4.2.2 Thiết kế cơ sở dữ liệu vật lý

Ở mức vật lý, hệ thống sử dụng PostgreSQL với các đặc trưng chính sau:

1. Quy ước khóa và kiểu dữ liệu

- Khóa chính của các bảng nghiệp vụ dùng `UUID`, phù hợp cho hệ phân tán và giảm nguy cơ lộ thông tin tuần tự.
- Trường thời gian dùng `timestamp` và được quản lý bởi các cột audit như `created_at`, `updated_at`, `deleted_at`.
- Các trường trạng thái dùng `ENUM` để ép ràng buộc miền giá trị ngay tại tầng dữ liệu.

2. Ràng buộc toàn vẹn và quan hệ

- Các quan hệ lõi được ràng buộc bằng foreign key, đi kèm chính sách `CASCADE` ở những nơi cần dọn dẹp dữ liệu phụ thuộc (ví dụ quan hệ user-role, user-organization, embedding-activity).
- Với phân quyền, bảng `user_roles` đặt unique index trên bộ `(userId, roleId, organizationId)` để chống gán trùng vai trò trong cùng phạm vi.
- Với cây tổ chức, hệ thống sử dụng mô hình `closure-table` để tối ưu truy vấn tổ tiên/hậu duệ trong cấu trúc phân cấp nhiều mức.

3. Chỉ mục và tối ưu truy vấn

- Các bảng nghiệp vụ có chỉ mục theo cột truy vấn thường xuyên như `activity_id`, `created_at`, tổ hợp `(templateId, status)`, `(userId, createdAt)`.
- Dữ liệu vector trong `ctxh_embeddings` sử dụng kiểu `vector(1024)`; chỉ mục HNSW với `vector_cosine_ops` được dùng để tăng tốc truy vấn tương tự cho chatbot.
- Metadata của embeddings lưu dạng `jsonb` và được bổ sung GIN index để hỗ trợ lọc theo thuộc tính ngữ cảnh.

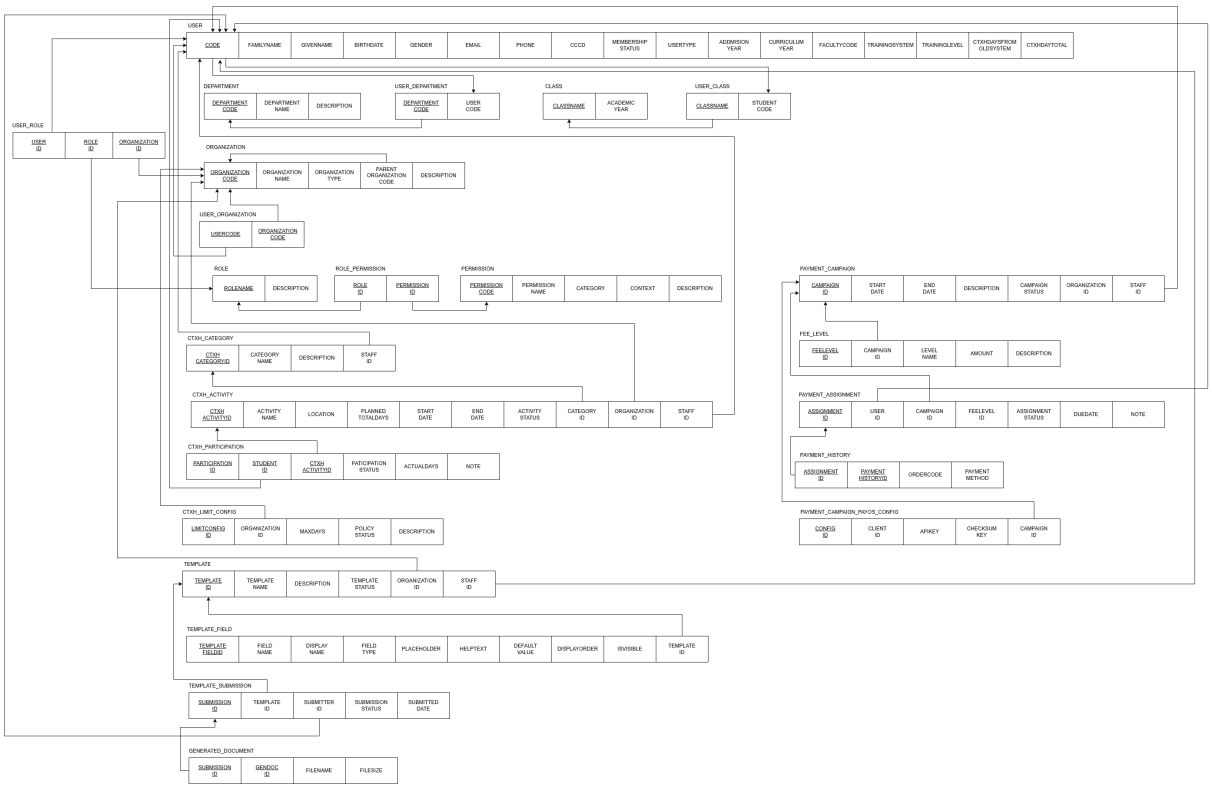
4. Cấu trúc dữ liệu theo trạng thái nghiệp vụ Các bảng quan trọng đều biểu diễn vòng đời xử lý thông qua cột trạng thái, ví dụ:

- `ctxh_activity`: `DRAFT`, `ACTIVE`, `COMPLETED`, `CANCELLED`.
- `payment_campaigns`: `DRAFT`, `ACTIVE`, `COMPLETED`, `CANCELLED`.
- `payment_assignments`: `PENDING`, `PAID`, `OVERDUE`, `EXEMPTED`.
- `template_submissions`: `draft`, `submitted`, `completed`, `failed`.

Thiết kế này giúp backend hiện thực rõ ràng các luật chuyển trạng thái và giảm rủi ro cập nhật dữ liệu sai ngữ cảnh nghiệp vụ.

5. Mở rộng và bảo trì dữ liệu Mô hình dữ liệu hiện tại đã được chuẩn hóa ở mức phù hợp cho hệ thống vận hành thực tế: tách riêng bảng theo miền nghiệp vụ, dùng khóa ngoại để bảo toàn quan hệ, và có không gian mở rộng cho các tính năng mới (thêm module mới, thêm thuộc tính metadata, mở rộng quy tắc phân quyền theo ngữ cảnh).

Nhờ đó, khi hệ thống tăng quy mô người dùng và khối lượng giao dịch, kiến trúc dữ liệu vẫn duy trì được tính nhất quán, khả năng truy vấn và mức độ dễ bảo trì.



Hình 46: Relational Data Model của toàn bộ hệ thống

4.2.3 Đối chiếu và kế thừa mô hình dữ liệu từ giai đoạn trước

Để bảo đảm tính liên tục học thuật giữa các học kỳ, nhóm thực hiện đối chiếu trực tiếp giữa mô hình dữ liệu tham khảo của giai đoạn trước và schema hiện tại. Nguyên tắc kế thừa được áp dụng là: giữ lại các khái niệm nghiệp vụ còn đúng, đồng thời chuẩn hóa lại tên trường và trạng thái theo mã nguồn thực tế ở phiên bản hiện hành.

Nhóm nghiệp vụ	**Mô hình tham khảo kỳ trước**	**Mô hình hiện tại (đang triển khai)**
Thành viên	USER, STUDENT, STAFF	`users` (một bảng thống nhất), phân loại bằng `userType` và các trường mở rộng học vụ
Đơn vị học thuật	DEPARTMENT, CLASS	`departments`, `classes` và liên kết đến `users`
Cấu trúc tổ chức	ORGANIZATION	`organizations` theo mô hình cây closure-table, kết hợp `user_organizations`
Phân quyền	ROLE, PERMISSION	`roles`, `permissions`, `user_roles`, `role_permissions`
CTXH	CTXH_CATEGORY, CTXH_ACTIVITY, PARTICIPATION, LIMIT_POLICY	`ctxh_category`, `ctxh_activity`, `ctxh_participation`, `ctxh_limit_config`
Biểu mẫu	TEMPLATE, TEMPLATE_FIELD, TEMPLATE_SUBMISSION, GENERATED_DOCUMENT	`templates`, `template_fields`, `template_submissions`; file sinh ra quản lý qua metadata + S3 URL
Đóng phí	PAYMENT_CAMPAIGN, FEE_LEVEL, PAYMENT_ASSIGNMENT, PAYMENT_HISTORY	`payment_campaigns`, `payment_fee_levels`, `payment_assignments`, `payment_history`, `payment_campaign_payos_configs`

Bảng 26: Đối chiếu mô hình dữ liệu tham khảo và mô hình hiện tại

Từ bảng đối chiếu có thể thấy phần lớn khái niệm lõi được kế thừa nguyên vẹn; thay đổi chủ yếu nằm ở mức hiện thực vật lý: gom bảng theo hướng thực dụng, bổ sung audit fields, chuẩn hóa enum trạng thái, và mở rộng dữ liệu để phục vụ tích hợp RAG và xử lý hàng loạt.

4.2.4 Thiết kế dữ liệu phục vụ Chatbot RAG

Bên cạnh dữ liệu giao dịch, hệ thống duy trì một lớp dữ liệu truy xuất ngữ nghĩa để hỗ trợ chatbot hỏi-đáp CTXH. Lớp dữ liệu này được thiết kế nhằm bảo đảm ba mục tiêu đồng thời: (1) truy vấn đúng ngữ cảnh câu hỏi tự nhiên, (2) truy vết được nguồn trả lời, (3) cập nhật linh hoạt khi dữ liệu hoạt động thay đổi.

1. Nguồn dữ liệu đầu vào cho RAG Nguồn dữ liệu gốc lấy từ thực thể hoạt động CTXH và các thực thể liên quan (danh mục, tổ chức). Các thuộc tính nghiệp vụ quan trọng được chuẩn hóa trước khi tạo embedding gồm:

- `id`, `name`, `description`
- `category.id`, `category.name`
- `organization.id`, `organization.name`
- `start_date`, `end_date`, `total_days`, `location`
- `status`

2. Bảng lưu vector và metadata Hệ thống sử dụng bảng `ctxh_embeddings` để lưu:

- Nội dung text đã chuẩn hóa để nhúng vector (`content_text`)
- Vector embedding (kiểu `vector`)
- Chỉ số phân mảnh nội dung (`chunk_index`)
- Metadata phục vụ lọc theo ngữ cảnh thời gian, tổ chức, danh mục, trạng thái

Thiết kế này cho phép vừa tìm kiếm ngữ nghĩa theo độ tương đồng vector, vừa lọc logic theo ràng buộc nghiệp vụ mà người dùng đặt ra trong câu hỏi.

3. Mẫu chuẩn hóa văn bản trước khi embedding Để tăng tính nhất quán ngữ nghĩa giữa các bản ghi hoạt động, hệ thống sử dụng mẫu hợp nhất các thuộc tính chính thành một đoạn văn chuẩn trước khi sinh embedding, ví dụ:

> Tên hoạt động: [name], Mô tả: [description], Danh mục: [category.name],
> Tổ chức: [organization.name], Thời gian: Từ [start_date] đến [end_date],
> Ngày CTXH được cấp: [total_days] ngày, Địa điểm: [location]

Mẫu này giúp giảm sai lệch định dạng giữa các bản ghi, từ đó ổn định chất lượng retrieval khi người dùng đặt câu hỏi đa dạng.

4. Cập nhật dữ liệu RAG theo vòng đời hoạt động Khi dữ liệu hoạt động CTXH thay đổi, lớp vector cần được đồng bộ tương ứng. Thiết kế hiện tại hỗ trợ:

- Tạo mới embeddings khi thêm hoạt động mới đủ điều kiện nghiệp vụ.
- Tái tạo embeddings khi các trường ngữ nghĩa quan trọng của hoạt động được cập nhật.
- Xóa embeddings liên quan khi hoạt động bị xóa hoặc không còn phù hợp cho truy xuất.

Ngoài cơ chế đồng bộ theo sự kiện, hệ thống còn hỗ trợ các tác vụ ingestion toàn phần và incremental để bảo trì định kỳ hoặc khởi tạo lại vector store khi cần.

5. Vai trò của lớp dữ liệu RAG trong toàn hệ thống Lớp dữ liệu RAG không thay thế dữ liệu giao dịch chính mà đóng vai trò chỉ mục ngữ nghĩa phục vụ truy vấn hội thoại. Nhờ tách lớp như vậy, hệ thống vừa giữ được tính toàn vẹn của dữ liệu nghiệp vụ ở PostgreSQL, vừa cung cấp khả năng hỏi-đáp linh hoạt theo ngôn ngữ tự nhiên cho người dùng cuối.

4.3 Các quyết định thiết kế và trade-off

Trong quá trình hiện thực, nhóm không chỉ lựa chọn công nghệ theo tiêu chí phổ biến mà còn cân nhắc rõ các đánh đổi kỹ thuật (trade-off) giữa hiệu năng, độ phức tạp triển khai, khả năng mở rộng và chi phí vận hành.

4.3.1 Tách module nghiệp vụ so với kiến trúc nguyên khối

Quyết định: sử dụng kiến trúc module trong cùng một backend NestJS thay vì một codebase nguyên khối không phân lớp.

Lợi ích:

- Tăng khả năng phát triển song song giữa các nhóm.
- Tái sử dụng được service chung (auth, roles, metrics, storage).
- Dễ khoanh vùng lỗi khi có sự cố ở từng nghiệp vụ.

Đánh đổi:

- Cần quản trị chặt dependency giữa các module để tránh vòng phụ thuộc.
- Cần thống nhất conventions DTO, response schema và guard để không phân mảnh.

4.3.2 RBAC theo ngữ cảnh tổ chức so với phân quyền toàn cục

Quyết định: dùng mô hình `user_roles` có `organizationId` tùy chọn thay vì chỉ gán role toàn hệ thống.

Lợi ích:

- Phản ánh đúng cấu trúc Đoàn – Hội nhiều cấp.
- Một tài khoản có thể đảm nhiệm vai trò khác nhau ở các đơn vị khác nhau.
- Giảm rủi ro cấp quyền quá rộng so với nhu cầu nghiệp vụ thực.

Đánh đổi:

- Truy vấn quyền phức tạp hơn (phải xét context).
- Frontend cần nhận diện rõ phạm vi quyền để hiển thị đúng chức năng.

4.3.3 PostgreSQL + pgvector so với tách riêng vector database

Quyết định: lưu dữ liệu giao dịch và dữ liệu vector trong cùng nền PostgreSQL, sử dụng extension pgvector.

Lợi ích:

- Giảm độ phức tạp hạ tầng so với vận hành thêm một vector DB độc lập.
- Dễ đồng bộ khóa ngoại giữa `ctxh_activity` và `ctxh_embeddings`.
- Thuận lợi backup/restore toàn bộ hệ thống dữ liệu.

Đánh đổi:

- Khi dữ liệu vector tăng mạnh, cần tinh chỉnh chỉ mục và chiến lược ingestion kỹ hơn.
- Cần cân bằng tài nguyên giữa workload OLTP và workload semantic search.

4.3.4 Batch processing cho import dữ liệu lớn

Quyết định: dùng batch + retry/fallback thay vì xử lý một transaction khổng lồ cho toàn file.
Lợi ích:

- Hạn chế timeout với file lớn.
- Cô lập lỗi theo lô, dễ rollback cục bộ và báo cáo lỗi rõ hơn.
- Cải thiện trải nghiệm người dùng qua trạng thái tiến trình.

Đánh đổi:

- Tăng độ phức tạp logic điều phối phiên import.
- Cần kiểm soát tính nhất quán khi có retry hoặc dừng giữa chừng.

4.3.5 Template động + lưu file S3 so với lưu BLOB trong database

Quyết định: tách metadata trong PostgreSQL và lưu file Word đầu vào/đầu ra trên S3-compatible storage.
Lợi ích:

- Giảm tải kích thước dữ liệu trong DB giao dịch.
- Tăng khả năng mở rộng dung lượng tài liệu dài hạn.
- Dễ triển khai chính sách lifecycle và backup cho file.

Đánh đổi:

- Cần cơ chế đồng bộ trạng thái giữa metadata và file thật.
- Cần kiểm soát quyền truy cập URL file đầu ra.

Nhìn chung, các quyết định trên cho thấy hệ thống ưu tiên tính khả thi triển khai và độ ổn định vận hành trong bối cảnh nghiệp vụ thực của nhà trường, đồng thời vẫn giữ biên mở rộng đủ lớn cho các yêu cầu nâng cấp ở giai đoạn sau.

Chương 5 Hiện thực hệ thống

5.1 Cấu trúc mã nguồn

Hệ thống được tổ chức theo mô hình monorepo, trong đó backend và frontend được tách thành hai workspace độc lập để thuận lợi cho phát triển song song và triển khai riêng vòng đời build/deploy.

5.1.1 Tổ chức thư mục tổng thể

Ở mức repository, mã nguồn được chia thành các khối chính:

- **backend/**: mã nguồn NestJS, mô hình dữ liệu, module nghiệp vụ, scripts migration/seed.
- **frontend/**: mã nguồn React + TypeScript + Vite, các feature UI, i18n, utils.
- **grafana/, prometheus*.yml**: cấu hình giám sát vận hành.
- **docker-compose*.yml**: cấu hình môi trường chạy nhiều dịch vụ.
- **report/LuanVan/**: tài liệu luận văn và hình minh họa.

Việc tách các khối theo vai trò giúp quy trình cộng tác rõ ràng: nhóm backend tập trung logic nghiệp vụ và dữ liệu, nhóm frontend tập trung trải nghiệm người dùng, nhóm vận hành tập trung hạ tầng và quan sát hệ thống.

5.1.2 Cấu trúc backend

Backend được xây dựng trên NestJS và tổ chức theo module nghiệp vụ. Mỗi module thường gồm:

- `controller`: định nghĩa endpoint và hợp đồng API.
- `service`: hiện thực logic nghiệp vụ và orchestration.
- `dto`: định nghĩa input/output contract và validation.
- `entity`: ánh xạ bảng dữ liệu qua TypeORM.

Các module lõi đã triển khai gồm xác thực/ủy quyền, người dùng, tổ chức, CTXH, thu phí, biểu mẫu, chatbot RAG, cấu hình S3, xuất báo cáo và quan sát metrics.

Luồng xử lý chuẩn trong backend:

1. Request đi vào controller.
2. Guard và pipe kiểm tra xác thực, quyền hạn, định dạng dữ liệu.
3. Service xử lý nghiệp vụ và gọi repository/entity.
4. Kết quả trả về dạng JSON chuẩn hóa cho frontend.

5.1.3 Cấu trúc frontend

Frontend sử dụng React + Vite + TypeScript. Cấu trúc mã nguồn được tổ chức theo feature để bám sát nghiệp vụ:

- **features/**: các phân hệ như người dùng, tổ chức, CTXH, payment, template, chatbot.
- **shared/**: component dùng chung, constants, hooks và kiểu dữ liệu dùng lại.
- **i18n/**: cấu hình đa ngôn ngữ và tài nguyên dịch.

- **utils/**: tiện ích xử lý dữ liệu phía client.

Frontend giao tiếp backend qua `axios` và quản lý vòng đời truy vấn bằng `React Query`. Cách tổ chức này giúp dữ liệu đồng bộ tốt giữa các màn hình, giảm gọi API dư thừa và kiểm soát rõ trạng thái loading/error.

5.2 Các module và API chính

5.2.1 Module xác thực và phân quyền

Module xác thực hiện thực các chức năng đăng nhập nội bộ, đăng nhập Google OAuth, refresh token, logout, lấy profile và lấy quyền hiện tại của người dùng.

Các endpoint điển hình:

- `POST /auth/login`
- `POST /auth/refresh`
- `POST /auth/logout`
- `GET /auth/profile`
- `GET /auth/me/permissions`
- `GET /auth/google`, `GET /auth/google/callback`

Sau xác thực, hệ thống áp dụng RBAC theo ngữ cảnh tổ chức. Người dùng có thể được cấp nhiều vai trò trên nhiều đơn vị khác nhau. Khi frontend gọi API nghiệp vụ, guard phân quyền sẽ đối chiếu quyền yêu cầu với bộ quyền hiệu lực của người dùng tại scope tương ứng.

5.2.2 Module quản lý người dùng, tổ chức, khoa, lớp

Các module quản trị dữ liệu nền hiện thực đầy đủ CRUD, phân trang, lọc, import/export Excel và một số luồng batch processing cho dữ liệu lớn.

Người dùng:

- `GET /users`, `GET /users/:id`, `POST /users`, `PATCH /users/:id`, `DELETE /users/:id`
- `POST /users/import/upload`
- `GET /users/import/:sessionId/validate`
- `POST /users/import/:sessionId/execute`
- `POST /users/import/batch` cho kịch bản import phía client theo lô lớn

Tổ chức:

- `GET /organizations/tree`, `GET /organizations/:id/children`
- `GET /organizations/stats`, `GET /organizations/accessible`
- Các endpoint tối ưu truy vấn theo độ sâu cây và theo loại/mã tổ chức

Khoa/Lớp:

- CRUD chuẩn cho `/departments` và `/classes`
- Hỗ trợ import theo session và import batch

5.2.3 Module CTXH

Module CTXH bao gồm quản lý danh mục, hoạt động, người tham gia, cấu hình giới hạn và báo cáo:

- `/ctxh/categories`: tạo/sửa/xóa/xem danh mục phân loại hoạt động CTXH.
- `/ctxh/activities`: tạo/sửa/xóa/lọc hoạt động theo trạng thái, thời gian, tổ chức, danh mục.
- `/ctxh/limit-config`: quản lý cấu hình số ngày CTXH tối đa theo tổ chức.
- `/ctxh/import/old-system`: validate và nhập dữ liệu CTXH từ hệ thống cũ.
- `/ctxh/reports/student/:studentCode`: báo cáo CTXH cá nhân theo MSSV.

Các luồng import/chấm điểm được thiết kế theo hướng chịu tải file lớn, có bước validate trước khi commit để giảm rủi ro lỗi dữ liệu hàng loạt.

5.2.4 Module thu phí

Module thu phí hiện thực theo mô hình campaign-based:

- `/payment-campaigns`: quản lý chiến dịch thu phí và vòng đời trạng thái.
- `/payment-fee-levels`: quản lý mức phí trong từng campaign.
- `/payment/assignments`: gán nghĩa vụ đóng phí cho người dùng, theo dõi trạng thái pending/paid/overdue.
- `/payment-campaigns/:id/payos-config`: cấu hình PayOS theo campaign.
- `/api/payment-admin/*`: các endpoint kiểm tra/sync dữ liệu giao dịch phục vụ vận hành.

Thiết kế này tách rõ: cấu hình nghiệp vụ (campaign/fee), phân công đối tượng (assignment), và lịch sử giao dịch thực tế (payment history), giúp đối soát dễ dàng và truy xuất đầy đủ audit trail.

5.2.5 Module biểu mẫu điện tử

Module biểu mẫu gồm hai phần chính:

- `/templates`: upload template Word, parse placeholder, cấu hình field động.
- `/submissions`: lưu draft, submit, sinh tài liệu đầu ra, tải file và lấy public URL.

Quy trình hiện thực:

1. Quản trị viên upload `.docx`.
2. Hệ thống parse placeholder và sinh schema field.
3. Người dùng điền biểu mẫu động.
4. Backend render file Word, lưu file lên S3 và lưu metadata vào database.

5.2.6 Bảng tổng hợp API chính theo module

Để thuận tiện theo dõi ở góc độ triển khai, bảng sau tổng hợp các nhóm API đại diện đang vận hành trong hệ thống.

Module	Nhóm endpoint chính	Mục đích nghiệp vụ
Auth & RBAC	`/auth/*`, `/roles/*`	Xác thực, quản lý phiên, lấy quyền và phân vai trò theo ngữ cảnh tổ chức
Users	`/users/*`, `/users/import/*`	Quản lý người dùng, import dữ liệu lớn, gán vai trò và lọc danh sách
Organizations	`/organizations/*`	Quản lý cây tổ chức, tra cứu theo cấp, thống kê và quyền truy cập theo user
Classes/Departments	`/classes/*`, `/departments/*`	Quản lý dữ liệu học thuật nền và import batch
CTXH	`/ctxh/activities/*`, `/ctxh/limit-config/*`, `/ctxh/reports/*`	Vận hành vòng đời hoạt động, chấm điểm, giới hạn ngày và báo cáo
Payment	`/payment-campaigns/*`, `/payment/assignments/*`, `/api/payment-admin/*`	Quản lý chiến dịch thu phí, phân công đóng phí, đối soát và vận hành giao dịch
Templates	`/templates/*`, `/submissions/*`	Upload template, cấu hình field động, nộp biểu mẫu và sinh tài liệu
RAG Chatbot	`/api/ctxh-chat/*`	Hỏi-đáp ngữ nghĩa, quản lý lịch sử hội thoại, ingestion và health check
Infra Support	`/aws-s3-config/*`, `/export/*`	Quản lý cấu hình lưu trữ và xuất dữ liệu nghiệp vụ

Bảng 27: Tổng hợp API chính theo module

5.3 Hiện thực Chatbot RAG

5.3.1 Mục tiêu hiện thực

Chatbot RAG được hiện thực để giải quyết bài toán tra cứu CTXH bằng ngôn ngữ tự nhiên trong bối cảnh dữ liệu hoạt động nhiều, thay đổi liên tục, và người dùng cần câu trả lời nhanh nhưng vẫn truy vết được nguồn.

Các mục tiêu kỹ thuật chính:

- Truy vấn ngữ nghĩa dựa trên dữ liệu nội bộ CTXH.
- Trả lời có nguồn trích dẫn (source attribution).
- Hỗ trợ hội thoại nhiều lượt với session.
- Tối ưu cân bằng giữa độ đúng, độ phủ và chi phí vận hành.

5.3.2 Kiến trúc kỹ thuật của module chatbot

Module chatbot đặt tại backend và tổ chức thành các service chuyên biệt:

- **RAG Chain Service:** điều phối luồng router, retrieval và generation.
- **Vector Store Service:** quản lý kết nối pgvector, similarity search, MMR, metadata filters.
- **Ingestion Service:** đồng bộ dữ liệu hoạt động sang vector store (full/incremental).
- **Memory Service:** quản lý lịch sử hội thoại theo session.
- **Advanced Retriever/Tooling:** mở rộng chiến lược truy vấn theo bối cảnh câu hỏi.

Thiết kế chia nhỏ service giúp dễ kiểm thử từng thành phần, đồng thời thuận lợi khi thay đổi mô hình LLM hoặc embedding provider.

5.3.3 Dữ liệu đầu vào cho RAG và chuẩn hóa embedding

Nguồn dữ liệu phục vụ chatbot lấy từ thực thể hoạt động CTXH đã được chuẩn hóa về nội dung. Các trường chính gồm:

- Định danh hoạt động: `id`, `name`
- Nội dung mô tả: `description`
- Danh mục: `category.id`, `category.name`
- Tổ chức: `organization.id`, `organization.name`
- Thời gian và giá trị CTXH: `start_date`, `end_date`, `total_days`
- Ngữ cảnh vận hành: `location`, `status`

Trước khi tạo embedding, dữ liệu được đưa về template văn bản thống nhất để giảm nhiễu định dạng và tăng chất lượng truy vấn ngữ nghĩa.

5.3.4 Pipeline ingestion và đồng bộ vector

Hệ thống hỗ trợ hai chế độ ingestion:

- **Full ingestion:** tái tạo vector cho toàn bộ tập hoạt động.
- **Incremental ingestion:** chỉ cập nhật các bản ghi thay đổi theo mốc thời gian.

Quy trình ingestion gồm các bước:

1. Đọc dữ liệu hoạt động và quan hệ liên quan.
2. Chuẩn hóa text, tách chunk nếu cần.
3. Tạo embedding theo provider cấu hình.
4. Ghi vector + metadata vào bảng `ctxh_embeddings`.
5. Cập nhật chỉ mục và thống kê trạng thái ingestion.

Thiết kế này cho phép khởi tạo nhanh dữ liệu AI khi triển khai mới và bảo trì linh hoạt trong vận hành định kỳ.

5.3.5 Pipeline trả lời câu hỏi

Luồng xử lý truy vấn người dùng được hiện thực theo các pha sau:

1. **Nhận request hội thoại:** API nhận `message`, `sessionId` và ngữ cảnh người dùng.
2. **Tiền xử lý câu hỏi:** router xác định intent, rewrite câu hỏi nếu cần theo ngữ cảnh hội thoại.
3. **Retrieval:** thực hiện tìm kiếm vector với chiến lược phù hợp (semantic/filter/MMR/multi-query).
4. **Generation:** gửi context truy xuất được vào LLM để sinh câu trả lời tự nhiên.
5. **Hậu xử lý:** chuẩn hóa output, gắn nguồn tham chiếu, ghi lịch sử hội thoại.

Module hỗ trợ cả chế độ trả lời thường và stream SSE, phù hợp cho giao diện chat thời gian thực.

5.3.6 API chatbot đã hiện thực

Các endpoint chính của module gồm:

- `POST /api/ctxh-chat/message`: gửi câu hỏi và nhận câu trả lời có nguồn.
- `SSE /api/ctxh-chat/stream`: nhận phản hồi dạng streaming.
- `GET /api/ctxh-chat/history/:sessionId`: lấy lịch sử hội thoại.
- `DELETE /api/ctxh-chat/session/:sessionId`: xóa session hội thoại.
- Nhóm endpoint ingestion/health phục vụ vận hành và kiểm tra trạng thái mô hình.

Trong response, hệ thống trả về cả metadata như `responseTime`, `sources`, `confidence` và có hỗ trợ chế độ evaluation để lấy full context cho kiểm thử chất lượng retrieval.

5.3.7 Giới hạn hiện tại và định hướng mở rộng

Mặc dù đã đáp ứng tốt nhu cầu tra cứu CTXH, chatbot vẫn có một số giới hạn thực tế:

- Phụ thuộc chất lượng dữ liệu đầu vào ở module CTXH.
- Một số câu hỏi nghiệp vụ nội bộ sâu có thể cần thêm nguồn tri thức ngoài activity data.
- Cần tối ưu thêm chiến lược rerank khi dữ liệu tăng rất lớn theo thời gian.

Định hướng mở rộng ưu tiên gồm: tăng chất lượng filter extraction, cải thiện đánh giá tự động theo bộ testset, và chuẩn hóa hơn nữa luồng quan sát hiệu năng retrieval-generation trong production.

5.3.8 Đánh đổi kỹ thuật trong hiện thực chatbot

Trong quá trình hiện thực, nhóm cân nhắc nhiều đánh đổi kỹ thuật để đạt cân bằng giữa độ chính xác, độ trễ phản hồi và chi phí vận hành.

1. Retrieval đa chiến lược vs chỉ semantic search

- Giải pháp hiện tại cho phép chọn giữa semantic, filtered search và diverse/MMR tùy ngữ cảnh câu hỏi.
- Đổi lại, logic điều phối phức tạp hơn so với một chiến lược retrieval duy nhất.
- Bù lại, chất lượng trả lời ổn định hơn trong cả câu hỏi tổng quát lẫn câu hỏi có điều kiện lọc cụ thể.

2. Memory theo session vs stateless chat

- Duy trì session giúp chatbot xử lý tốt câu hỏi nối tiếp nhiều lượt.
- Tuy nhiên phải kiểm soát TTL và cắt lịch sử để tránh phình bộ nhớ, đặc biệt khi số phiên đồng thời lớn.

3. Ingestion tức thời vs ingestion theo lô

- Ingestion theo lô và incremental giúp ổn định hệ thống và dễ kiểm soát chất lượng dữ liệu vector.
- Đổi lại, sẽ có độ trễ nhỏ giữa thời điểm dữ liệu nghiệp vụ đổi và thời điểm tri thức mới xuất hiện trong chatbot.

4. Một cơ sở dữ liệu tích hợp vs tách hạ tầng riêng

- Dùng PostgreSQL + pgvector giúp đơn giản hóa hạ tầng và backup.
- Đổi lại, cần giám sát tài nguyên kỹ để tránh cạnh tranh tải giữa truy vấn giao dịch và truy vấn vector.

Các đánh đổi trên phản ánh định hướng thực dụng của hệ thống: ưu tiên giải pháp có thể triển khai ổn định trong môi trường thật, rồi tối ưu sâu dần theo dữ liệu vận hành.

5.4 Các quyết định thiết kế hiện thực và trade-off

Phần này tổng hợp các quyết định kỹ thuật quan trọng ở tầng hiện thực để làm rõ vì sao nhóm chọn hướng triển khai hiện tại thay vì các phương án thay thế.

5.4.1 Xử lý dữ liệu lớn theo batch thay vì đồng bộ toàn cục

Lý do chọn:

- Dữ liệu import có thể rất lớn (người dùng, lớp, dữ liệu CTXH từ hệ cũ).
- Batch processing giảm nguy cơ timeout và cho phép retry cục bộ.

Trade-off:

- Tăng phức tạp quản lý trạng thái phiên import.
- Cần cơ chế tổng hợp lỗi rõ ràng để người dùng xử lý sau import.

5.4.2 Bảo mật theo guard tập trung thay vì kiểm tra thủ công ở từng service

Lý do chọn:

- Dùng `JwtAuthGuard` + `PermissionGuard` giúp nhất quán kiểm soát truy cập.
- Giảm lặp code kiểm tra quyền trong từng endpoint.

Trade-off:

- Yêu cầu đặc tả permission codes đầy đủ và đồng bộ với UI.
- Khi permission matrix thay đổi lớn, cần seed/migration dữ liệu quyền tương ứng.

5.4.3 Tách metadata và file nhị phân

Lý do chọn:

- Database lưu metadata có cấu trúc; storage ngoài (S3) lưu file đầu vào/đầu ra dung lượng lớn.
- Tối ưu chi phí và hiệu năng truy vấn dữ liệu nghiệp vụ.

Trade-off:

- Cần bảo đảm tính nhất quán giữa bản ghi metadata và file vật lý.
- Cần thêm cơ chế test kết nối và rotate cấu hình lưu trữ theo môi trường.

5.4.4 Quan sát hệ thống ngay từ giai đoạn triển khai

Lý do chọn:

- Tích hợp Prometheus và Grafana sớm giúp phát hiện điểm nghẽn thực tế ở các luồng import, chat và payment.
- Giảm thời gian xử lý sự cố khi hệ thống mở rộng số lượng người dùng.

Trade-off:

- Tăng khối lượng cấu hình vận hành ban đầu.
- Cần duy trì dashboard và ngưỡng cảnh báo phù hợp theo từng giai đoạn tải.

5.5 Giao diện hệ thống

5.5.1 Nguyên tắc hiện thực giao diện

Giao diện được hiện thực theo ba nguyên tắc:

- **Role-aware UI:** chức năng hiển thị theo quyền của người dùng.
- **Task-oriented flow:** ưu tiên các luồng nghiệp vụ thường dùng (import, tạo hoạt động, phân công thu phí, nộp biểu mẫu).
- **Feedback rõ ràng:** mọi thao tác có trạng thái loading/success/error để tránh mơ hồ khi thao tác trên dữ liệu thật.

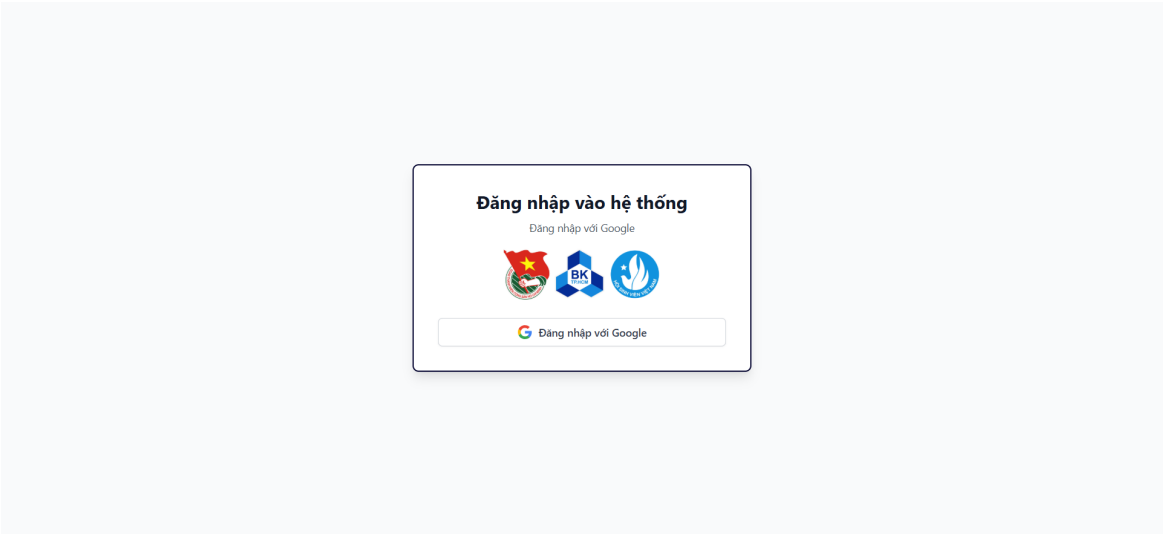
5.5.2 Các nhóm màn hình chính

Hệ thống giao diện đã hiện thực các nhóm màn hình:

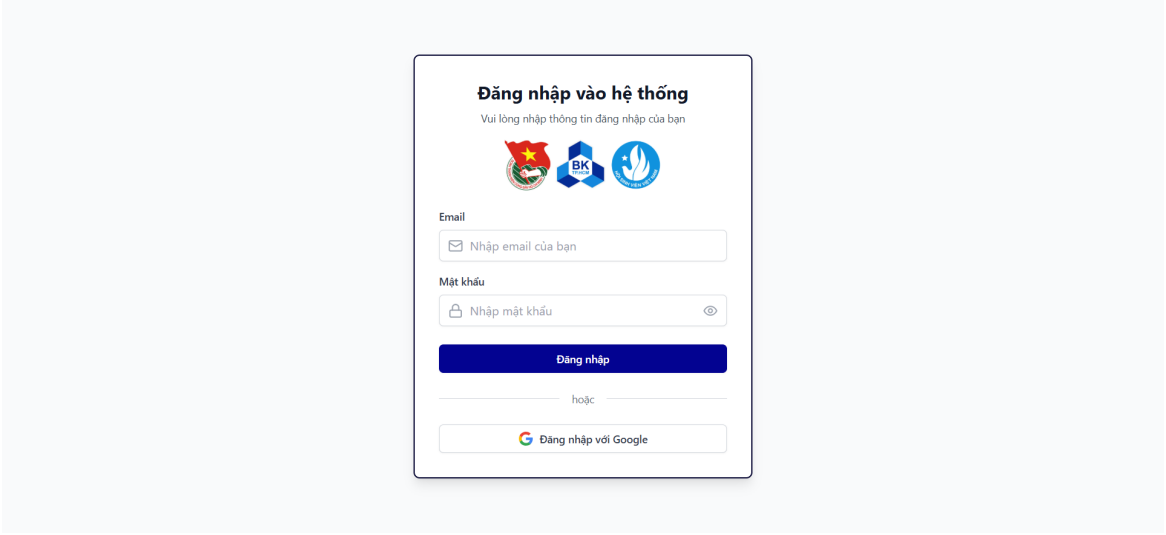
- **Nhóm xác thực:** đăng nhập nội bộ, callback OAuth, quản lý phiên.
- **Nhóm quản trị dữ liệu nền:** người dùng, tổ chức, khoa, lớp, vai trò và quyền.
- **Nhóm quản lý công tác xã hội:** danh mục, hoạt động, danh sách tham gia, chấm điểm, import từ hệ cũ, báo cáo.
- **Nhóm thu phí:** campaign, mức phí, phân công, theo dõi trạng thái thanh toán.
- **Nhóm biểu mẫu:** quản lý template, cấu hình field, nộp biểu mẫu, tải file sinh tự động.
- **Nhóm chatbot:** giao diện hội thoại, hiển thị nguồn tham chiếu và lịch sử session.

5.5.3 Minh họa giao diện theo phân hệ

Các hình dưới đây minh họa giao diện đã hiện thực theo từng nhóm chức năng chính trong hệ thống.
Nhóm xác thực.

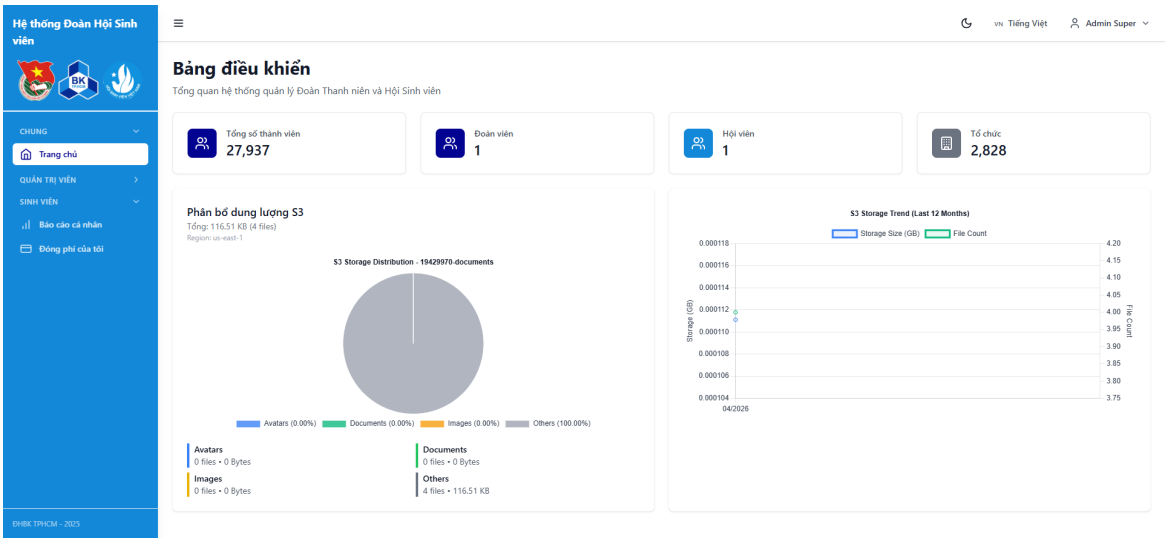


Hình 47: Giao diện đăng nhập hệ thống

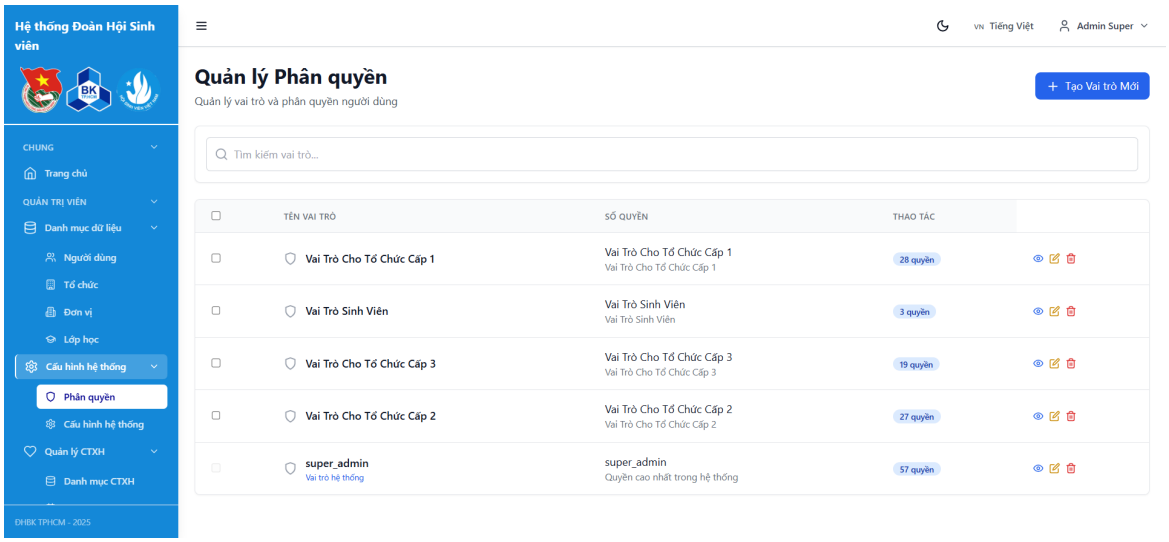


Hình 48: Giao diện đăng nhập quản trị viên

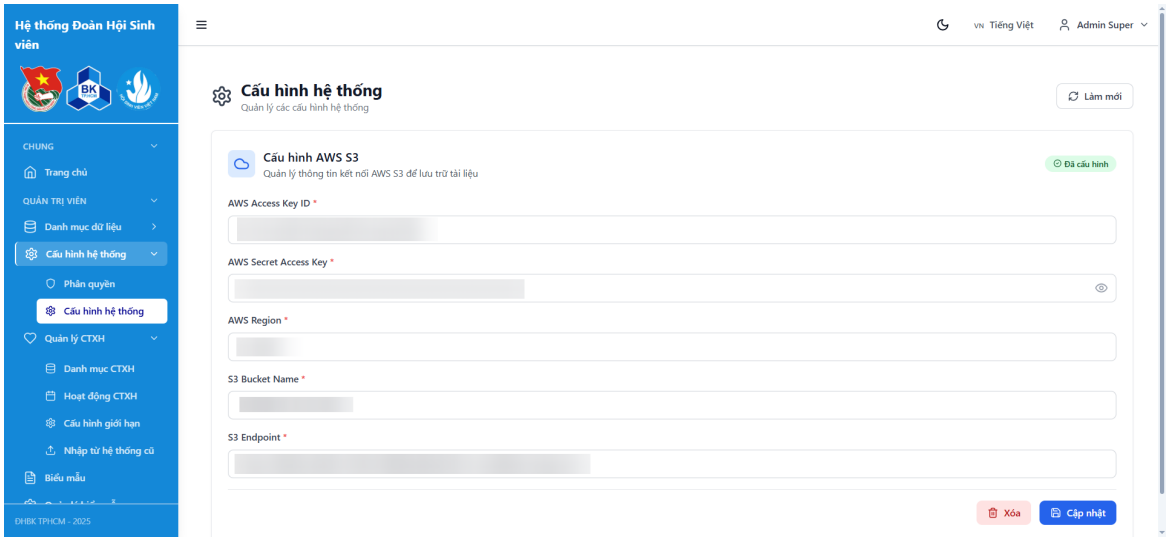
Nhóm dashboard và cấu hình hệ thống.



Hình 49: Giao diện dashboard tổng quan

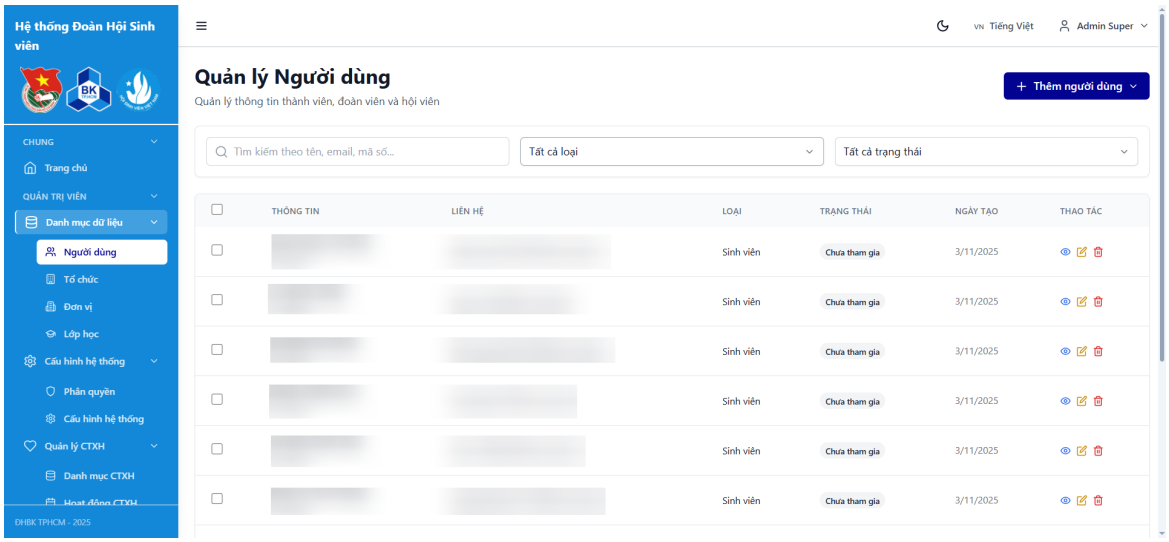


Hình 50: Màn hình quản lý phân quyền RBAC

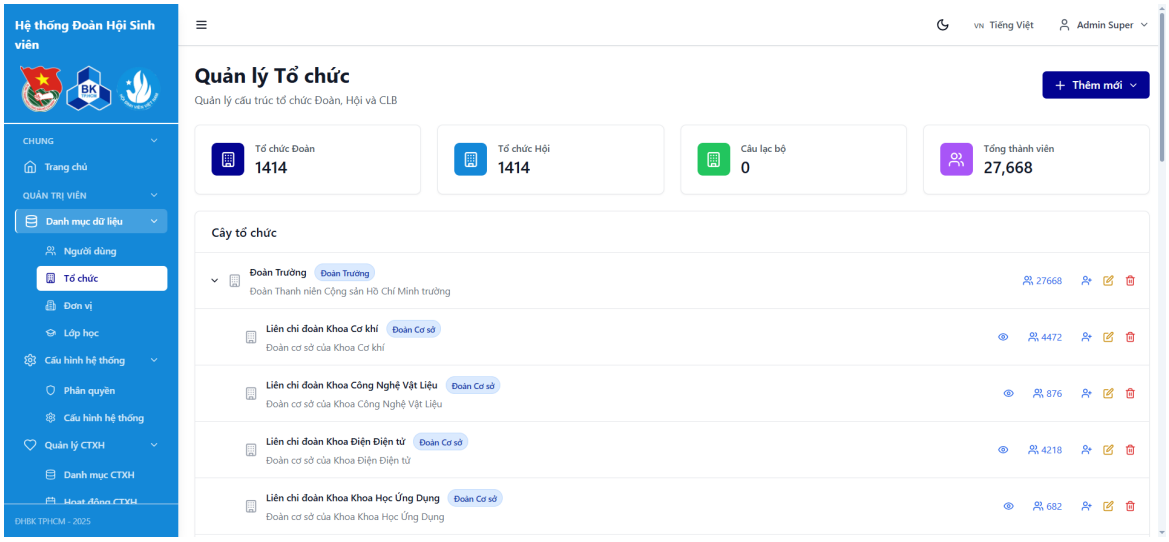


Hình 51: Màn hình cấu hình hệ thống S3

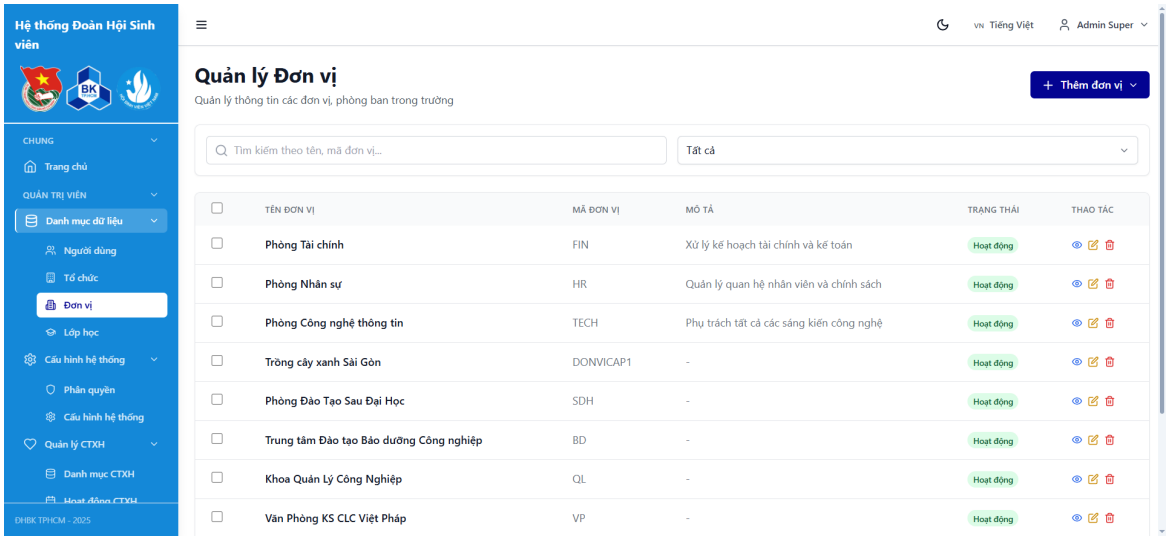
Nhóm quản trị dữ liệu nền.



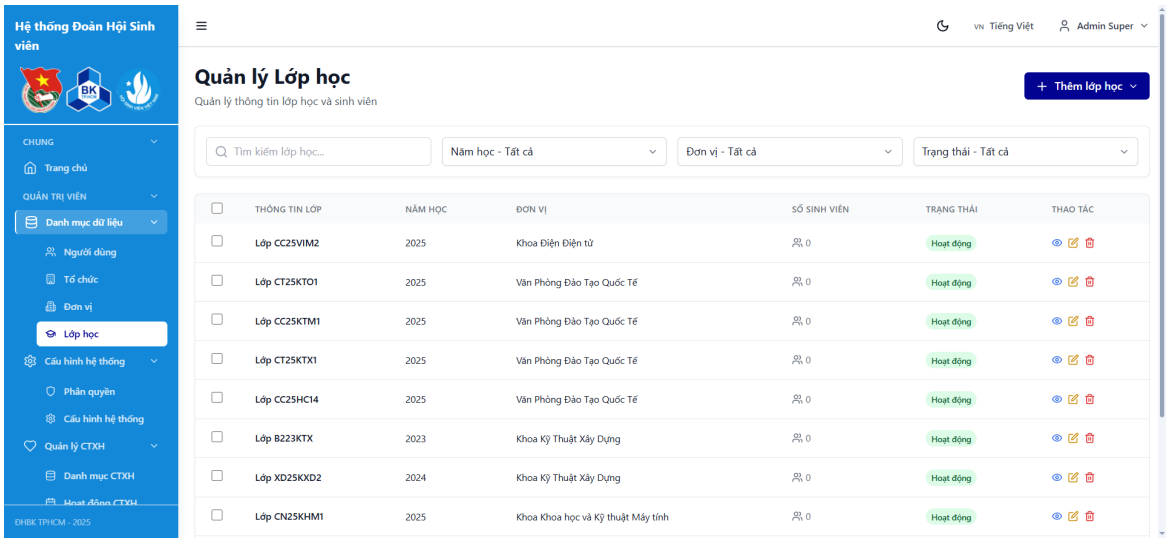
Hình 52: Giao diện quản lý người dùng



Hình 53: Giao diện quản lý tổ chức

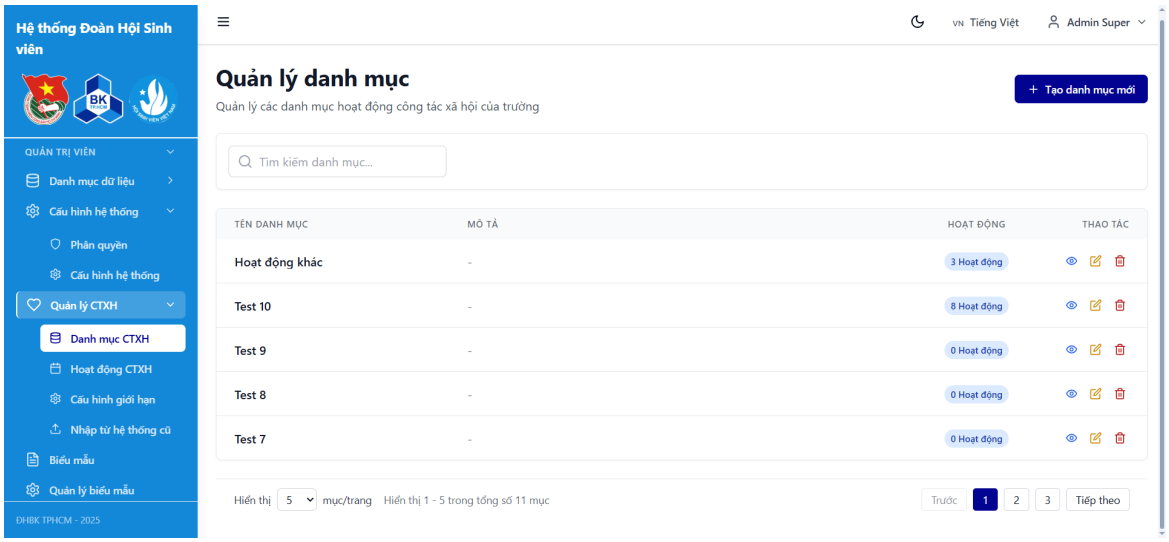


Hình 54: Giao diện quản lý đơn vị/khoa

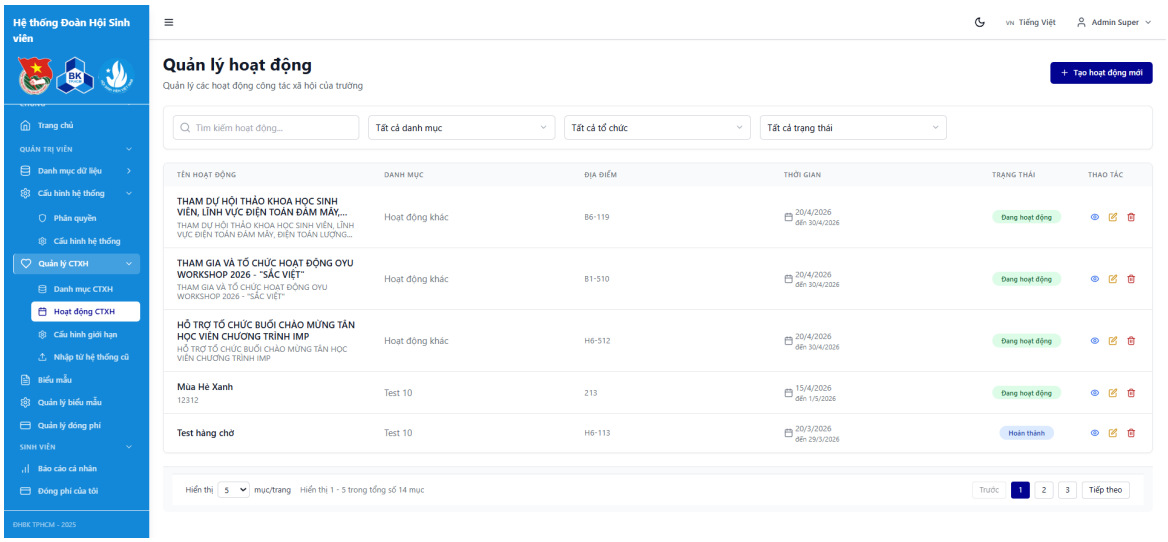


Hình 55: Giao diện quản lý lớp học

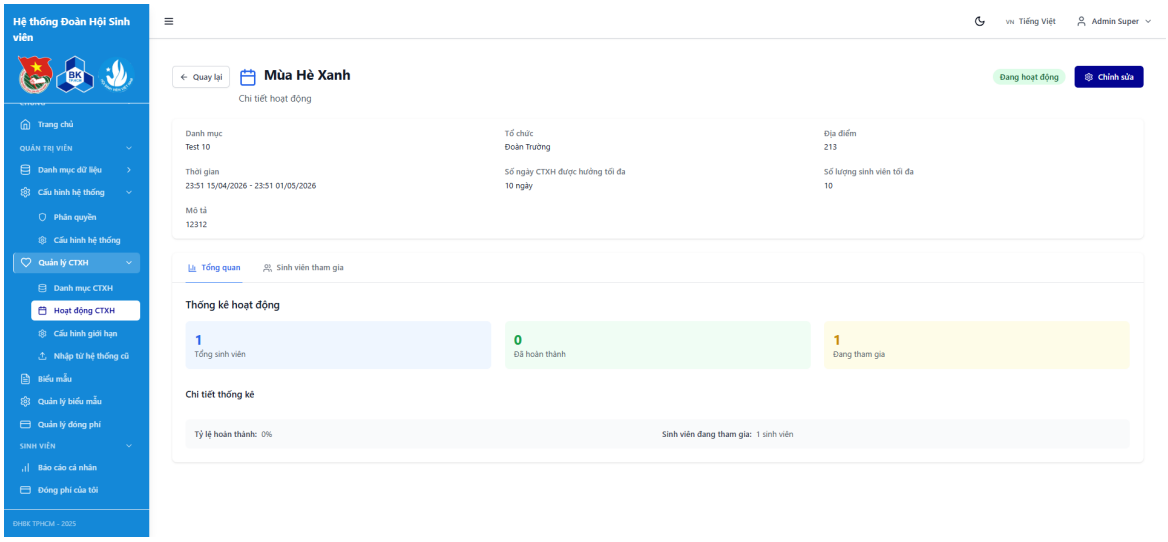
Nhóm quản lý công tác xã hội.



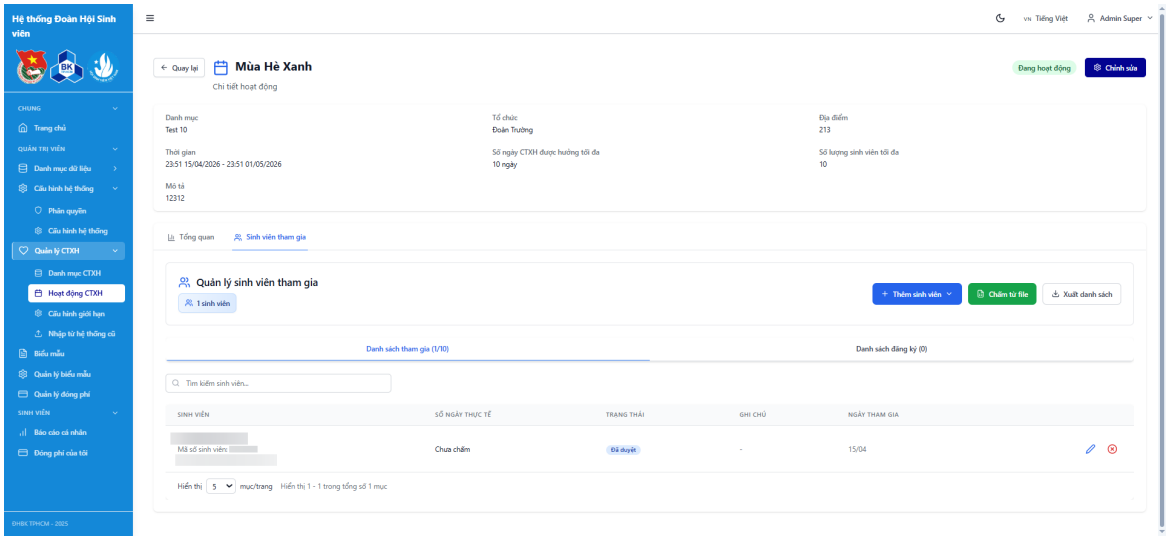
Hình 56: Giao diện quản lý danh mục CTXH



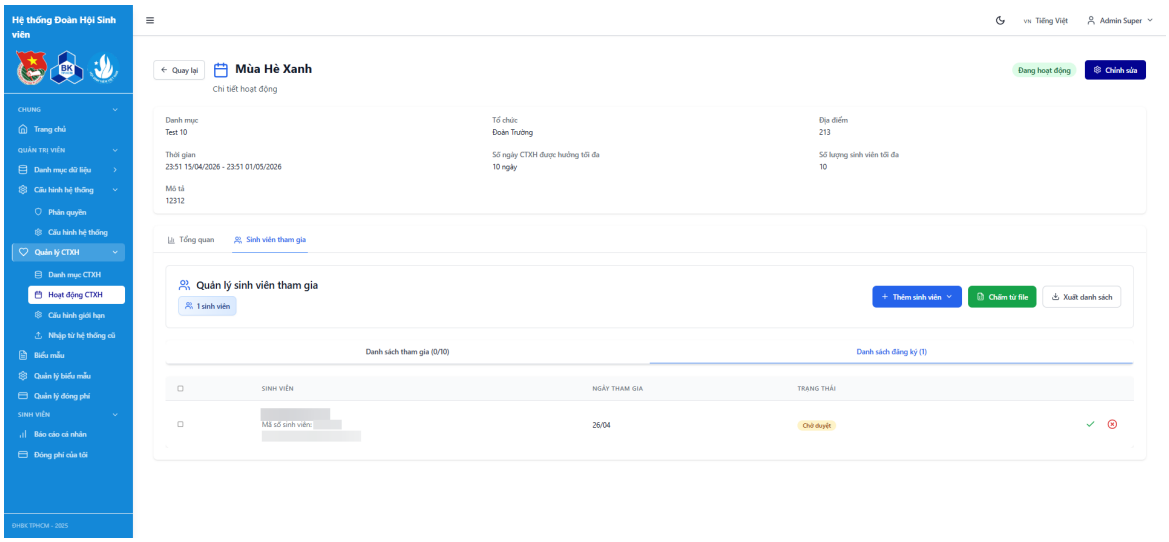
Hình 57: Giao diện quản lý hoạt động CTXH theo danh mục



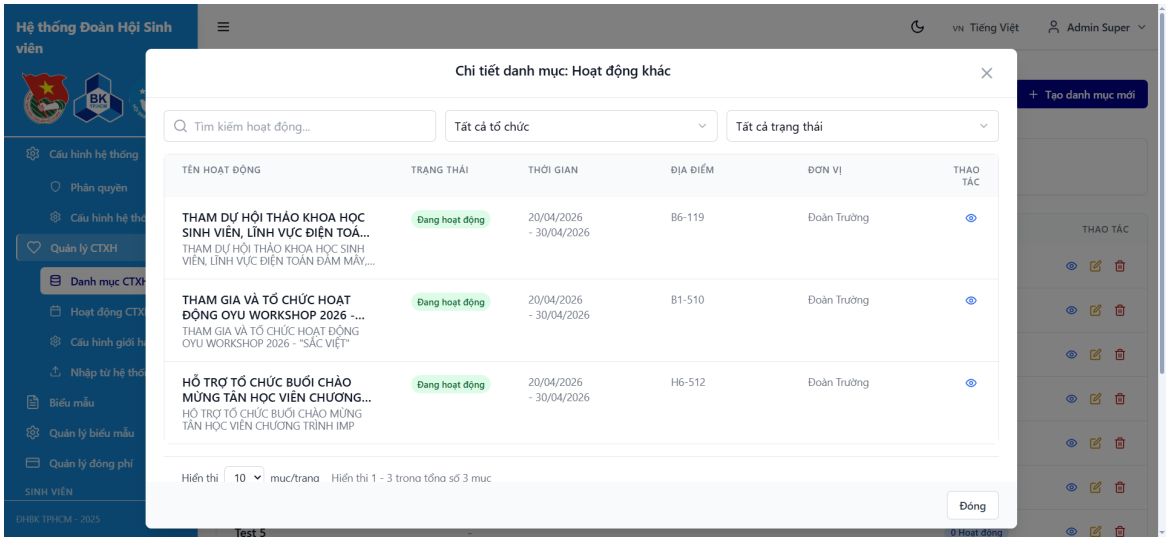
Hình 58: Giao diện chi tiết hoạt động (tab tổng quan)



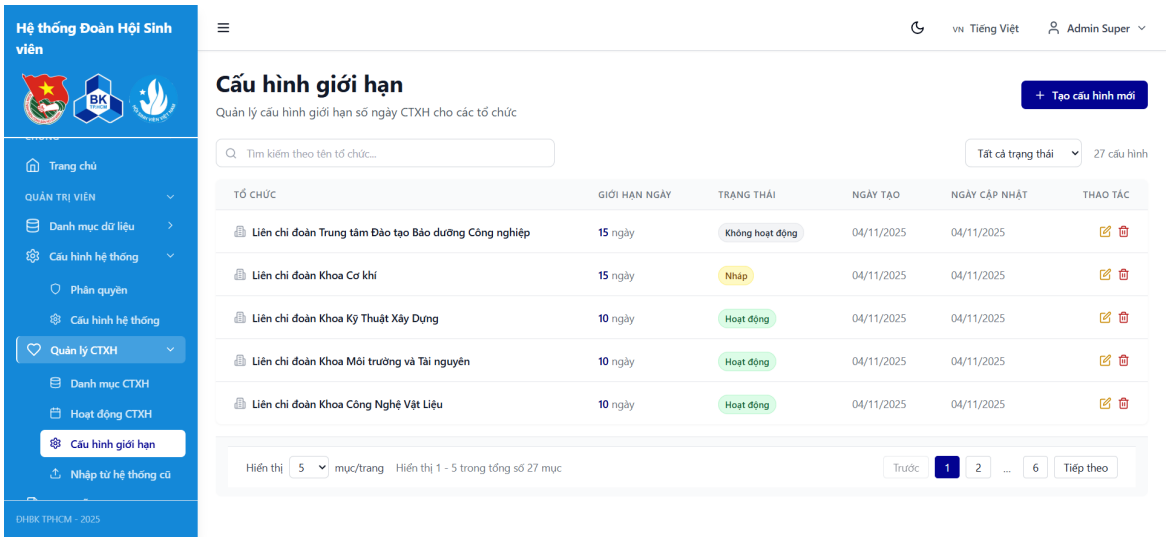
Hình 59: Giao diện danh sách sinh viên tham gia hoạt động CTXH



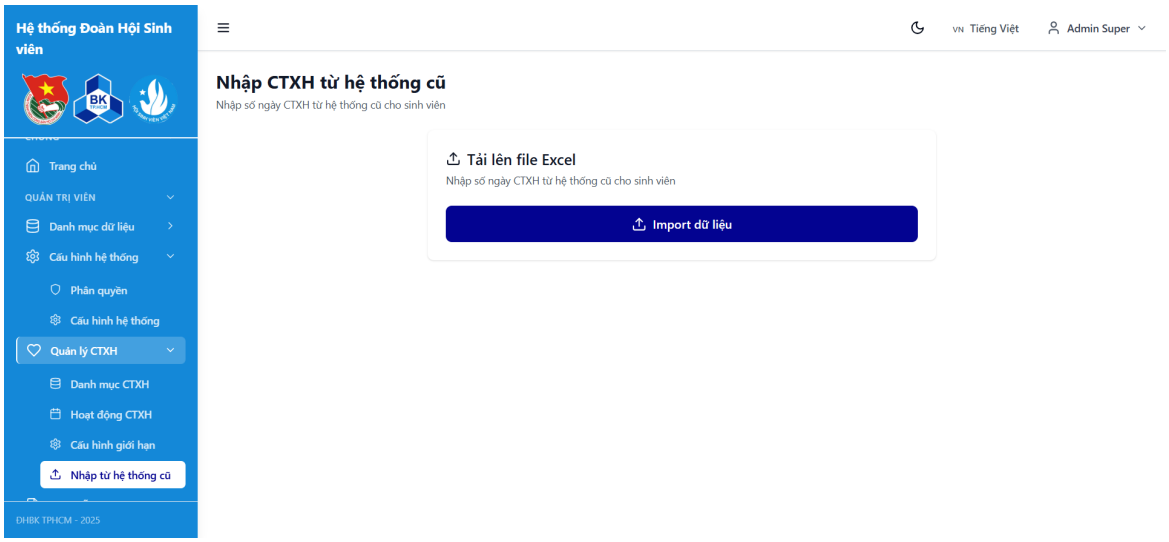
Hình 60: Giao diện danh sách đăng ký chờ duyệt của hoạt động CTXH



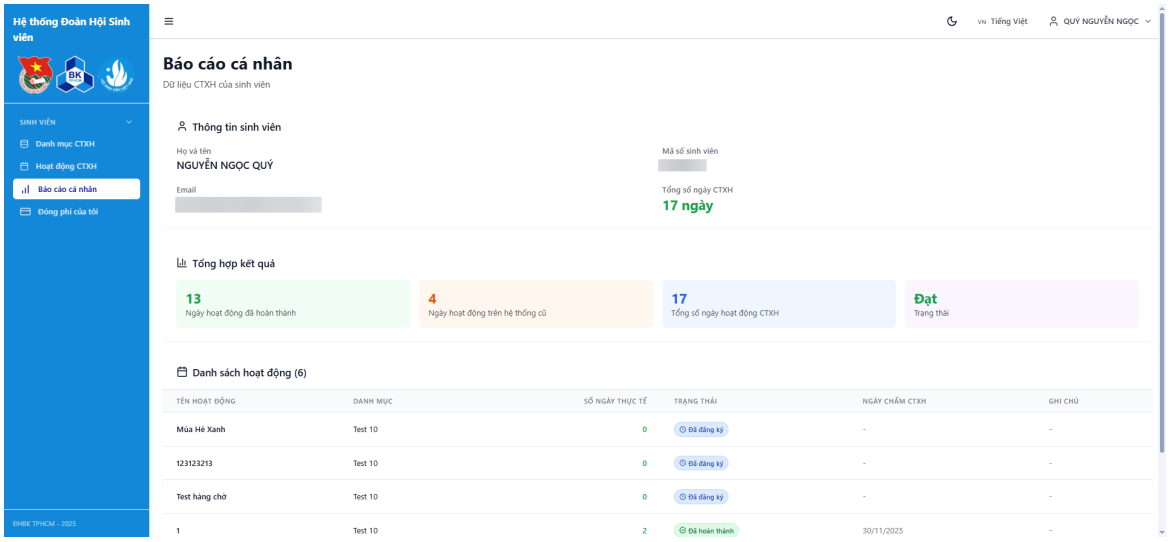
Hình 61: Màn hình xem chi tiết danh mục và danh sách hoạt động thuộc danh mục



Hình 62: Giao diện cấu hình giới hạn ngày CTXH

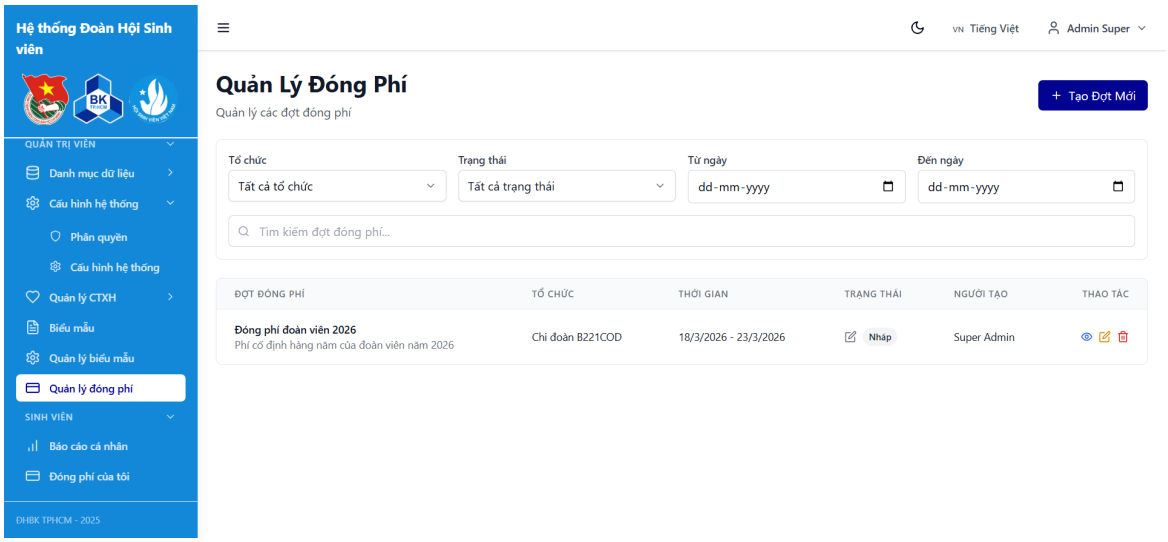


Hình 63: Giao diện nhập dữ liệu CTXH từ hệ thống cũ

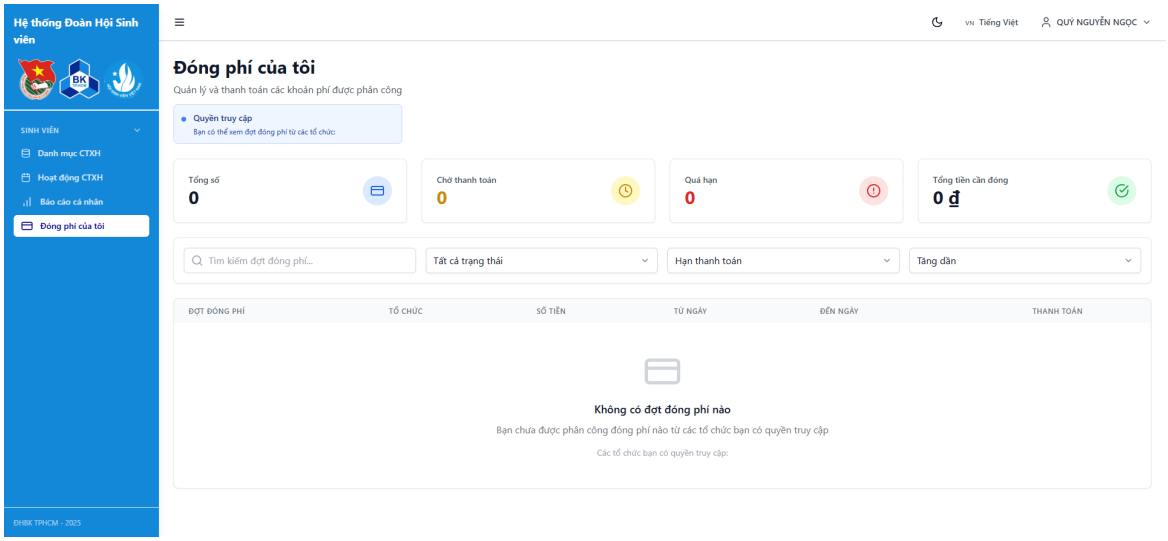


Hình 64: Giao diện báo cáo CTXH cá nhân của sinh viên

Nhóm thu phí.

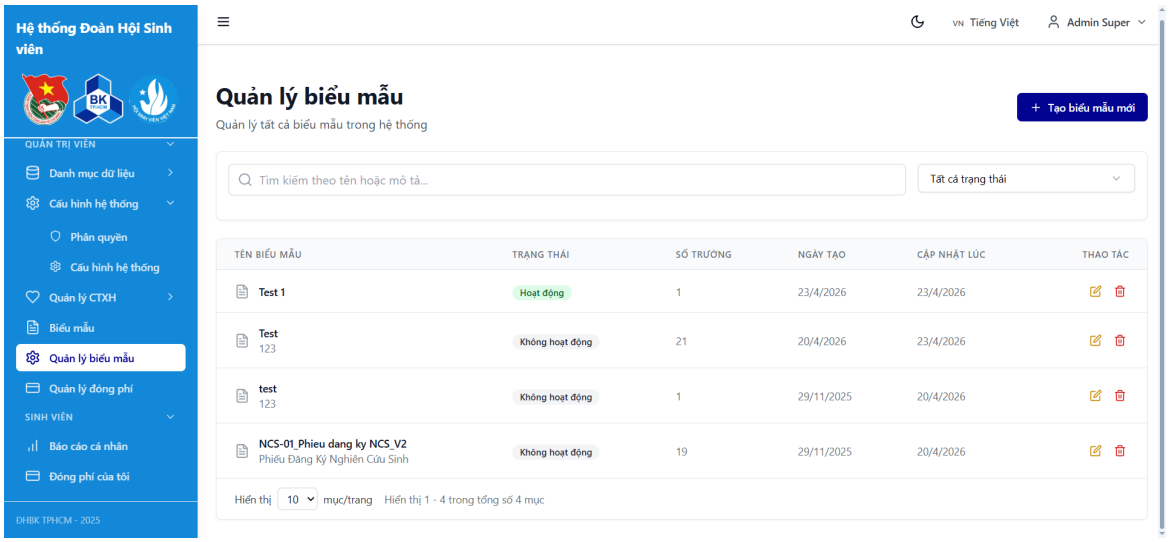


Hình 65: Giao diện quản lý đóng phí theo đợt

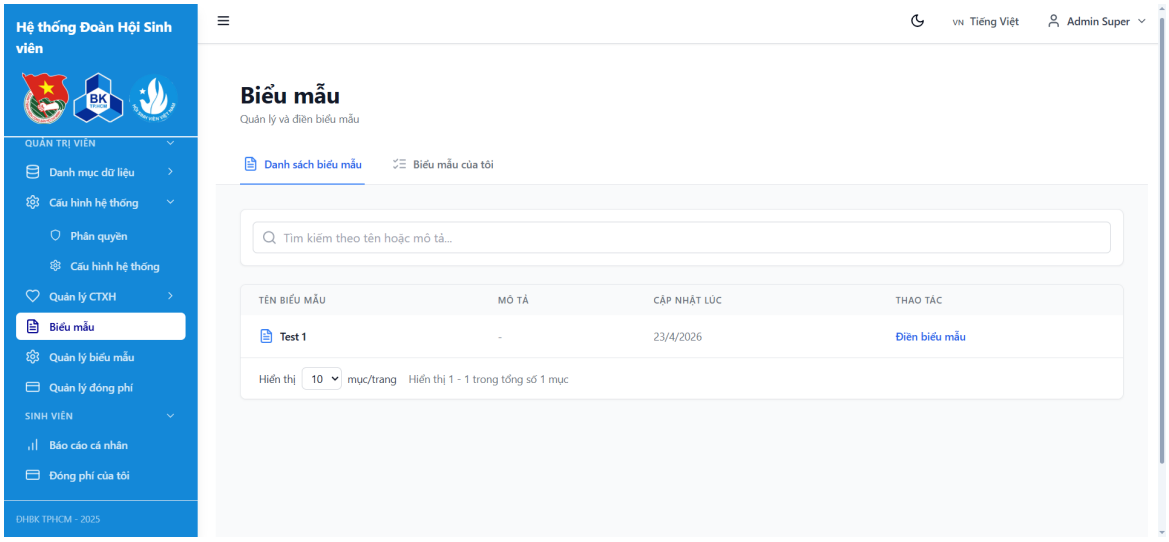


Hình 66: Giao diện đóng phí của sinh viên

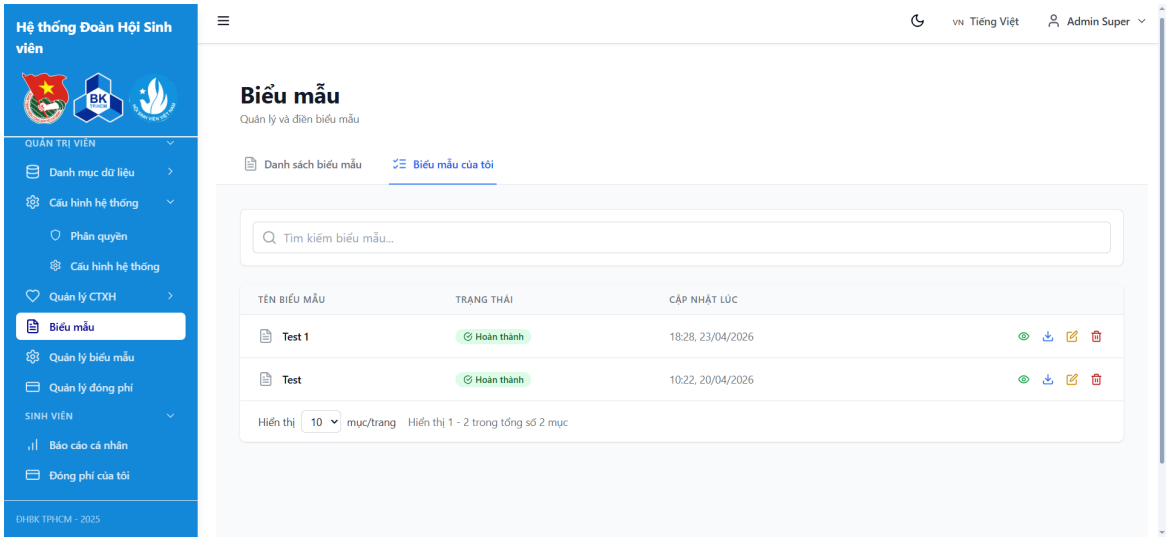
Nhóm biểu mẫu.



Hình 67: Giao diện quản lý biểu mẫu

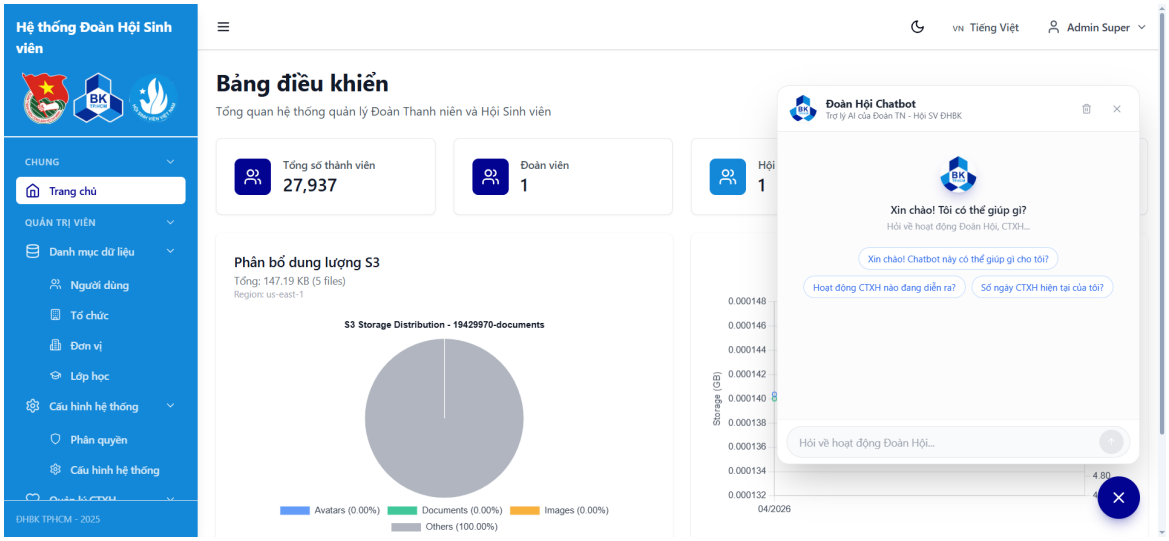


Hình 68: Giao diện danh sách biểu mẫu có thể điền



Hình 69: Giao diện theo dõi biểu mẫu của tôi

Nhóm chatbot.



Hình 70: Giao diện hội thoại chatbot CTXH

5.5.4 Hiện thực trải nghiệm cho tác vụ dữ liệu lớn

Các tác vụ nghiệp vụ có dữ liệu lớn được hiện thực với cơ chế giảm tải ở UI:

- Phân trang và tìm kiếm phía server cho danh sách lớn.
- Import theo batch kèm tiến trình và tổng kết lỗi.
- Caching truy vấn bằng React Query để hạn chế gọi API lặp.
- Các màn hình cây tổ chức dùng chiến lược lazy loading để giảm tải dữ liệu ban đầu.

5.5.5 Kết nối giữa giao diện và backend

Mỗi màn hình frontend được gắn với tập API rõ ràng theo module. Cách tổ chức này giúp truy vết lỗi nhanh theo chiều dọc (UI → API → Service → DB) và thuận lợi khi bảo trì theo từng miền nghiệp vụ.

Về mặt vận hành, giao diện được đóng gói build tĩnh và phục vụ qua Nginx; backend cung cấp API qua domain chung, hỗ trợ CORS theo môi trường và đồng nhất cơ chế xác thực JWT cho toàn hệ thống.

Chương 6 Kiểm thử

6.1 Kiểm thử Web

6.1.1 Mục tiêu kiểm thử

Mục tiêu của chương kiểm thử là xác nhận hệ thống Đoàn – Hội Portal đã đáp ứng đúng các yêu cầu nghiệp vụ cốt lõi trước khi triển khai chính thức. Nhóm tập trung vào ba lớp kiểm thử:

- **Kiểm thử chức năng giao diện web**: xác nhận các luồng thao tác chính hoạt động đúng với quyền người dùng.
- **Kiểm thử API/backend**: xác nhận hợp đồng API, phân quyền và dữ liệu trả về.
- **Kiểm thử tải cơ bản**: đánh giá phản hồi hệ thống dưới mức tải đồng thời ở quy mô triển khai luận văn.

6.1.2 Môi trường kiểm thử

Môi trường kiểm thử giao diện và API được chuẩn hóa để bảo đảm kết quả có thể tái lập:

- Hệ điều hành máy kiểm thử: Windows 11 23H2.
- Trình duyệt kiểm thử chính: Google Chrome 124.x.
- Frontend: React + Vite, chạy qua Nginx container.
- Backend: NestJS, chạy tại cổng dịch vụ nội bộ 9200.
- Cơ sở dữ liệu: PostgreSQL 15 có extension pgvector.
- Dịch vụ hỗ trợ: Redis (queue/cache), Prometheus, Grafana.

Các ca kiểm thử được thực hiện theo hướng *black-box*: thao tác từ UI/API và đối chiếu kết quả thực tế với kết quả mong đợi theo yêu cầu chức năng đã phân tích ở Chương 3.

6.1.3 Kiểm thử chức năng trang xác thực và phân quyền

Mã	Tiền điều kiện/Thao tác	Kết quả dự kiến	Kết quả
A1	Truy cập màn hình đăng nhập, nhập đúng tài khoản/mật khẩu nội bộ	Đăng nhập thành công, điều hướng về dashboard theo quyền	Đạt
A2	Nhập sai mật khẩu tài khoản hợp lệ	Hiển thị thông báo đăng nhập thất bại, không tạo phiên	Đạt
A3	Để trống tài khoản hoặc mật khẩu	Nút đăng nhập bị vô hiệu hóa hoặc backend trả lỗi validation	Đạt
A4	Đăng nhập bằng Google OAuth với tài khoản đã được cấp trong hệ thống	Đăng nhập thành công qua callback, nhận access token/refresh token	Đạt
A5	Đăng nhập bằng Google OAuth với tài khoản chưa tồn tại trong hệ thống	Chuyển về trang callback với trạng thái lỗi user_not_found	Đạt
A6	Người dùng đã đăng nhập truy cập màn hình có quyền hạn chế	Chỉ hiển thị menu/chức năng thuộc tập quyền được cấp	Đạt

Bảng 28: Bảng kiểm thử chức năng xác thực và phân quyền

6.1.4 Kiểm thử chức năng quản trị người dùng – tổ chức

Mã	Tiền điều kiện/Thao tác	Kết quả dự kiến	Kết quả
B1	Truy cập danh sách người dùng với tài khoản có quyền xem	Hiển thị danh sách có phân trang, lọc và tìm kiếm theo tiêu chí	Đạt
B2	Tạo người dùng mới và gán vai trò theo tổ chức	Người dùng được tạo thành công, vai trò được lưu đúng phạm vi tổ chức	Đạt
B3	Cập nhật thông tin người dùng hiện có	Dữ liệu hiển thị sau cập nhật phản ánh đúng giá trị mới	Đạt
B4	Xóa mềm người dùng không còn sử dụng	Bản ghi bị loại khỏi danh sách mặc định, không phát sinh lỗi quan hệ	Đạt
B5	Upload file Excel import users rồi validate session import	Trả về báo cáo lỗi/cảnh báo theo dòng, không ghi dữ liệu khi chưa execute	Đạt
B6	Execute import session hợp lệ	Dữ liệu người dùng được ghi theo batch, trả tổng hợp thành công/thất bại	Đạt
B7	Truy cập cây tổ chức (organization tree) theo maxDepth	Hiển thị đúng cấu trúc phân cấp và số lượng nút theo độ sâu	Đạt
B8	Truy vấn danh sách tổ chức accessible theo quyền người dùng	Chỉ trả tổ chức mà người dùng có quyền thao tác tương ứng	Đạt

Bảng 29: Bảng kiểm thử chức năng quản trị người dùng – tổ chức

6.1.5 Kiểm thử chức năng CTXH

Mã	Tiền điều kiện/Thao tác	Kết quả dự kiến	Kết quả
C1	Truy cập danh sách danh mục CTXH với quyền `ctxh.categories.view`	Hiển thị danh mục có phân trang/tìm kiếm đúng dữ liệu	Đạt
C2	Tạo mới danh mục CTXH	Danh mục mới xuất hiện trong danh sách, không trùng tên trong cùng phạm vi	Đạt
C3	Tạo hoạt động CTXH với thông tin thời gian, tổ chức, tổng ngày	Hoạt động được tạo và ở trạng thái khởi tạo hợp lệ	Đạt
C4	Cập nhật trạng thái hoạt động theo vòng đời nghiệp vụ	Hệ thống cho phép chuyển trạng thái hợp lệ, chặn chuyển trái quy tắc	Đạt
C5	Import dữ liệu CTXH từ hệ thống cũ: validate trước import	Trả về danh sách MSSV hợp lệ/không hợp lệ, chưa ghi DB ở bước validate	Đạt
C6	Thực thi import CTXH old-system sau khi validate đạt	Dữ liệu ngày CTXH cũ được cập nhật đúng trường mục tiêu	Đạt
C7	Truy vấn báo cáo CTXH theo MSSV	Trả về tổng hợp hoạt động và ngày CTXH đúng theo dữ liệu tham gia	Đạt
C8	Cấu hình giới hạn ngày CTXH theo tổ chức và truy vấn active config	Trả đúng cấu hình active theo tổ chức, hỗ trợ filter status	Đạt

Bảng 30: Bảng kiểm thử chức năng phân hệ CTXH

6.1.6 Kiểm thử chức năng thu phí và biểu mẫu

Mã	Tiền điều kiện/Thao tác	Kết quả dự kiến	Kết quả
D1	Tạo chiến dịch thu phí mới cho tổ chức hợp lệ	Chiến dịch được tạo với trạng thái khởi tạo và thông tin thời gian đúng	Đạt
D2	Tạo mức phí trong chiến dịch và gán danh sách đối tượng đóng phí	Sinh bản ghi assignment đúng số lượng, trạng thái ban đầu PENDING	Đạt
D3	Đồng bộ trạng thái thanh toán từ cổng thanh toán về hệ thống	Assignment và payment history được cập nhật nhất quán	Đạt
D4	Upload template Word `.docx` và parse placeholder	Trả danh sách placeholder, cho phép cấu hình field động	Đạt
D5	Người dùng tạo submission dạng nháp rồi submit	Sinh tài liệu đầu ra, cập nhật trạng thái submission thành công	Đạt
D6	Tải file đã sinh hoặc lấy public URL xem trực tuyến	Trả file đúng định dạng và metadata URL hợp lệ	Đạt
D7	Kiểm tra cấu hình S3 đang active và test kết nối cấu hình mới	Trả kết quả `success=true` với cấu hình hợp lệ	Đạt

Bảng 31: Bảng kiểm thử chức năng thu phí, biểu mẫu và lưu trữ

6.1.7 Kiểm thử chatbot RAG CTXH

Mã	Tiền điều kiện/Thao tác	Kết quả dự kiến	Kết quả
E1	Gửi câu hỏi CTXH qua endpoint `/api/ctxh-chat/message`	Nhận câu trả lời có nội dung liên quan và có danh sách nguồn tham chiếu	Đạt
E2	Gửi nhiều lượt hội thoại trong cùng `sessionId`	Chatbot giữ ngữ cảnh, phản hồi nhất quán giữa các lượt hỏi đáp	Đạt
E3	Truy vấn lịch sử qua `/api/ctxh-chat/history/:sessionId`	Trả đúng thứ tự message user/assistant và metadata phiên	Đạt
E4	Xóa phiên hội thoại qua `/api/ctxh-chat/session/:sessionId`	Phiên được dọn sạch, truy vấn lại lịch sử trả rỗng hoặc không còn dữ liệu	Đạt
E5	Kiểm thử streaming qua SSE	Luồng phản hồi trả từng phần liên tục và kết thúc bằng sự kiện hoàn tất	Đạt

Bảng 32: Bảng kiểm thử chức năng chatbot RAG

6.2 Kiểm thử API và Backend

6.2.1 Kịch bản kiểm thử API

Nhóm kiểm thử API bằng Postman Collection của dự án, kết hợp kiểm tra thủ công các ca quan trọng theo ba nhóm:

- **Nhóm xác thực và bảo mật**: login, refresh token, logout, truy cập tài nguyên có bảo vệ JWT.
- **Nhóm nghiệp vụ**: users, organizations, CTXH, payment, templates/submissions, export.
- **Nhóm vận hành**: metrics endpoint, kiểm tra kết nối S3, trạng thái import session, chatbot health/ingestion.

Các tiêu chí kiểm thử API gồm: mã trạng thái HTTP đúng chuẩn, schema response đúng hợp đồng, xử lý lỗi rõ ràng, và ràng buộc phân quyền đúng theo permission code.

6.2.2 Bảng kiểm thử API đại diện

Mã	API	Phương thức	Kết quả kỳ vọng	KQ
F1	`/auth/login`	POST	Trả access token, refresh token và thông tin user khi credential hợp lệ	Đạt
F2	`/auth/me/permissions`	GET	Trả danh sách quyền theo organization context nếu có truyền tham số	Đạt
F3	`/users/import/upload`	POST	Nhận file thành công, tạo session import và metadata tương ứng	Đạt
F4	`/organizations/tree`	GET	Trả cây tổ chức đúng cấu trúc, hỗ trợ maxDepth/type/code	Đạt
F5	`/ctxh/import/old-system/validate`	POST	Trả báo cáo validation theo từng bản ghi đầu vào	Đạt
F6	`/ctxh/reports/student/:studentCode`	GET	Trả báo cáo tổng hợp CTXH của sinh viên theo MSSV	Đạt
F7	`/templates/upload`	POST	Upload template `.docx` thành công, lưu metadata mẫu	Đạt
F8	`/submissions/:id/submit`	POST	Sinh file đầu ra, cập nhật trạng thái submission và đường dẫn file	Đạt
F9	`/api/ctxh-chat/message`	POST	Trả câu trả lời chatbot, có sources và responseTime	Đạt
F10	`/aws-s3-config/:id/test`	POST	Trả `success=true` với cấu hình hợp lệ, báo lỗi khi sai credentials	Đạt

Bảng 33: Bảng kiểm thử API backend đại diện

6.2.3 Kiểm thử tải cơ bản cho backend

Để đánh giá sơ bộ khả năng chịu tải, nhóm sử dụng Postman Performance Runner trên tập endpoint đại diện, với cấu hình thử nghiệm:

- Virtual Users: 80.
- Test Duration: 5 phút/collection.
- Load Profile: Fixed.
- Hạ tầng test: backend container + PostgreSQL + Redis trong cùng mạng Docker.

Collection	Thời gian	Tổng request	Request/s	RT trung bình (ms)	Tỷ lệ lỗi
Auth & RBAC	5 phút	8.964	29,88	286	0,00%
Users & Organizations	5 phút	11.427	38,09	344	0,09%
CTXH	5 phút	10.286	34,29	337	0,12%
Payment	5 phút	7.918	26,39	391	0,18%
Templates & Submissions	5 phút	6.305	21,02	418	0,16%
Ctxh-chat (non-stream)	5 phút	4.732	15,77	682	0,25%

Bảng 34: Bảng số liệu kiểm thử tải cơ bản các collection API

Kết quả cho thấy các endpoint CRUD và truy vấn dữ liệu giao dịch duy trì thời gian phản hồi ổn định ở mức dưới 450 ms trong điều kiện tải kiểm thử; nhóm chatbot có độ trễ cao hơn do bao gồm retrieval + generation, phù hợp với đặc thù xử lý AI.

6.2.4 Tổng hợp kết quả kiểm thử

Từ các nhóm kiểm thử trên, có thể rút ra các nhận xét chính:

- Hệ thống đáp ứng đầy đủ các chức năng lõi đã đặc tả trong phạm vi luận văn.
- Cơ chế phân quyền theo vai trò và tổ chức hoạt động nhất quán giữa UI và API.
- Các luồng dữ liệu lớn (import, sinh tài liệu, đồng bộ chatbot) vận hành ổn định với cơ chế kiểm soát lỗi theo phiên/lô.
- Hệ thống đạt mức sẵn sàng để triển khai vận hành thực tế ở quy mô người dùng mục tiêu của đề tài.

Chương 7 Triển khai

7.1 Triển khai hệ thống

7.1.1 Mục tiêu triển khai

Mục tiêu triển khai của đề tài là đưa hệ thống từ môi trường phát triển sang môi trường có thể vận hành ổn định, dễ giám sát và dễ mở rộng theo nhu cầu thực tế của đơn vị sử dụng. Các nguyên tắc triển khai được áp dụng gồm:

- Tách biệt frontend, backend và các dịch vụ dữ liệu theo container độc lập.
- Quản lý cấu hình theo biến môi trường để chuyển đổi giữa môi trường phát triển và production.
- Bổ sung lớp reverse proxy và quan sát hệ thống ngay từ giai đoạn đầu vận hành.

7.1.2 Kiến trúc triển khai thực tế

Hệ thống được đóng gói và triển khai theo mô hình container hóa với `docker-compose`. Ở môi trường production, các dịch vụ chính gồm:

- **Frontend**: ứng dụng React build tĩnh, phục vụ qua Nginx container.
- **Backend**: dịch vụ NestJS API chạy ở cổng nội bộ 9200 (map host 2681).
- **PostgreSQL + pgvector**: lưu dữ liệu giao dịch và dữ liệu vector.
- **Prometheus**: thu thập metrics từ backend và hệ thống giám sát.
- **Grafana**: trực quan hóa metrics và dashboard vận hành.

Trong môi trường phát triển, hệ thống mở rộng thêm `pgAdmin`, `Redis`, `Ollama` để hỗ trợ kiểm thử dữ liệu, xử lý hàng đợi và thử nghiệm module chatbot cục bộ.

7.1.3 Cấu hình reverse proxy và tên miền

Lớp Nginx ngoài cùng chịu trách nhiệm định tuyến request theo đường dẫn chức năng:

- `/` chuyển tiếp đến frontend.
- `/api/` chuyển tiếp đến backend API.
- `/grafana/` chuyển tiếp đến Grafana.
- `/prometheus/` chuyển tiếp đến Prometheus.

Cấu hình này giúp gom toàn bộ dịch vụ về một domain thống nhất, đồng thời vẫn tách rõ lớp ứng dụng và lớp quan sát vận hành.

7.1.4 Quy trình triển khai môi trường production

Quy trình triển khai được chuẩn hóa theo các bước:
Bước 1 – Chuẩn bị máy chủ và công cụ vận hành

- Cài đặt Docker Engine, Docker Compose Plugin, Git.
- Mở các cổng cần thiết cho reverse proxy và dịch vụ nội bộ theo chính sách mạng của đơn vị triển khai.

Bước 2 – Lấy mã nguồn và chuẩn bị biến môi trường

- Clone repository lên máy chủ triển khai.
- Tạo file `.env` chứa thông tin nhạy cảm: mật khẩu DB, JWT secrets, OAuth keys, cấu hình PayOS, cấu hình domain frontend/backend.

Bước 3 – Khởi tạo stack production

- Dùng file `docker-compose.prod.yml` để khởi chạy các dịch vụ production.
- Kiểm tra healthcheck PostgreSQL và trạng thái backend sau khi phụ thuộc sẵn sàng.

Bước 4 – Cấu hình Nginx và HTTPS

- Cấu hình server block cho domain triển khai.
- Cập nhật đường dẫn chứng chỉ SSL, bật chuyển hướng HTTP sang HTTPS.
- Cấu hình timeout và header proxy phù hợp cho API và dashboard giám sát.

Bước 5 – Kiểm tra sau triển khai

- Kiểm tra truy cập frontend và đăng nhập hệ thống.
- Kiểm tra `/api/health` hoặc endpoint tương đương của backend.
- Kiểm tra trang `/grafana/` và `/prometheus/` hoạt động đúng subpath.

7.1.5 Quy trình cập nhật phiên bản

Khi có phiên bản mới, quy trình cập nhật được thực hiện theo hướng giảm thời gian gián đoạn:

1. Cập nhật mã nguồn hoặc image version theo tag phát hành.
2. Thực hiện backup dữ liệu PostgreSQL trước khi nâng cấp.
3. Triển khai lại stack bằng `docker compose up -d`.
4. Chạy kiểm thử smoke test cho các chức năng trọng yếu (đăng nhập, dashboard, CTXH, payment, chatbot).
5. Theo dõi metrics 15–30 phút đầu sau cập nhật để phát hiện hồi quy hiệu năng.

7.1.6 Các lưu ý vận hành và an toàn

Trong quá trình vận hành thực tế, nhóm áp dụng các lưu ý sau để giảm rủi ro:

- Không hard-code secrets trong mã nguồn; toàn bộ secrets đặt ở `.env` hoặc secret manager.
- Tách tài khoản quản trị cho DB, Grafana và ứng dụng; đổi mật khẩu mặc định trước khi đưa vào production.
- Giới hạn quyền truy cập các route quan sát hệ thống theo IP/VPN nội bộ khi cần.
- Thiết lập backup định kỳ cho dữ liệu giao dịch và file tài liệu sinh từ module biểu mẫu.

7.2 Đánh giá hệ thống

7.2.1 Đánh giá theo tiêu chí chức năng

Dựa trên kết quả kiểm thử ở Chương 6, các phân hệ chính (xác thực – phân quyền, quản trị người dùng/tổ chức, CTXH, thu phí, biểu mẫu, chatbot) đều đạt yêu cầu chức năng trong phạm vi bài toán. Cụ thể:

- Luồng nghiệp vụ chính vận hành đúng với đặc tả, không phát hiện lỗi chặn nghiệp vụ trong ca kiểm thử chuẩn.
- Cơ chế phân quyền theo ngữ cảnh tổ chức thể hiện rõ tính phù hợp với mô hình Đoàn – Hội đa cấp.
- Tính năng import và xử lý dữ liệu lớn hoạt động ổn định nhờ chia lô và cơ chế validate trước khi ghi.

7.2.2 Đánh giá theo tiêu chí hiệu năng

Kết quả kiểm thử tải cơ bản cho thấy:

- Nhóm API giao dịch thông thường giữ thời gian phản hồi trung bình ở mức phù hợp cho tác vụ quản trị thường ngày.
- Nhóm API chatbot có độ trễ cao hơn do đặc thù retrieval + generation, nhưng vẫn đảm bảo trải nghiệm hội thoại trong ngưỡng chấp nhận được.
- Tỷ lệ lỗi tổng thể thấp trong các phiên chạy tải cố định 5 phút/collection.

Điều này cho thấy kiến trúc hiện tại phù hợp với quy mô triển khai mục tiêu của luận văn, đồng thời có dư địa tối ưu cho giai đoạn mở rộng người dùng.

7.2.3 Đánh giá theo tiêu chí vận hành

Hệ thống đạt mức sẵn sàng vận hành nhờ các yếu tố:

- Có pipeline triển khai rõ ràng theo container hóa.
- Có lớp giám sát Prometheus – Grafana hỗ trợ theo dõi tài nguyên và chỉ số ứng dụng.
- Có khả năng cập nhật cấu hình hạ tầng linh hoạt (S3 config, domain routing, biến môi trường).

Tiêu chí	Kết quả đánh giá	Mức độ
Độ đầy đủ chức năng	Bao phủ đầy đủ các phân hệ nghiệp vụ cốt lõi của cổng thông tin Đoàn – Hội	Tốt
Tính đúng của phân quyền	RBAC theo tổ chức hoạt động đúng ở cả tầng API và tầng giao diện	Tốt
Hiệu năng API giao dịch	Thời gian phản hồi trung bình ổn định với tải kiểm thử cơ bản	Tốt
Ổn định chatbot RAG	Trả lời có nguồn tham chiếu, hỗ trợ session và streaming	Khá
Khả năng quan sát hệ thống	Có dashboard metrics và luồng giám sát phục vụ vận hành	Tốt
Khả năng mở rộng hạ tầng	Có thể mở rộng ngang theo container và tách dịch vụ khi cần	Khá

Bảng 35: Tổng hợp đánh giá hệ thống sau triển khai

7.2.4 Nhận xét tổng quát

Qua quá trình triển khai và đánh giá, hệ thống đạt được mục tiêu đề tài ở cả ba khía cạnh: đúng nghiệp vụ, khả thi kỹ thuật và sẵn sàng vận hành. Tuy nhiên, ở giai đoạn mở rộng quy mô lớn hơn, cần tiếp tục đầu tư cho tối ưu hiệu năng truy vấn vector, chiến lược cache cho các màn hình dữ liệu lớn và tự động hóa sâu hơn quy trình CI/CD.

Chương 8 Tổng kết

8.1 Đánh giá kết quả đạt được

8.1.1 Đối với mục tiêu đề tài

Đề tài đặt mục tiêu xây dựng một cổng thông tin Đoàn – Hội thống nhất, hỗ trợ số hóa quy trình quản trị tổ chức, quản lý hoạt động CTXH, quản lý nghĩa vụ tài chính và tự động hóa xử lý biểu mẫu. Qua quá trình thực hiện, nhóm đã hoàn thành các mục tiêu trọng tâm sau:

- Xây dựng hệ thống xác thực và phân quyền theo vai trò, có xét ngữ cảnh tổ chức đa cấp.
- Hiện thực đầy đủ các phân hệ nghiệp vụ lõi: người dùng, tổ chức, khoa/lớp, CTXH, thu phí, biểu mẫu.
- Tích hợp chatbot RAG cho tra cứu CTXH bằng ngôn ngữ tự nhiên, có nguồn tham chiếu.
- Triển khai hệ thống theo kiến trúc container hóa, có thành phần giám sát vận hành.

Các kết quả này cho thấy đề tài không dừng ở mức mô phỏng mà đã tiến tới mức hệ thống có thể vận hành thực tế trong phạm vi nghiệp vụ nhà trường.

8.1.2 Đối với kiến thức và công nghệ

Trong quá trình thực hiện, nhóm đã vận dụng và củng cố năng lực ở nhiều lớp công nghệ:

- **Backend**: NestJS, TypeORM, PostgreSQL, pgvector, JWT/OAuth, queue và xử lý batch.
- **Frontend**: React, TypeScript, Vite, React Query, tổ chức mã nguồn theo feature.
- **Hạ tầng**: Docker Compose, Nginx reverse proxy, Prometheus, Grafana.
- **AI ứng dụng**: embedding, retrieval, prompt orchestration và quản lý hội thoại theo session.

Việc kết hợp giữa công nghệ web truyền thống và lớp AI truy xuất tri thức giúp đề tài có chiều sâu kỹ thuật, đồng thời phản ánh xu hướng chuyển đổi số có tích hợp trí tuệ nhân tạo trong bài toán quản trị giáo dục.

8.1.3 Đối với quy trình phát triển phần mềm

Nhóm triển khai đề tài theo chu trình phát triển tương đối đầy đủ:

1. Khảo sát nghiệp vụ và phân tích yêu cầu.
2. Thiết kế kiến trúc tổng thể và thiết kế dữ liệu.
3. Hiện thực theo module và kiểm thử từng phần.
4. Tích hợp liên module, kiểm thử toàn hệ thống.
5. Triển khai thử nghiệm và đánh giá sau triển khai.

Kết quả thực hiện cho thấy cách tiếp cận module hóa và kiểm thử theo luồng nghiệp vụ giúp kiểm soát rủi ro tốt hơn so với phát triển dàn trải, đặc biệt trong bối cảnh hệ thống có nhiều phân hệ liên quan dữ liệu chéo.

8.1.4 Đối với giá trị thực tiễn

Hệ thống đem lại một số giá trị thực tiễn rõ rệt:

- Giảm thao tác thủ công trong quản lý thành viên, tổ chức và tổng hợp dữ liệu.
- Chuẩn hóa quy trình ghi nhận hoạt động CTXH và truy xuất báo cáo theo mã sinh viên.
- Tăng tính minh bạch trong quản lý nghĩa vụ đóng phí nhờ mô hình campaign – assignment – history.
- Rút ngắn thời gian xử lý biểu mẫu với cơ chế sinh tài liệu tự động.
- Hỗ trợ người dùng tra cứu thông tin nhanh hơn qua chatbot RAG.

8.1.5 Đánh giá định lượng theo dữ liệu nội bộ

Do hệ thống chưa đưa vào vận hành production, phần đánh giá định lượng trong chương này chỉ sử dụng số liệu nội bộ từ quy trình chạy thử, kiểm thử chức năng và log vận hành thử nghiệm. Bảng **??** tổng hợp một số tiêu chí so sánh tiêu biểu.

Bảng 36: So sánh định lượng trước và sau số hóa theo dữ liệu nội bộ

Tiêu chí	**Trước số hóa**	**Sau số hóa**
Thời gian xử lý một quy trình nghiệp vụ chính	Phụ thuộc nhập tay, tổng hợp thủ công, mất nhiều bước đối chiếu	Rút ngắn nhờ số hóa quy trình và tự động hóa một phần thao tác
Số bước thao tác đối với CTXH, thu phí, biểu mẫu	Nhiều bước rời rạc giữa các khâu	Giảm số bước nhờ gom luồng xử lý trên một hệ thống thống nhất
Lỗi nhập liệu / lỗi đối chiếu dữ liệu	Dễ phát sinh khi nhập và tổng hợp thủ công	Giảm nhờ kiểm tra hợp lệ, ràng buộc dữ liệu và luồng import có kiểm soát

Kết quả trên cho thấy giá trị chính của hệ thống nằm ở việc giảm thao tác thủ công, tăng tính nhất quán dữ liệu và hỗ trợ tra cứu nhanh hơn trong các quy trình nghiệp vụ trọng tâm.

8.2 Những thuận lợi và khó khăn trong việc thực hiện đề tài

8.2.1 Thuận lợi

Trong quá trình thực hiện, nhóm có các thuận lợi chính:

- Bài toán có nhu cầu thực tiễn cao, giúp định hướng ưu tiên chức năng tương đối rõ.
- Hệ sinh thái công nghệ sử dụng (NestJS, React, PostgreSQL, Docker) có tài liệu phong phú và cộng đồng lớn.
- Cấu trúc nhóm theo module nghiệp vụ giúp chia việc song song hiệu quả.
- Việc áp dụng giám sát Prometheus – Grafana sớm giúp phát hiện và xử lý điểm nghẽn nhanh hơn khi tích hợp hệ thống.

8.2.2 Khó khăn

Bên cạnh thuận lợi, đề tài gặp các khó khăn tiêu biểu:

- **Độ phức tạp nghiệp vụ cao**: mô hình tổ chức đa cấp và phân quyền theo phạm vi khiến thiết kế dữ liệu và guard phức tạp hơn.

- **Dữ liệu lịch sử không đồng nhất**: quá trình import từ nguồn cũ cần nhiều bước validate, làm sạch và chuẩn hóa.
- **Tích hợp liên miền nghiệp vụ**: CTXH, payment, templates và chatbot có quan hệ dữ liệu chéo, đòi hỏi kiểm thử tích hợp kỹ.
- **Cân bằng hiệu năng và chi phí**: chatbot RAG yêu cầu thêm tài nguyên xử lý, cần tối ưu liên tục để giữ chi phí vận hành hợp lý.

8.2.3 Bài học kinh nghiệm

Qua quá trình triển khai, nhóm rút ra các kinh nghiệm quan trọng:

- Cần chuẩn hóa dữ liệu và quy ước đặt mã nghiệp vụ ngay từ đầu để giảm chi phí tích hợp về sau.
- Với hệ thống có phân quyền phức tạp, nên đặc tả permission matrix sớm và dùng test case bám theo matrix đó.
- Với dữ liệu lớn, luôn tách bước validate trước execute và thiết kế theo batch có khả năng retry.
- Với thành phần AI, cần cơ chế giám sát chất lượng trả lời và truy vết nguồn, không chỉ đo đúng/sai ở mức API.

8.3 Hướng phát triển trong tương lai

8.3.1 Mở rộng chức năng nghiệp vụ

Trong các phiên bản tiếp theo, hệ thống có thể mở rộng theo các hướng:

- Bổ sung workflow phê duyệt nhiều cấp cho hoạt động CTXH và biểu mẫu.
- Mở rộng dashboard điều hành theo thời gian thực với chỉ số cảnh báo chủ động.
- Bổ sung kênh thông báo đa phương thức (email, app push, OA) cho các sự kiện quan trọng.

8.3.2 Nâng cao chất lượng chatbot RAG

Để tăng chất lượng trợ lý tra cứu, nhóm đề xuất:

- Mở rộng nguồn tri thức ngoài dữ liệu hoạt động (quy chế, hướng dẫn, FAQ chính thức).
- Áp dụng reranking và đánh giá tự động theo bộ testset chuẩn hóa theo nghiệp vụ.
- Bổ sung cơ chế phản hồi người dùng (feedback loop) để hiệu chỉnh retrieval/prompt theo thời gian.

8.3.3 Nâng cấp hạ tầng và quy trình vận hành

Ở góc độ vận hành, các hướng phát triển ưu tiên gồm:

- Tự động hóa CI/CD đầy đủ kèm quality gates (test, lint, bảo mật).
- Chuẩn hóa cơ chế backup/restore và diễn tập phương án khôi phục sự cố.
- Triển khai cảnh báo vận hành theo ngưỡng tài nguyên, độ trễ API và tỷ lệ lỗi.
- Từng bước tách các workload nặng (ví dụ ingestion vector lớn) sang dịch vụ chuyên biệt khi quy mô tăng.

8.3.4 Định hướng nghiên cứu – ứng dụng

Về lâu dài, đề tài có thể mở rộng theo hướng nghiên cứu ứng dụng AI trong quản trị sinh viên:

- Dự báo xu hướng tham gia hoạt động theo nhóm đối tượng.
- Phát hiện bất thường trong dữ liệu đóng phí và dữ liệu điểm CTXH.
- Gợi ý hoạt động phù hợp theo hồ sơ tham gia của sinh viên.

Những định hướng này giúp hệ thống chuyển từ vai trò quản trị tác nghiệp sang nền tảng hỗ trợ ra quyết định, nâng cao giá trị sử dụng thực tế của cổng thông tin trong các giai đoạn tiếp theo.

Tài liệu tham khảo

1. NestJS (2025). *NestJS - A progressive Node.js framework*.
   `https://nestjs.com/` (Truy cập: 15/09/2025).

2. Meta Platforms, Inc. (2025). *React - A JavaScript library for building user interfaces*.
   `https://react.dev/` (Truy cập: 22/09/2025).

3. SheetJS LLC (2025). *SheetJS Community Edition - Spreadsheet Data Toolkit*.
   `https://sheetjs.com/` (Truy cập: 08/10/2025).

4. Edgar Hipp (2025). *Docxtemplater Documentation*.
   `https://docxtemplater.com/` (Truy cập: 25/10/2025).

5. Open XML Templating (2025). *PizZip - A library for handling zip files*.
   `https://github.com/open-xml-templating/pizzip` (Truy cập: 03/11/2025).

6. Shailja Gupta, Rajesh Ranjan, Surya Narayan Singh (2024). *A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions*.

7. Avthar Sewrathan (2023). *How to Build LLM Applications With Pgvector Vector Store in LangChain*.
   `https://www.tigerdata.com/blog/how-to-build-llm-applications-with-pgvector-vector-store-` (Truy cập: 20/10/2025).

8. langchainjs (2025). *LangChain.js GitHub Repository*.
   `https://github.com/langchain-ai/langchainjs` (Truy cập: 20/10/2025).

9. LangChain (2025). *PGVectorStore Documentation*.
   `https://docs.langchain.com/oss/python/integrations/vectorstores/pgvectorstore` (Truy cập: 20/10/2025).

10. TypeORM (2026). *TypeORM - ORM for TypeScript and JavaScript*.
    `https://typeorm.io/` (Truy cập: 21/04/2026).

11. PostgreSQL Global Development Group (2026). *PostgreSQL Documentation*.
    `https://www.postgresql.org/docs/` (Truy cập: 21/04/2026).

12. pgvector (2026). *pgvector - Open-source vector similarity search for Postgres*.
    `https://github.com/pgvector/pgvector` (Truy cập: 21/04/2026).

13. Docker, Inc. (2026). *Docker Documentation*.
    `https://docs.docker.com/` (Truy cập: 21/04/2026).

14. Prometheus Authors (2026). *Prometheus Documentation*.
    `https://prometheus.io/docs/` (Truy cập: 21/04/2026).

15. Grafana Labs (2026). *Grafana Documentation*.
    `https://grafana.com/docs/grafana/latest/` (Truy cập: 21/04/2026).

16. Vite Team (2026). *Vite Documentation*.
    `https://vite.dev/guide/` (Truy cập: 21/04/2026).

17. TanStack (2026). *TanStack Query Documentation*.
    `https://tanstack.com/query/latest` (Truy cập: 21/04/2026).

18. i18next (2026). *i18next Documentation*.
    `https://www.i18next.com/` (Truy cập: 21/04/2026).

19. Tailwind Labs (2026). *Tailwind CSS Documentation*. `https://tailwindcss.com/docs` (Truy cập: 21/04/2026).

20. OpenAPI Initiative (2026). *OpenAPI Specification*. `https://swagger.io/specification/` (Truy cập: 21/04/2026).

21. Auth0 (2026). *JSON Web Token Introduction*. `https://jwt.io/introduction` (Truy cập: 21/04/2026).